New Zealand Skills and Education College

School of Tech

Bachelor of Business Information Management

Cover Sheet and Student Declaration

Course Title:	**Capstone Project (DA)**	**Course code:**	BBIM713
Student Name:	**Gurjot Singh, Virender Singh, Lovepreet Singh**	**Student ID:**	**764707182**
Assessment No & Type:	Assessment 2, Research Project Report	**Cohort:**	**BBIM7124C**
Due Date:		**Date Submitted:**	
Tutor's Name:	**Sara Zandi**		
Assessment Weighting	65%		
Total Marks	100		

Student Declaration:

I declare that:

- I have read the New Zealand School of Education Ltd policies and regulations on assessments and understand what plagiarism is.
- I am aware of the penalties for cheating and plagiarism as laid down by the New Zealand School of Education Ltd.
- This is an original assessment and is entirely my own work.
- Where I have quoted or made use of the ideas of other writers, I have acknowledged the source.
- This assessment has been prepared exclusively for this course and has not been or will not be submitted as assessed work in any other course.
- It has been explained to me that this assessment may be used by NZSE Ltd, for internal and/or external moderation.

Student signature:
Date:

Assessor only to complete:		
Assessment results:		
	Total Marks: /100	
LO1 Requirements	**Met ☐** **Not Met ☐**	**Assessor Signature**
LO2 Requirements	**Met ☐** **Not Met ☐**	**Assessor Signature**
LO3 Requirements	**Met ☐** **Not Met ☐**	**Assessor Signature**

LO4 Requirements	**Met ☐** **Not Met ☐**	**Assessor Signature**
LO5 Requirements	**Met ☐** **Not Met ☐**	**Assessor Signature**

AI-DRIVEN AIR QUALITY ANALYSIS AND FORECASTING IN AUCKLAND CENTRE

Abstract

This capstone project created a Minimum Viable Product (MVP) for the AI analysis and short term air quality forecasting in Auckland Centre focusing on Queen Street. The project aimed to fill the business need of timely environmental intelligence for urban planning purposes, as well as communication and emission management for traffic, both for public health. Three different heterogeneous datasets were used for processing: ***traffic observations***, ***air-quality index data*** and ***pollutant observations***. Data engineering, exploratory data analysis, time-series feature engineering and supervised machine-learning models were used for implementation in Python. In this view, six models were developed: ***Random Forest Regression, Gradient Boosting Regression, Neural Network Regression (MLP), Extra Trees Regression, Histogram-Based Gradient Boosting Regression (Hist Gradient Boosting)*** and, ***Ridge Regression*** with a comparison between them based on chronological partitions of training, validation and testing. After temporal resampling, there were 171461 hourly observations available in the available dataset, but there was significant variability in target availability amongst pollutants. The final implementation included forecasting for $PM_{2.5}$ and NO_2 only, as O_3 was not available in the supplied data, and hence could not be modelled.

The walk-forward validation RMSE results for the revised data showed that the Extra Trees model was selected for $PM_{2.5}$ and NO_2. On the final holdout test, Extra Trees achieved an RMSE of 1.4076 and an R^2 of 0.7829 for $PM_{2.5}$, while its RMSE was 3.9392 and R^2 was 0.6456 for NO_2. It is important to note that the best validation model does not necessarily equate to the best holdout model (as seen in the cases of NO_2 with Hist Gradient Boosting having the lowest validation model with the highest holdout model). The updated pipeline also yielded recursive 24-hour forecasts, with mean predicted concentrations of ~11.32 for $PM_{2.5}$ and ~35.18 for NO_2. Target leakage was taken into account by removing same-time measurements of targets from the predictors and using time-lags in pollutant measurements. The project also created an executive dashboard in Power BI to investigate model comparisons, trends in forecasts, and actual versus predicted observations. However, there is variation in 24H backtesting performance and the need for dashboard KPI aggregation corrections as well as future weather and traffic inputs, and dashboard testing prior to deployment.

Keywords: Artificial Intelligence, Machine Learning, Air Quality Forecasting, Time-Series Analysis, Data Mining, Auckland

TABLE OF CONTENTS

Abstract 4

1. Introduction 8

1.1 Project Context 8

1.2 Business Problem and Rationale 8

1.3 Data and Analytical Challenges 9

1.4 MVP Scope and Key Limitations 10

1.5 Project Team and Report Structure 11

2. Project Questions 11

3. Project Design 12

3.1 Overall Solution Architecture 12

3.2 Data Sources and Ingestion 13

3.3 Data Quality and Missing-Data Strategy 14

3.4 Feature Engineering 17

3.5 Machine-Learning Design 19

3.6 Evaluation Design 23

3.7 Explainability and Decision Support 24

3.8 Security, Compliance and Ethical Design 26

4. Project Development 26

4.1 Implementation 26

4.2 Decision-Support Outputs 37

4.3 Testing and Evaluation 41

4.4 Ethical Evaluation 43

4.5 Maintenance and Scaling 43

5. Individual Discussion 43

6. Conclusion and Future Directions 45

6.1 Conclusion 45

6.2 Practical Implications 45

6.3 Future Directions 46

7. Projects and GitHub 47

8. References 47

9. Appendices 48

1. Introduction

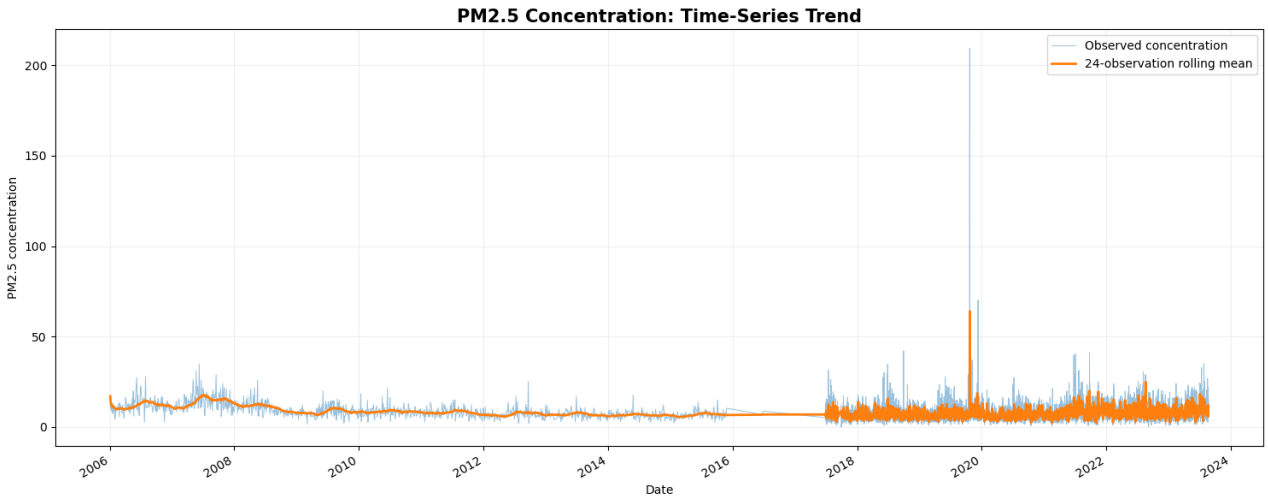
1.1 Project Context

Urban air quality is an environmental and business problem that is complex, as pollutant levels change over time as a result of inter-dependent meteorological, traffic and temporal factors. Auckland Centre is of particular interest as it represents commercial activity, pedestrian movement, road traffic and dense urban development (Chuang & Chen, 2025). Hence, reliable short-term forecasts can help organizations making decisions on environmental management by giving them a heads up on potential environmental conditions rather than just relying on past or current measurements.

To build a Minimum Viable Product (MVP) with the help of artificial intelligence (AI) and machine-learning techniques to predict urban air pollutants up to 24 hours in advance. It was designed for Auckland Council, where timely air-quality information may be useful for environmental monitoring purposes, public advisories, and traffic-related decision-making.

1.2 Business Problem and Rationale

The main challenge in the central business is the challenge of converting huge volumes of heterogeneous traffic and air quality observations into meaningful, actionable information. The traditional monitoring offers measurements of existing conditions, but is not able to give a good indication of how the concentrations of pollutants might change in the future.



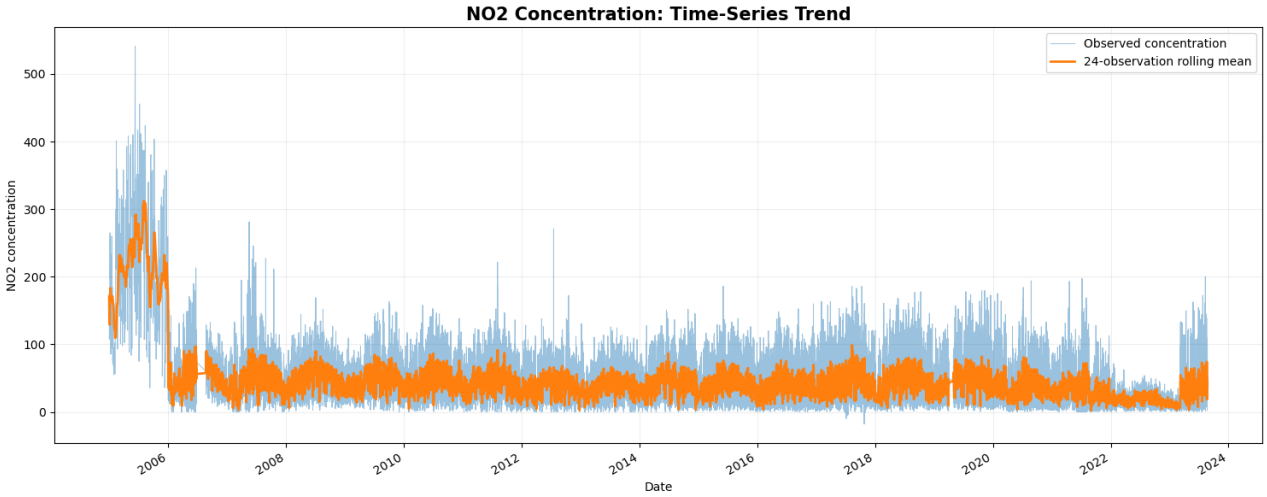


Figure 1: Concentration of pollutant

A forecasting-based analytical system would be helpful for decision-making in order to take necessary actions to meet the changes expected.

The project thus responded to the following questions:

1. What are some ways to transform traffic, air quality and environmental data into a unified analytical dataset?
2. What are some of the important temporal and environmental factors that give information for short-term pollutant forecasting?
3. What is the accuracy of various machine-learning models predicting the $PM_{2.5}$ and NO_2 levels?
4. How to convert model outputs to user-friendly decision support information for intended user?

The project was designed to build an end-to-end analytical pipeline from data preparation, exploratory data analysis, feature engineering, machine learning development, evaluation and 24-hour forecasting. The broader goal was to show the potential of predictive analytics in real-world decisions for the environment and traffic management.

1.3 Data and Analytical Challenges

The implementation involved three datasets:

1. 45528 traffic records,

2. 877795 AQI observations and
3. 245955 pollution observations.

These datasets were not structured similarly and were not of the same size, which meant that they needed to be systematically cleaned and transformed. For traffic data, all "timestamps" were standardized and converted to hourly observations, and for air-quality data, all data were converted from "long" to "wide" form so that the pollutants and the environmental variables could be analysed together.

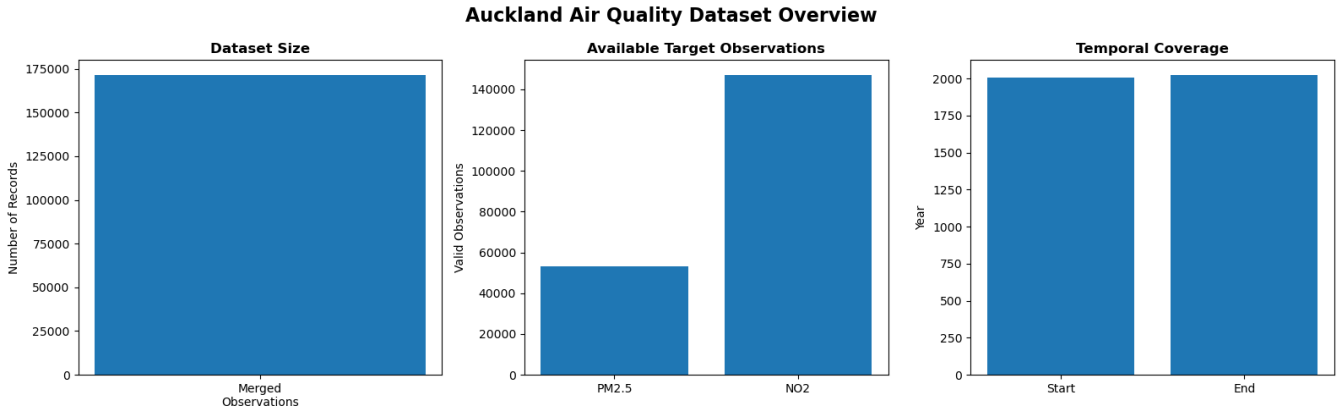


Figure 2: Dataset Overview

The datasets were then merged on the basis of the time-stamps, and have been interpolated and forward or backward filled for missing numerical observations, to a certain extent.

Pollutant distribution, temporal pollutant patterns, correlations and relationships between traffic and pollutant concentrations were explored. The time-series features were then generated such as lagged values, rolling statistics, calendar variables and cyclic time representations. ***Random Forest Regression***, ***Gradient Boosting Regression*** and a feed-forward ***Neural Network*** were developed, and compared.

1.4 MVP Scope and Key Limitations

The end-to-end air-quality forecasting pipeline for $PM_{2.5}$ and NO_2 is provided by the MVP. O_3 was not included due to lack of data. The new implementation overcomes the target leakage problem by removing the same target-pollutant from the predictor set and by incorporating historical lag features. Chronological walk-forward validation and an independent last test period were used to compare six regression models. This enhances the reliability of the evaluation but, through 24 hour back testing, their

predictive performance can be seen to be different for various forecast periods. But the future outlook for weather and traffic is also uncertain and this is still a prototype, not a proven operational weather forecasting service.

1.5 Project Team and Report Structure

The project has been developed in partnership. Gurjot Singh helped with literature review, theoretical development and academic foundation. Lovepreet Singh helped with data collection, cleaning and pre-processing and Virender Singh helped with the development of AI models, evaluation, interpretation, business implications and recommendations. In particular, the technical implementation of data-processing, machine learning and implementation in the MVP is reported.

The rest of the report covers questions from the project, solution design, implementation, testing and evaluation, ethical issues, reflections by the individual, and conclusions and future development plans.

2. Project Questions

The overall question that guided the project was:

How to develop and apply AI-based models to understand and forecast air pollution levels throughout Queen Street, Auckland Centre, to inform proactive and smart planning and public health interventions?

This general question was broken down into specific technical and business needs in three supporting questions:

RQ1: What practical steps can be taken to integrate and preprocess the various environmental datasets, such as traffic data, meteorology and sensor signals, to improve the accuracy of air quality forecasts?

This question is related to data-integration problem. The datasets provided were in different structures, time and measurement systems. Prior to the modelling, the timestamps had to be standardized, numerical data had to be converted, duplicate records deleted, pollutants had to be normalized, and the data had to be transformed into the common hourly analytical structure (Martínez-García & Hernández-

Lemus, 2022). Traffic data had to be harmonized with air quality observations as well. The question raised here is whether or not data from different sources can be converted into a common analytic basis.

RQ2: What are some helpful performative AI techniques that are at the same time sufficiently interpretable for decision makers in order to predict changes in air quality?

To compare, six regression models were used: Random Forest, Gradient Boosting, Neural Network, Extra Trees, Hist Gradient Boosting and Ridge Regression. The use of historical pollutant lags, not contemporaneous target values, was the solution chosen for leakages to the target. Five-fold walk-forward validation and a separate chronological test period were employed to evaluate the performance using the metrics of MAE, RMSE and R^2. For interpretation, the feature importance was explored.

RQ3: What is the best way to convey 24-hour pollutant forecasts as decision-support information to environmental and traffic-management stakeholders?

Forecast and model-evaluation datasets were created in Python, and then incorporated into a Power BI executive dashboard. The dashboard shows model comparisons, model trend, actual vs. predicted observations and a model-selection slicer. It is designed to help the exploration of analytical results and communication. Before being used operationally, dashboard measurements and the filtering behaviour need to be validated.

3. Project Design

3.1 Overall Solution Architecture

The MVP was built to be an end-to-end data analytics pipeline:

Data Sources → Data Ingestion → Cleaning and Integration → Hourly Resampling → Feature Engineering → Exploratory or Data Mining Analysis → ML Training → Validation and Testing → Model Selection → 24-Hour Forecasting → Visualisation and Decision Dashboard → Future Feedback and Retraining

3.2 Data Sources and Ingestion

```
traffic = pd.read_excel(TRAFFIC_PATH)
aqi = pd.read_excel(AIR_AQI_PATH)
pollution = pd.read_excel(AIR_POLLUTION_PATH)

print("Traffic shape:", traffic.shape)
print("AQI shape:", aqi.shape)
print("Pollution shape:", pollution.shape)
```

```
Traffic shape: (45528, 14)
AQI shape: (877795, 5)
Pollution shape: (245955, 5)
```

Figure 3: Datasets used

The primary sources of data were three workbooks in Excel format. The traffic data had 45528 rows and 14 columns. There were 877795 rows and 5 columns in AQI dataset and 245955 rows and 5 columns in pollution dataset.

	Site	Date	Time	Parameter	Value
0	QUS	2006-01-01	00:00:00	AQI	25.0
1	QUS	2006-01-01	01:00:00	AQI	26.0
2	QUS	2006-01-01	02:00:00	AQI	19.0
3	QUS	2006-01-01	03:00:00	AQI	18.0
4	QUS	2006-01-01	04:00:00	AQI	24.0
...	...	...	...	...	...
877790	QUS	2023-03-27	23:00:00	AQI	12.0
877791	QUS	2023-03-28	00:00:00	AQI	11.0
877792	QUS	2023-03-28	01:00:00	AQI	6.0
877793	QUS	2023-03-28	02:00:00	AQI	7.0
877794	QUS	2023-03-28	03:00:00	AQI	8.0

877795 rows × 5 columns

Figure 4: AQI Dataset

The air-quality data was presented in a long format with site, date, time, parameter and value columns. A reusables' preparation process changed date, time to single datetime field, and changed measurements to numbers. All parameters were standardized to ensure consistency between datasets.

	calendar_date	time_hh24	2007-1	2007-2	2007-3	2007-4	2007-5	2007-6	2007-7	2007-8	2007-9	2007-10	2007-11	Unnamed: 13
0	2018-01-01	0.0	72	98	68	139	88	36	38	37	92	103	141	912
1	2018-01-01	1.0	94	72	95	78	21	34	29	41	56	62	69	651
2	2018-01-01	2.0	157	159	170	117	54	40	44	63	123	170	122	1219
3	2018-01-01	3.0	192	192	151	177	101	50	55	86	118	179	184	1485
4	2018-01-01	4.0	155	186	108	168	105	47	47	62	125	93	173	1269
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
45523	NaT	NaN	0	78	151	156	178	0	93	0	80	0	47	783
45524	NaT	NaN	0	78	157	99	118	0	76	0	65	0	66	659
45525	NaT	NaN	0	100	154	154	180	0	82	0	105	0	56	831
45526	NaT	NaN	0	72	166	116	139	0	68	0	83	0	78	722
45527	NaT	NaN	0	101	147	147	165	0	76	0	108	0	67	811

45528 rows × 14 columns

Figure 5: Traffic Dataset

Traffic information was recoded as a datetime data type with calendar date and hourly time. Where available the total traffic field was used as supplied. The traffic observations thus obtained were then segmented by time and sorted by time.

The air quality and traffic datasets were then merged by a timestamp based method. Traffic observations were correlated with environmental observations using a nearest time approach with a one-hour tolerance. This combined data was then resampled to an hourly timescale.

3.3 Data Quality and Missing-Data Strategy

The data quality stage was crucial due to the differences in the time periods represented in the project data, as well as the varying degrees of availability of the pollutants. Project notes indicate that year-to-year pollutant coverage was significantly different as well. For instance, the coverage of $PM_{2.5}$ was very low in 2012 and 2016, and much greater in the following years. This variability should be taken into account when interpreting model results, since the data set may not be complete.

The implementation eliminated invalid timestamps and non-numeric measurements and eliminated exact duplicate parameter or timestamp records. The numerical variables were limitedly interpolated and then restrictedly filled forward and backward. The limits were intentionally not extended to any gaps, thereby minimizing the potential for the creation of synthetic observations over extended periods.

Descriptive statistics:

	datetime	AQI	CITY_CENTRE_TVCOUNT	NO
count	171461	145204.000000	45852.000000	151794.000000
mean	2013-11-11 02:00:00	25.714256	20225.122795	55.882255
min	2004-01-31 00:00:00	-32.000000	390.000000	-1.700000
25%	2008-12-21 01:00:00	16.000000	7702.750000	14.497537
50%	2013-11-11 02:00:00	24.000000	20473.500000	37.400000
75%	2018-10-02 03:00:00	34.000000	30971.250000	78.172943
max	2023-08-23 04:00:00	697.000000	51393.000000	792.090000
std	NaN	16.221047	12813.598665	57.573809

	NO2	NOx	PM10	PM2.5
count	158648.000000	155954.000000	122469.000000	74588.000000
mean	47.590662	150.127758	17.494280	7.750319
min	-17.800000	-9.900000	0.000000	0.000000
25%	23.004942	52.900000	12.858333	5.400000
50%	37.806115	113.800000	16.500000	7.000000
75%	56.760000	201.147667	21.000000	9.300000
max	540.962500	4334.414712	386.900000	209.400000
std	46.529490	142.935501	7.281462	3.808623

	RH	TEMP	TOTAL_PEDESTRIANS	TRAFFICV
count	23698.000000	57500.000000	46050.000000	45816.000000
mean	65.303089	16.794335	5994.811852	674.198955
min	27.800000	0.000000	0.000000	0.000000
25%	56.800000	14.000000	1262.000000	401.000000
50%	66.000000	17.000000	4812.500000	663.000000
75%	74.700000	20.000000	9579.000000	936.000000
max	89.300000	29.000000	26634.000000	1995.000000
std	11.838942	4.044614	5234.007104	342.304661

	WD	WS	traffic_volume
count	49963.000000	49646.000000	45084.000000
mean	196.065317	0.522919	676.544743
min	0.000000	0.000000	0.000000
25%	156.000000	0.300000	402.000000
50%	189.000000	0.500000	663.500000
75%	236.000000	0.600000	945.000000
max	354.000000	52.800000	1995.000000
std	74.959731	0.431446	343.307478

Figure 6: Descriptive statistics of Merged Dataset

The data for each hour is a database of 171461 observations after temporal processing. Availability of individual variables was still uneven, though. The final resampled dataset contained 74588 (NO_2

contained 158648) non-missing observations for $PM_{2.5}$ (NO_2). This difference is the reason for the need for pollutant specific modelling.

3.4 Feature Engineering

The cleaned data was then structured into a time-series modelling structure through the process of feature engineering. The calendar had hour, day, day of week, month and quarter features. The hour and month were represented cyclically by making use of sine and cosine representations. These variables help to represent the circularity of time, such as 23:00 and 00:00 are adjacent in time, yet the numbers are significantly different.

```python
def make_model_table(source, target):
    d = source.sort_values("datetime").copy()
    d = d.drop_duplicates("datetime", keep="last")
    d = d.reset_index(drop=True)

    # Use a regular hourly time grid.
    d = d.set_index("datetime").asfreq("h").reset_index()

    # Causal filling of explanatory variables only.
    other_targets = [
        t for t in ["PM2.5", "NO2", "O3"]
        if t in d.columns
    ]

    numeric_cols = d.select_dtypes(
        include=np.number
    ).columns.tolist()

    exog_cols = [
        c for c in numeric_cols
        if c not in other_targets
    ]

    d[exog_cols] = d[exog_cols].ffill(limit=6)

    # Calendar features are known in advance.
    d["hour"] = d["datetime"].dt.hour
    d["dayofweek"] = d["datetime"].dt.dayofweek
    d["month"] = d["datetime"].dt.month
    d["dayofyear"] = d["datetime"].dt.dayofyear

    d["hour_sin"] = np.sin(2 * np.pi * d["hour"] / 24)
    d["hour_cos"] = np.cos(2 * np.pi * d["hour"] / 24)
    d["doy_sin"] = np.sin(
        2 * np.pi * d["dayofyear"] / 365.25
    )
    d["doy_cos"] = np.cos(
        2 * np.pi * d["dayofyear"] / 365.25
    )

    feature_cols = [
        "hour", "dayofweek", "month", "dayofyear",
        "hour_sin", "hour_cos", "doy_sin", "doy_cos"
    ]
```

Figure 7: Create Time-series Features

Lag variables were generated at 1, 3, 6, 12, 24, 48 and 72 hours. The rolling means and standard deviations were calculated with 3, 6, 12 and 24-hour periods. Additional variables such as traffic lag and 24-hour rolling traffic were also included.

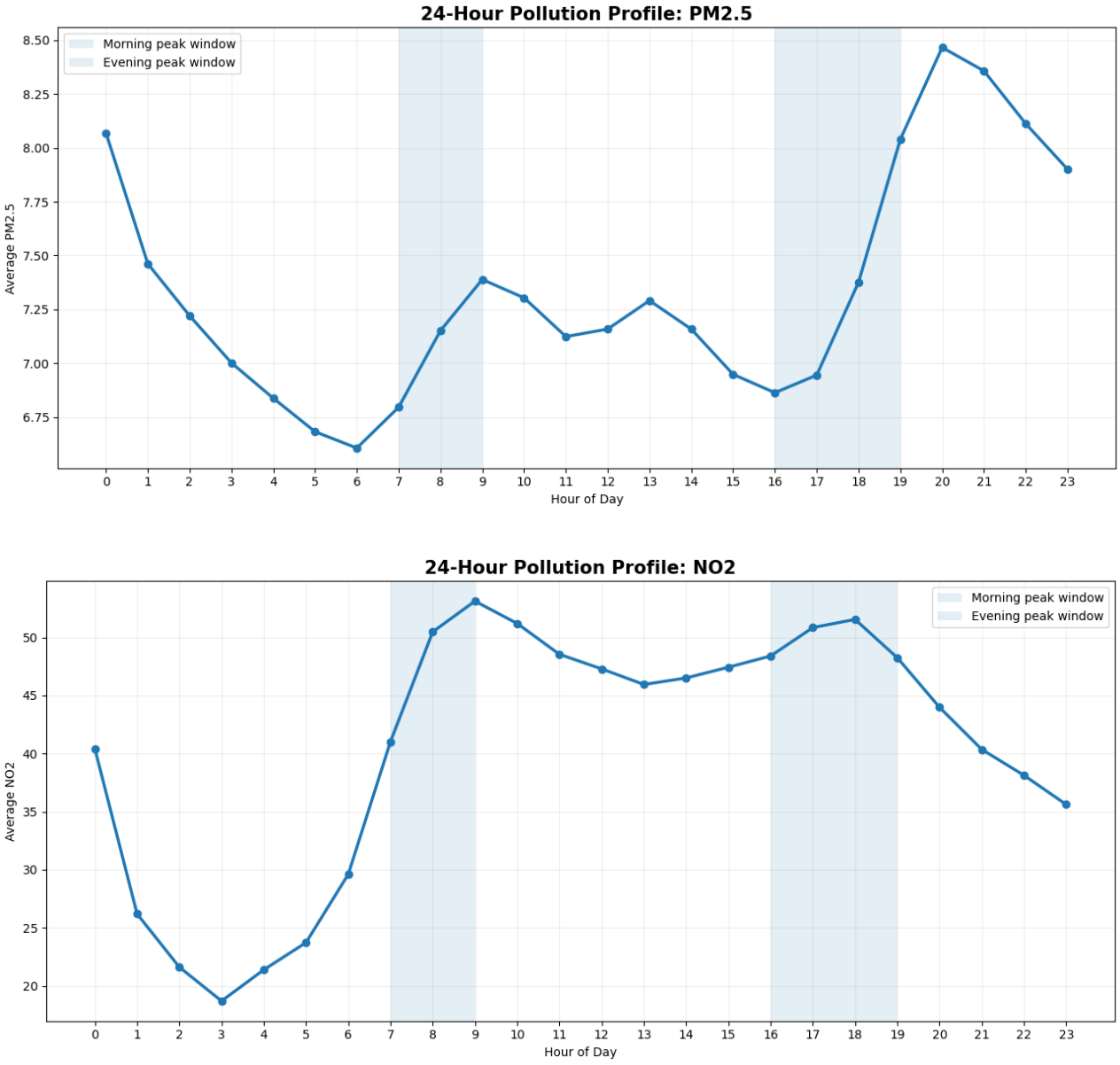


Figure 8: 24-hour Pollution Profile for Pollutant

These features enable data mining with the model being able to learn temporal persistence, short-term variations, daily cycles and traffic-pollution relationships.

3.5 Machine-Learning Design

The regression model that were applied were six supervised models namely, Random Forest, Gradient Boosting, Extra Trees, Neural Network (MLP), Hist Gradient Boosting, Ridge Regression. The ensemble tree models represent the nonlinear relationship while Ridge Regression represents the linear baseline.

In the updated configuration, the Neural Network is standardised via a standardisation pipeline and early stopping is turned off.

```
all_test_results = []
all_cv_results = []
all_predictions = []
model_objects = {}

for target in TARGETS:

    print(f"\n========== {target} ==========")

    model_table, features = make_model_table(data, target)

    X = model_table[features]
    y = model_table[target]
    timestamps = model_table["datetime"]

    n = len(model_table)

    if n < 100:
        print(f"Insufficient modelling rows for {target}: {n}")
        continue

    # Final 15% is reserved as untouched test data.
    test_start = int(n * 0.85)

    X_dev, y_dev = X.iloc[:test_start], y.iloc[:test_start]
    X_test, y_test = X.iloc[test_start:], y.iloc[test_start:]

    time_test = timestamps.iloc[test_start:]

    # Walk-forward validation uses only development data.
    splitter = TimeSeriesSplit(n_splits=5)
    cv_scores = []
    models = build_models()

    for model_name, prototype in models.items():

        fold_number = 0

        for train_idx, val_idx in splitter.split(X_dev):

            fold_number += 1
            model = clone(prototype)

            X_train_fold = X_dev.iloc[train_idx]
            y_train_fold = y_dev.iloc[train_idx]
            X_val_fold = X_dev.iloc[val_idx]
            y_val_fold = y_dev.iloc[val_idx]

            model.fit(X_train_fold, y_train_fold)
            val_pred = model.predict(X_val_fold)

            metrics = calculate_metrics(y_val_fold, val_pred)

            cv_scores.append({
                "Pollutant": target,
                "Model": model_name,
                "Fold": fold_number,
                **metrics
            })

    cv_df = pd.DataFrame(cv_scores)
    all_cv_results.append(cv_df)

    # Select using mean walk-forward RMSE only.
    cv_summary = (
        cv_df.groupby(["Pollutant", "Model"], as_index=False)
        [["MAE", "RMSE", "R2"]]
        .mean()
        .rename(columns={
            "MAE": "CV_MAE",
            "RMSE": "CV_RMSE",
            "R2": "CV_R2"
        })
```

Figure 9: Model Training Function

The value of $PM_{2.5}$ and NO_2 were not included in the predictor set so as to avoid target leakage. The pollutants and the explanatory variables were lagged as appropriate, calendar features were used when available, and historical pollutant lags were used. Explanatory variables were forward-filled only to a maximum of 6 hours, and variables where the target variable was not observed or not robust enough were removed.

Data was segmented according to the chronological order. The final 15% of eligible observations were set aside for final test, the prior 85% for development and 5-fold "walk forward" test. The mean validation RMSE was used as a metric for model selection. The chosen model was then retrofitted to the available historical data to be used for future predictions.

Rationale for Model Selection

The six supervised regression models were chosen for comparison to predict the $PM_{2.5}$ and NO_2 concentrations from the integrated Auckland Centre air-quality, traffic and environmental data. The data consists of time-dependent observations, relationships between the pollutants that are nonlinear, traffic measurements, meteorological variables and historical pollutant values. Thus, a better basis for choosing appropriate forecasting procedures is to make comparisons between models that use different learning mechanisms than with a single algorithm.

Random Forest Regression was chosen because it takes into account the predictions from each of the decision trees and has the potential of capturing the nonlinear relationship between traffic, environmental conditions and pollutant concentrations. It can be used in data sets that have interactions among multiple explanatory variables and is fairly robust to complex feature relationships.

Gradient Boosting Regression was added since it develops a tree ensemble sequentially where the next tree tries to correct errors of the previous tree ensemble. This can be used to identify complex relationships and allow for more accurate predictions to be made if pollutant concentrations are reliant on interacting temporal and environmental factors.

Another tree-based ensemble approach, namely Extra Trees Regression, was chosen. It adds extra randomness to the creation of the decision trees, and can decrease variance and boost generalisation.

Its performance was especially relevant due to the fact that the data set is composed of many engineered predictors such as historical pollutant lags, rolling statistics and traffic related variables.

Hist Gradient Boosting Regression was added as an efficient boosting algorithm with histogram-based feature binning. Can be used to model non-linear patterns and interactions, and provides computational benefits on relatively large data sets. This is why boosting was chosen to be compared with Random Forest, Gradient Boosting and Extra Trees.

A Multilayer Perceptron (MLP) neural network regression was chosen to explore the ability of a feed-forward neural network to learn complex nonlinear relationships between the engineered temporal, traffic and environmental predictors. A preprocessing pipeline was used to handle the differences in feature scales that are sensitive to neural networks, as a result of which feature standardisation was added. Its performance was not assumed to be superior but rather compared with the tree-based models.

Ridge Regression was added as a baseline model in the form of a regularised linear model. It approximates linear relationships between the predictors and the concentrations of the pollutants and uses an L2 penalty term to minimise the impact of large coefficients. This model is useful as a reference to evaluate if a more complicated nonlinear model can improve the understanding of a problem, compared to a simpler one.

The six models were compared separately for $PM_{2.5}$ and NO_2 due to different data availability, concentration patterns and traffic and environmental associations of the pollutants. To minimize the leakage into the target, the current target was not included in the predictor set, and the lagged targets were used if applicable.

3.6 Evaluation Design

```
fitted_models = {}

for model_name, prototype in models.items():

    model = clone(prototype)
    model.fit(X_dev, y_dev)

    pred = model.predict(X_test)
    metrics = calculate_metrics(y_test, pred)

    all_test_results.append({
        "Pollutant": target,
        "Model": model_name,
        "Train_Observations": len(X_dev),
        "Test_Observations": len(X_test),
        "Test_Start": time_test.iloc[0],
        "Test_End": time_test.iloc[-1],
        **metrics
    })

    prediction_rows = pd.DataFrame({
        "datetime": time_test.values,
        "Pollutant": target,
        "Model": model_name,
        "Actual": y_test.values,
        "Predicted": pred
    })

    all_predictions.append(prediction_rows)
    fitted_models[model_name] = model

# Refit selected model on all available eligible observations
# for use in the future forecast only.
final_model = clone(models[best_name])
final_model.fit(X, y)

model_objects[target] = {
    "model": final_model,
    "model_name": best_name,
    "features": features,
    "history": data.sort_values("datetime").copy(),
    "model_table": model_table
}

print(cv_summary.sort_values("CV_RMSE").to_string(index=False))
```

Figure 10: Model Evaluation Design

The Mean Absolute Error (MAE), Root Mean Squared Error (RMSE) and R^2 were used to measure the model performance. The smaller the MAE and RMSE, the smaller the error in making the prediction and the higher the R^2, the more variation is explained in relation to a baseline prediction based on the mean (Piotrowski *et al.,* 2022).

The development period has been split into five folds by `TimeSeriesSplit` with chronological order. For each pollutant the model having the lowest mean validation RMSE was chosen. All six models were then tested on a same (and untested) final chronological holdout period.

Forecasts were tested with a separate 24-hour walk-forward test, using various historical forecast origins. This extra test evaluated multi-step forecasting actions instead of one-step test forecasts. Note that negative R^2 values for individual backtest windows mean that some predictions fared worse than.

3.7 Explainability and Decision Support

The updated workflow enables interpretation by means of the temporal feature design, comparison of models and analysis of available outputs of feature-importance. Unfortunately, the notebook you uploaded was not able to provide SHAP explanations for the selected Extra Trees models since the model type was not supported in the SHAP section of the notebook. SHAP explanations are thus an ongoing effort rather than a final product of the new implementation.

```
if SHAP_AVAILABLE:

    for target, obj in model_objects.items():

        model = obj["best_model"]
        features = obj["features"]
        model_data = obj["data"]

        # SHAP TreeExplainer for tree-based models
        if isinstance(
            model,
            (
                RandomForestRegressor,
                GradientBoostingRegressor
            )
        ):

            X_sample = model_data[
                features
            ].tail(500)

            explainer = shap.TreeExplainer(
                model
            )

            shap_values = explainer.shap_values(
                X_sample
            )

            plt.figure()

            shap.summary_plot(
                shap_values,
                X_sample,
                show=False
            )

            plt.tight_layout()

            plt.savefig(
                os.path.join(
                    OUTPUT_DIR,
                    f"{target}_SHAP.png"
                ),
                dpi=300,
                bbox_inches="tight"
            )

            plt.show()

else:

    print(
        "\nSHAP is not installed. "
        "Install it with: pip install shap"
    )
```

Figure 11: SHAP Explainability

It should be noted that the feature importance does not imply causation. Additional research should employ a suitable explainability technique for the models used and be sure to confirm that the explanations match the forecasting design.

3.8 Security, Compliance and Ethical Design

The project does not seek to actively gather individual level personal data, but rather relies on observations of the environment and of traffic. However, ethical handling is required as it is possible that environmental and mobility data may be sensitive if merged with other data.

The Government Data Strategy for Aotearoa New Zealand highlights practices that build trust in data, data sharing and privacy, data quality, and data accountability and responsibilities towards Te Tiriti o Waitangi and Māori data sovereignty (New Zealand Government Chief Data Steward, 2021).

The project should therefore adhere to the appropriate access controls, only keep necessary datasets, and follow the client's instructions in terms of data return and deletion at the end of the project, documenting data provenance. The Privacy Act 2020 and its information privacy principles would then directly apply, such as the purpose and collection, security and retention and disclosure requirements (Office of the Privacy Commissioner, 2026).

The project also acknowledges the need for consideration of Māori interests and meaningful engagement to ensure that data and decisions are not impacting on Māori communities. Ngā Tikanga Paihere includes guidance to ensure culturally appropriate and responsible data practice, which includes accountability, community participation and consideration of positive and negative consequences (New Zealand Government Chief Data Steward, 2020).

4. Project Development

4.1 Implementation

The development stage turned the proposed analytical architecture into an working MVP which included data ingestion, cleaning, integration, exploratory analysis, model training, model evaluation and forecasting, and data visualisation.

Dataset Loading and Initial Inspection

Three excel files were loaded and checked to find out their size and shape. There were 45528 records in the traffic dataset, 877795 records in the AQI dataset and 245,55 records in the pollution dataset.

The various structures and sizes indicated that systematic data preparation prior to modelling was required.

Traffic Data Preparation

```
traffic["datetime"] = (
    pd.to_datetime(traffic["calendar_date"], errors="coerce")
    + pd.to_timedelta(traffic["time_hh24"], unit="h")
)

traffic = traffic.dropna(subset=["datetime"]).copy()

# Use the supplied total traffic column where available
if "Unnamed: 13" in traffic.columns:
    traffic["traffic_volume"] = pd.to_numeric(
        traffic["Unnamed: 13"], errors="coerce"
    )
else:
    # Otherwise sum available traffic columns
    traffic_cols = [
        c for c in traffic.columns
        if c not in ["calendar_date", "time_hh24", "datetime"]
    ]
    traffic["traffic_volume"] = traffic[traffic_cols].apply(
        pd.to_numeric, errors="coerce"
    ).sum(axis=1)

traffic = traffic[["datetime", "traffic_volume"]]

traffic = (
    traffic.groupby("datetime", as_index=False)["traffic_volume"]
    .mean()
    .sort_values("datetime")
)

print("\nTraffic data:")
print(traffic.head())
```

```
Traffic data:
             datetime  traffic_volume
0 2018-01-01 00:00:00           912.0
1 2018-01-01 01:00:00           651.0
2 2018-01-01 02:00:00          1219.0
3 2018-01-01 03:00:00          1485.0
4 2018-01-01 04:00:00          1269.0
```

Figure 12: Clean Traffic Data

The traffic dates and hourly times were merged together into one datetime field. Records with invalid data were deleted and traffic was grouped by datetime. This resulted in a long time series of hourly traffic data available for use with air-quality measurements.

Air-Quality Data Transformation

```python
def prepare_air_data(df):
    data = df.copy()

    data["Date"] = pd.to_datetime(data["Date"], errors="coerce")
    data["Time"] = pd.to_timedelta(data["Time"].astype(str), errors="coerce")

    data["datetime"] = data["Date"] + data["Time"]

    data["Value"] = pd.to_numeric(data["Value"], errors="coerce")

    data["Parameter"] = (
        data["Parameter"]
        .astype(str)
        .str.strip()
        .str.upper()
    )

    data = data.dropna(
        subset=["datetime", "Value", "Parameter"]
    )

    return data[["datetime", "Parameter", "Value"]]


aqi_clean = prepare_air_data(aqi)
pollution_clean = prepare_air_data(pollution)

air = pd.concat(
    [aqi_clean, pollution_clean],
    ignore_index=True
)
```

```python
# Remove exact duplicate records
air = air.drop_duplicates(
    subset=["datetime", "Parameter"],
    keep="first"
)
```

```python
# Convert long format into columns
air = air.pivot_table(
    index="datetime",
    columns="Parameter",
    values="Value",
    aggfunc="mean"
).reset_index()

air.columns.name = None

air = air.sort_values("datetime")

print("\nAvailable air-quality variables:")
print([c for c in air.columns if c != "datetime"])
```

```
Available air-quality variables:
['AQI', 'CITY_CENTRE_TVCOUNT', 'NO', 'NO2', 'NOX', 'PM10', 'PM2.5', 'RH', 'TEMP', 'TOTAL_PEDESTRIANS', 'TRAFFICV', 'WD', 'WS']
```

Figure 13: Air Quality Data Cleaning and Transformation

Common pre-processing of AQI and pollution data were applied. The date and time values have been converted to datetime, measurement values to numeric values, and parameter names have been normalised. All datasets were merged together, data with duplicate dates from the datetime-parameter were deleted and long-format data was transformed into variables.

```
data = pd.merge_asof(
    air.sort_values("datetime"),
    traffic.sort_values("datetime"),
    on="datetime",
    direction="nearest",
    tolerance=pd.Timedelta("1 hour")
)

data = data.sort_values("datetime")

print("\nMerged dataset:")
print(data.head())
print("Shape:", data.shape)
```

```
Merged dataset:
    datetime  AQI  CITY_CENTRE_TVCOUNT  NO  NO2         NOx  PM10  PM2.5  RH  \
0 2004-01-31  NaN                  NaN NaN  NaN  208.021944   NaN    NaN NaN
1 2004-02-01  NaN                  NaN NaN  NaN  170.431389   NaN    NaN NaN
2 2004-02-02  NaN                  NaN NaN  NaN  238.845556   NaN    NaN NaN
3 2004-02-03  NaN                  NaN NaN  NaN  167.775556   NaN    NaN NaN
4 2004-02-04  NaN                  NaN NaN  NaN  162.131111   NaN    NaN NaN

   TEMP  TOTAL_PEDESTRIANS  TRAFFICV  WD  WS  traffic_volume
0   NaN                NaN       NaN NaN NaN             NaN
1   NaN                NaN       NaN NaN NaN             NaN
2   NaN                NaN       NaN NaN NaN             NaN
3   NaN                NaN       NaN NaN NaN             NaN
4   NaN                NaN       NaN NaN NaN             NaN
Shape: (155603, 15)
```

Figure 14: Merged Datasets

The final dataset consisted of AQI, NO, NO2, NOx, PM10, PM2.5, RH, TEMP, TOTAL_PEDESTRIANS, TRAFFICV, WD, WS and CITY_CENTRE_TVCOUNT.

```
rename_map = {}

for col in air.columns:
    clean = col.replace("_", "").replace(".", "")

    if clean == "PM25":
        rename_map[col] = "PM2.5"

    elif clean == "PM10":
        rename_map[col] = "PM10"

    elif clean == "NO2":
        rename_map[col] = "NO2"

    elif clean == "O3":
        rename_map[col] = "O3"

    elif clean == "NOX":
        rename_map[col] = "NOx"

    elif clean == "NO":
        rename_map[col] = "NO"

    elif clean == "AQI":
        rename_map[col] = "AQI"

air = air.rename(columns=rename_map)

print("\nStandardised variables:")
print(air.columns.tolist())
```

```
Standardised variables:
['datetime', 'AQI', 'CITY_CENTRE_TVCOUNT', 'NO', 'NO2', 'NOx', 'PM10', 'PM2.5', 'RH', 'TEMP', 'TOTAL_PEDESTRIANS', 'TRAFFICV', 'WD', 'WS']
```

Figure 15: Standardised Pollutants Name

A target for O_3 was not available. In view of the limitations, therefore, the available pollutants, $PM_{2.5}$ and NO_2, have been selected as the target pollutants instead of constructing missing observations.

Exploratory Data Mining

```
Correlation matrix:
                              AQI  CITY_CENTRE_TVCOUNT        NO       NO2  \
AQI                      1.000000             0.147711  0.501967  0.655124
CITY_CENTRE_TVCOUNT      0.147711             1.000000  0.495974  0.527074
NO                       0.501967             0.495974  1.000000  0.805883
NO2                      0.655124             0.527074  0.805883  1.000000
NOx                      0.252035             0.527761  0.622812  0.549852
PM10                     0.259823             0.149382  0.215480  0.193442
PM2.5                    0.313627             0.001597  0.152958  0.126960
RH                      -0.108885            -0.370731 -0.083509 -0.129975
TEMP                     0.039956             0.239408  0.017990 -0.105006
TOTAL_PEDESTRIANS        0.138002             0.894863  0.435060  0.487276
TRAFFICV                 0.229438             0.731762  0.428769  0.522105
WD                      -0.002961             0.055066 -0.172695 -0.134448
WS                      -0.026338             0.125728 -0.111132 -0.099635
traffic_volume           0.222811             0.734705  0.431860  0.523750

                              NOx      PM10      PM2.5        RH      TEMP  \
AQI                      0.252035  0.259823  0.313627 -0.108885  0.039956
CITY_CENTRE_TVCOUNT      0.527761  0.149382  0.001597 -0.370731  0.239408
NO                       0.622812  0.215480  0.152958 -0.083509  0.017990
NO2                      0.549852  0.193442  0.126960 -0.129975 -0.105006
NOx                      1.000000  0.138105  0.152454 -0.104392 -0.122088
PM10                     0.138105  1.000000  0.833254 -0.052873  0.082482
PM2.5                    0.152454  0.833254  1.000000  0.117993 -0.026488
RH                      -0.104392 -0.052873  0.117993  1.000000 -0.287520
TEMP                    -0.122088  0.082482 -0.026488 -0.287520  1.000000
TOTAL_PEDESTRIANS        0.472073  0.104579 -0.016583 -0.394582  0.268174
TRAFFICV                 0.477539  0.084969  0.032525 -0.350584  0.192925
WD                      -0.166609 -0.076483 -0.105209 -0.139511  0.140838
WS                      -0.111439  0.093354  0.051974 -0.007080  0.003678
traffic_volume           0.480183  0.086146  0.033371 -0.346046  0.195735

                         TOTAL_PEDESTRIANS  TRAFFICV        WD        WS  \
AQI                               0.138002  0.229438 -0.002961 -0.026338
CITY_CENTRE_TVCOUNT               0.894863  0.731762  0.055066  0.125728
NO                                0.435060  0.428769 -0.172695 -0.111132
NO2                               0.487276  0.522105 -0.134448 -0.099635
NOx                               0.472073  0.477539 -0.166609 -0.111439
PM10                              0.104579  0.084969 -0.076483  0.093354
PM2.5                            -0.016583  0.032525 -0.105209  0.051974
RH                               -0.394582 -0.350584 -0.139511 -0.007080
TEMP                              0.268174  0.192925  0.140838  0.003678
TOTAL_PEDESTRIANS                 1.000000  0.759693  0.049020  0.123375
TRAFFICV                          0.759693  1.000000  0.004132  0.040930
WD                                0.049020  0.004132  1.000000  0.067019
WS                                0.123375  0.040930  0.067019  1.000000
traffic_volume                    0.761994  0.999842  0.002726  0.037286

                         traffic_volume
AQI                            0.222811
CITY_CENTRE_TVCOUNT            0.734705
NO                             0.431860
NO2                            0.523750
NOx                            0.480183
PM10                           0.086146
PM2.5                          0.033371
RH                            -0.346046
TEMP                           0.195735
TOTAL_PEDESTRIANS              0.761994
TRAFFICV                       0.999842
WD                             0.002726
WS                             0.037286
traffic_volume                 1.000000
```

Figure 16: Correlation matrix

Descriptive statistics, pollutant distributions, time-series plots, hourly profiles, day-hour heatmaps, correlations and traffic-pollution relationships were created. Mean $PM_{2.5}$ was 7.75 and mean NO_2 was 47.59. The concentrations of $PM_{2.5}$ and NO_2 were in the range 0 to 209.40 and -17.8 to 540.96,

respectively, which may suggest sensor or processing issues that warrant additional quality control measures (as shown in Figure 6).

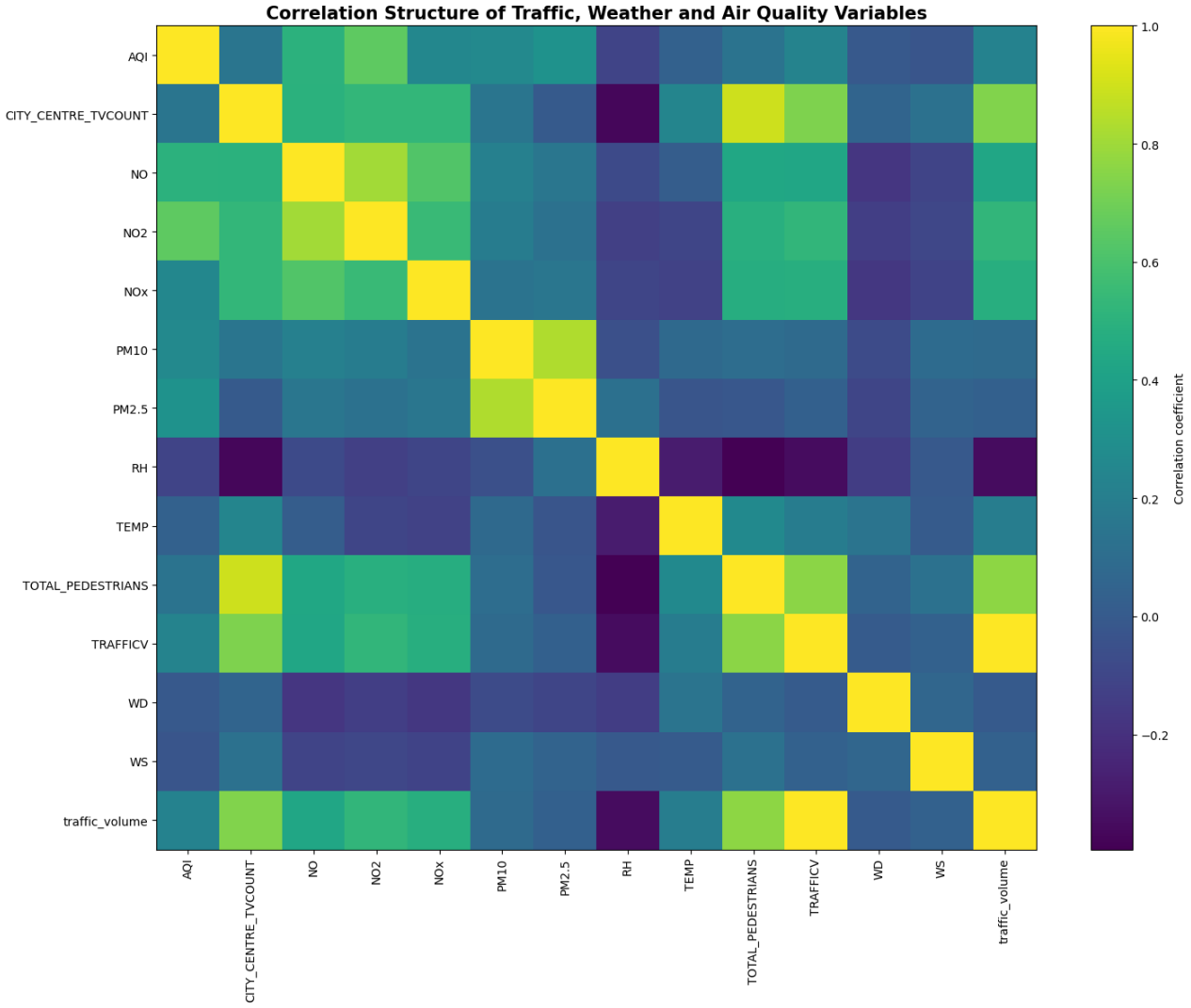


Figure 17: Correlation Heatmap

The results showed a good correlation between NO_2 and NO (0.807), and between $PM_{2.5}$ and PM_{10} (0.767). The positive relationship between traffic volume and NO_2 was around 0.524, while the correlation of traffic volume with $PM_{2.5}$ was only 0.033, indicating there are differences in relation between pollutants, which necessitate multivariable analysis.

Time-Series Feature Engineering

The final modelling structures comprised 40 features, such as temporal variables, lagged pollution measurements, rolling statistics and traffic information. The temporal persistence and recent environmental conditions were recorded by lag and rolling features.

Model Development

```
model_results = pd.DataFrame(all_test_results)
walk_forward_results = pd.concat(
    all_cv_results, ignore_index=True
)
prediction_results = pd.concat(
    all_predictions, ignore_index=True
)

cv_summary_all = (
    walk_forward_results
    .groupby(["Pollutant", "Model"], as_index=False)
    [["MAE", "RMSE", "R2"]]
    .mean()
    .rename(columns={
        "MAE": "CV_MAE",
        "RMSE": "CV_RMSE",
        "R2": "CV_R2"
    })
)

model_results = model_results.merge(
    cv_summary_all,
    on=["Pollutant", "Model"],
    how="left"
)

model_results.to_csv(
    os.path.join(OUTPUT_DIR, "model_comparison.csv"),
    index=False
)

walk_forward_results.to_csv(
    os.path.join(OUTPUT_DIR, "walk_forward_results.csv"),
    index=False
)

prediction_results.to_csv(
    os.path.join(OUTPUT_DIR, "test_predictions.csv"),
    index=False
)

print("\nFINAL HOLDOUT MODEL COMPARISON")
print(
    model_results.sort_values(
        ["Pollutant", "RMSE"]
    ).to_string(index=False)
)
```

```
FINAL HOLDOUT MODEL COMPARISON
Pollutant                Model  Train_Observations  Test_Observations          Test_Start            Test_End      MAE     RMSE       R2    CV_MAE   CV_RMSE     CV_R2
      NO2 Hist Gradient Boosting                6711               1185 2022-10-15 23:00:00 2023-02-28 12:00:00 2.796586 3.662421 0.693690  6.682957  9.200446  0.628532
      NO2      Gradient Boosting                6711               1185 2022-10-15 23:00:00 2023-02-28 12:00:00 2.851642 3.690969 0.688896  6.548966  9.127713  0.643487
      NO2          Random Forest                6711               1185 2022-10-15 23:00:00 2023-02-28 12:00:00 3.018983 3.888791 0.654654  6.561182  9.098245  0.645176
      NO2            Extra Trees                6711               1185 2022-10-15 23:00:00 2023-02-28 12:00:00 3.084981 3.939201 0.645643  6.532988  9.003493  0.643608
      NO2       Ridge Regression                6711               1185 2022-10-15 23:00:00 2023-02-28 12:00:00 3.845315 4.814247 0.470724  8.123337 10.496627  0.493845
      NO2         Neural Network                6711               1185 2022-10-15 23:00:00 2023-02-28 12:00:00 4.510964 5.806199 0.230145 11.944186 15.094704 -0.082369
    PM2.5            Extra Trees               14803               2613 2022-10-25 11:00:00 2023-02-28 12:00:00 0.954209 1.407619 0.782864  1.108544  1.738572  0.728976
   PM2.5 Hist Gradient Boosting               14803               2613 2022-10-25 11:00:00 2023-02-28 12:00:00 0.971355 1.429569 0.776040  1.113815  1.751670  0.724505
    PM2.5          Random Forest               14803               2613 2022-10-25 11:00:00 2023-02-28 12:00:00 0.985751 1.440607 0.772568  1.127469  1.769904  0.719900
    PM2.5      Gradient Boosting               14803               2613 2022-10-25 11:00:00 2023-02-28 12:00:00 0.977685 1.456604 0.767489  1.126625  1.793698  0.711515
    PM2.5       Ridge Regression               14803               2613 2022-10-25 11:00:00 2023-02-28 12:00:00 1.065898 1.578104 0.727082  1.178290  1.807279  0.708944
    PM2.5         Neural Network               14803               2613 2022-10-25 11:00:00 2023-02-28 12:00:00 1.174591 1.661837 0.697352  1.663409  2.370730  0.479436
```

Figure 18: Model Evaluation

Each target pollutant was combined with six regression models that were created and tested in the revised pipeline. Leakage was handled by not including the target measurements that were taken at the

same time. Five-fold walk forward validation was performed using the development data and the data was evaluated using a separate chronological holdout.

Final holdout model performance:

Pollutant	Model	MAE	RMSE	R^2
$PM_{2.5}$	Extra Trees	0.9542	1.4076	0.7829
$PM_{2.5}$	Hist Gradient Boosting	0.9714	1.4296	0.7760
$PM_{2.5}$	Random Forest	0.9858	1.4406	0.7726
$PM_{2.5}$	Gradient Boosting	0.9777	1.4566	0.7675
$PM_{2.5}$	Ridge Regression	1.0659	1.5781	0.7271
$PM_{2.5}$	Neural Network	1.1746	1.6618	0.6974
NO_2	Hist Gradient Boosting	2.7966	3.6624	0.6937
NO_2	Gradient Boosting	2.8516	3.6910	0.6889
NO_2	Random Forest	3.0190	3.8888	0.6547
NO_2	Extra Trees	3.0850	3.9392	0.6456
NO_2	Ridge Regression	3.8453	4.8142	0.4707
NO_2	Neural Network	4.5110	5.8062	0.2301

To choose the pollutants, the RMSE of the mean for the Walk-Forward validation technique was used for Extra Trees. The validation RMSE for Extra Trees for $PM_{2.5}$ was 1.7386, and the holdout RMSE was 1.4076. In the case of NO_2, it had RMSE of 9.0035 and 3.9392 for validation and holdout respectively.

The holdout results demonstrate that Extra Trees had the best performance on $PM_{2.5}$ whereas Hist Gradient Boosting had the smallest holdout RMSE for NO_2. It is worth noting that the distinction is made on the basis of the results of the validation and not the final test. There were 2613 $PM_{2.5}$ observations and 1185 NO_2 observations during the test period; the different start dates reflect the availability of the data for each gas.

The new scores are far more realistic than were previously achieved, which was near perfect. The enhancement in the design of the evaluation is useful in giving a more useful measure of the predictive power, but does not imply that the model will have the same predictive power under all future conditions.

Feature Analysis

The updated feature design includes historical pollutant measurements, lagged explanatory variables, rolling statistics and calendar variables. Target values not present during the model's training are not used, lowering the chance of the model getting a favourably high score simply because it had access to information which was not at prediction time. However, one must exercise caution in using feature importance or model explainability, especially in the case of incomplete or correlated environmental observations.

24-Hour Forecasting

The updated pipeline produces hourly predictions of the next 24 hours for each pollutant with the chosen Extra Trees model. All the historical target lags are updated recursively when a forecast is generated. The workflow outputs hourly predictions and forecast summaries, such as mean, max, min, std and forecast-hour-count.

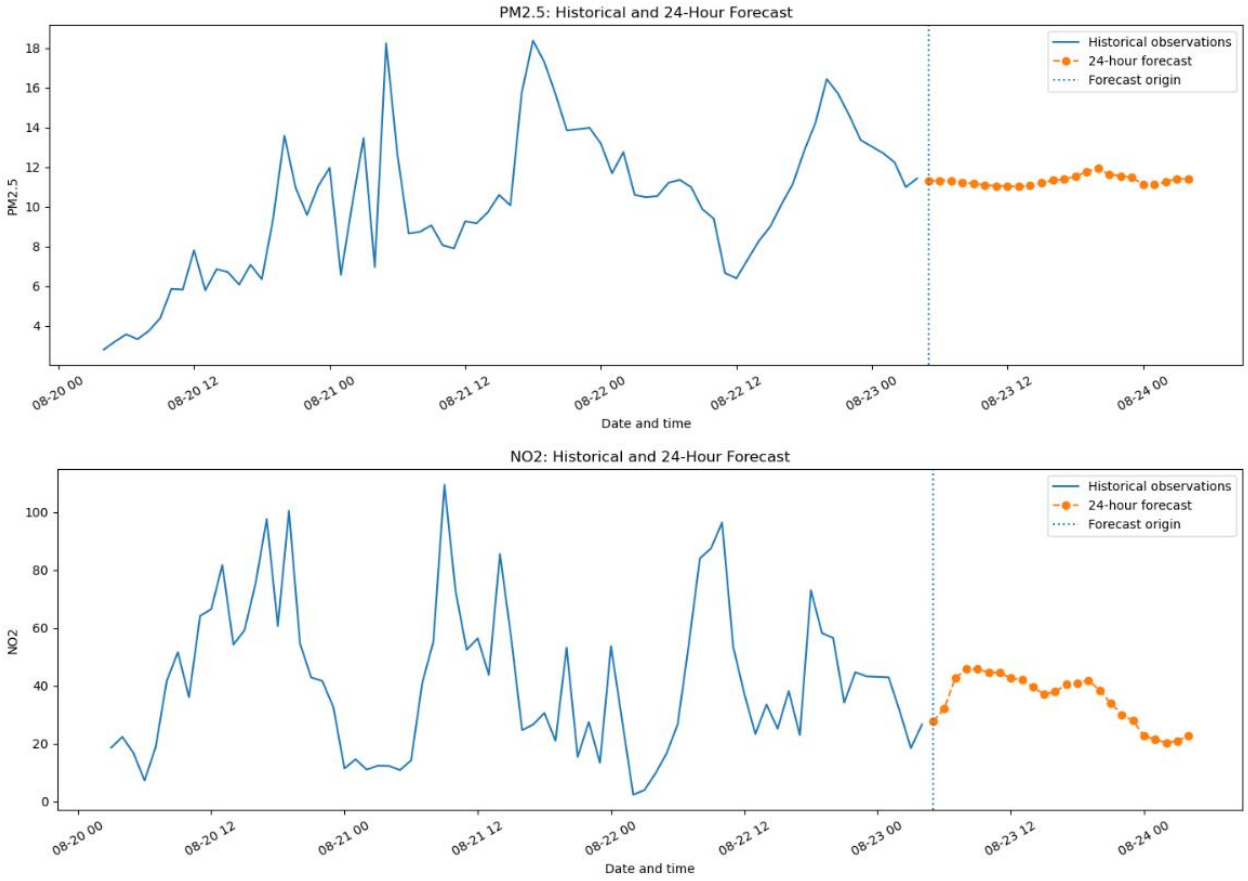


Figure 19: 24-hour forecasts

The resulting forecasts were:

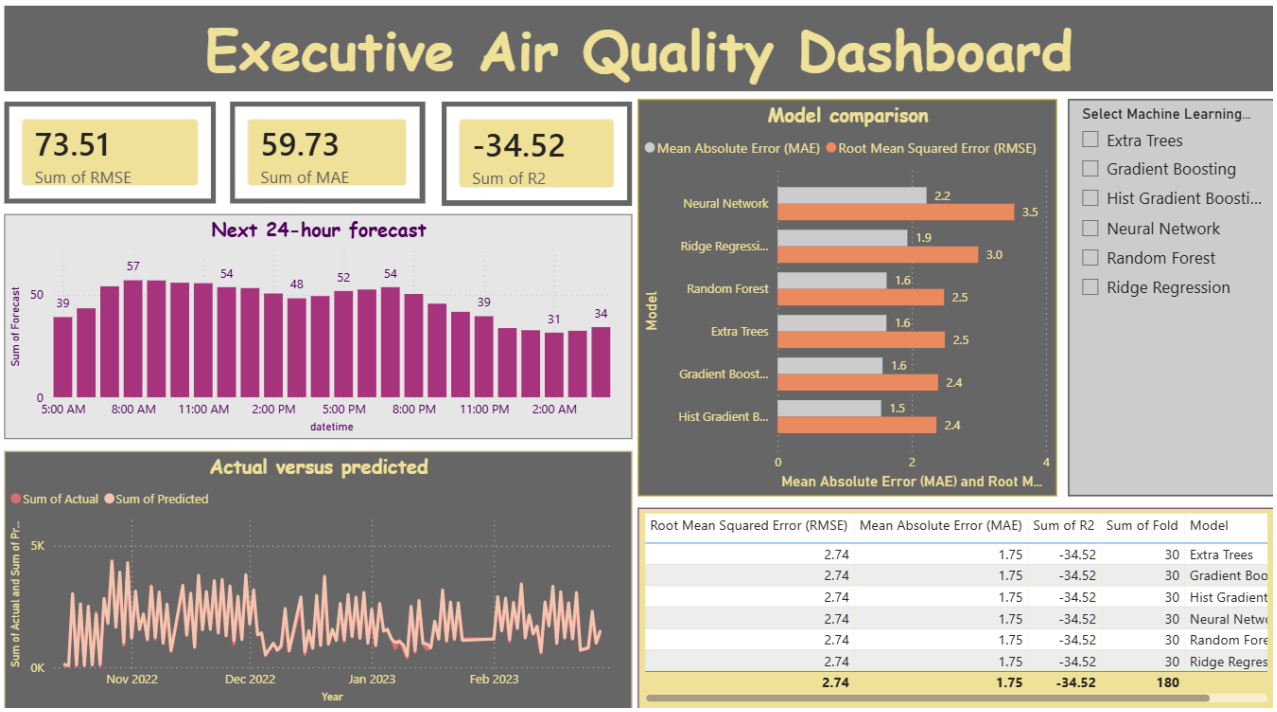
Pollutant	Selected Model	Mean	Maximum	Minimum
$PM_{2.5}$	Extra Trees	11.3164	11.9244	11.0062
NO_2	Extra Trees	35.1848	45.7605	20.2130

In addition, a separate walk-forward backtest was also conducted, where forecasts were compared to the values that were observed later in time, for all historical forecast origins. It is important because the performance of good holdout is not sufficient to assure that the multi-step forecasts are accurate. The backtest includes some negative R^2 values, indicating that the forecast accuracy can fluctuate significantly over time.

The forecast produced is a technical demonstration, not a confirmed public-health forecast. Further study of future traffic and meteorological input, monitoring coverage and uncertainty estimates to operational baselines is needed prior to deployment.

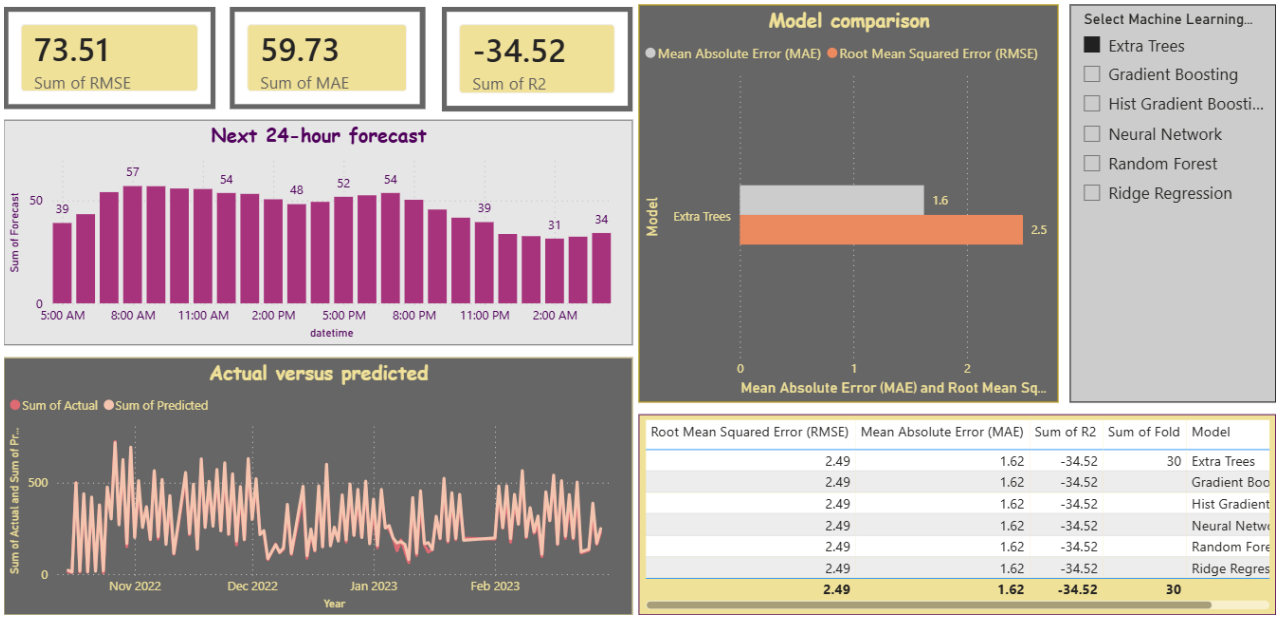
4.2 Decision-Support Outputs

The project generated output in CSV and Excel format, such as model metrics, test predictions, walk-forward validation results, historical observations, 24-hour forecasts, and forecast summaries generated in Python. The data sets were utilized to create an interactive executive dashboard in Power BI.



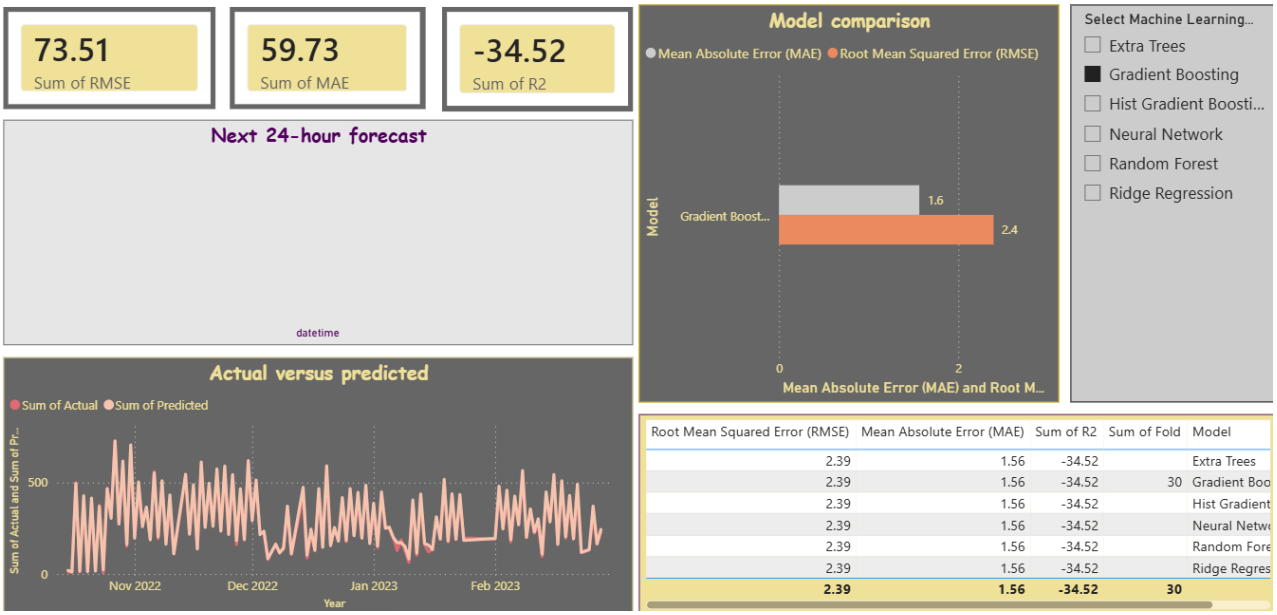
Root Mean Squared Error (RMSE)	Mean Absolute Error (MAE)	Sum of R2	Sum of Fold	Model
2.74	1.75	-34.52	30	Extra Trees
2.74	1.75	-34.52	30	Gradient Boo
2.74	1.75	-34.52	30	Hist Gradient
2.74	1.75	-34.52	30	Neural Netwo
2.74	1.75	-34.52	30	Random Fore
2.74	1.75	-34.52	30	Ridge Regres
2.74	**1.75**	**-34.52**	**180**	

Executive Air Quality Dashboard



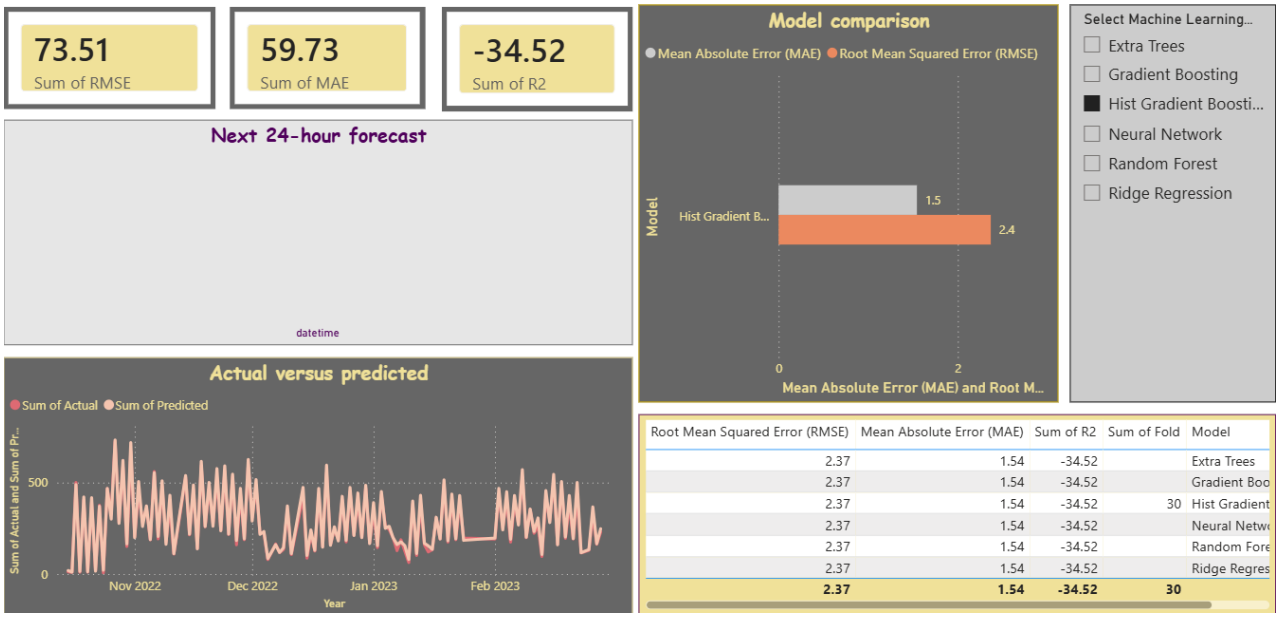
Root Mean Squared Error (RMSE)	Mean Absolute Error (MAE)	Sum of R2	Sum of Fold	Model
2.49	1.62	-34.52	30	Extra Trees
2.49	1.62	-34.52		Gradient Boo
2.49	1.62	-34.52		Hist Gradient
2.49	1.62	-34.52		Neural Netwo
2.49	1.62	-34.52		Random Fore
2.49	1.62	-34.52		Ridge Regres
2.49	**1.62**	**-34.52**	**30**	

Executive Air Quality Dashboard



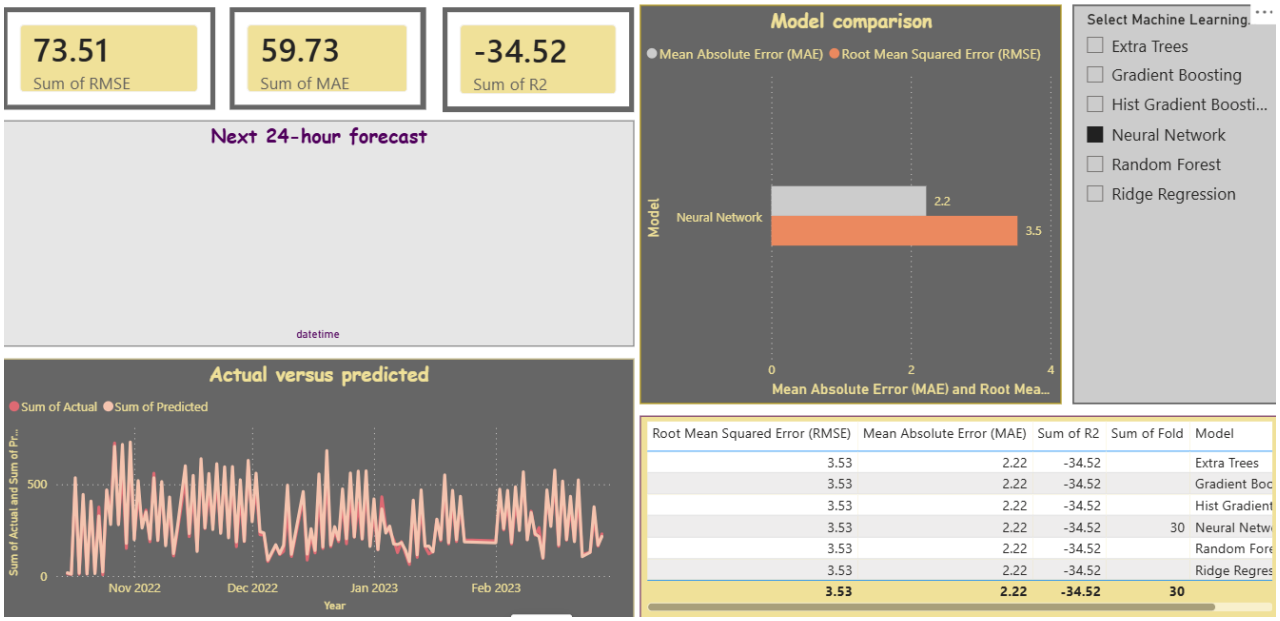
Root Mean Squared Error (RMSE)	Mean Absolute Error (MAE)	Sum of R2	Sum of Fold	Model
2.39	1.56	-34.52		Extra Trees
2.39	1.56	-34.52	30	Gradient Boo
2.39	1.56	-34.52		Hist Gradient
2.39	1.56	-34.52		Neural Netwo
2.39	1.56	-34.52		Random Fore
2.39	1.56	-34.52		Ridge Regres
2.39	**1.56**	**-34.52**	**30**	

Executive Air Quality Dashboard



Root Mean Squared Error (RMSE)	Mean Absolute Error (MAE)	Sum of R2	Sum of Fold	Model
2.37	1.54	-34.52		Extra Trees
2.37	1.54	-34.52		Gradient Boo
2.37	1.54	-34.52	30	Hist Gradient
2.37	1.54	-34.52		Neural Netw
2.37	1.54	-34.52		Random Fore
2.37	1.54	-34.52		Ridge Regres
2.37	**1.54**	**-34.52**	**30**	

Executive Air Quality Dashboard



Root Mean Squared Error (RMSE)	Mean Absolute Error (MAE)	Sum of R2	Sum of Fold	Model
3.53	2.22	-34.52		Extra Trees
3.53	2.22	-34.52		Gradient Boo
3.53	2.22	-34.52		Hist Gradient
3.53	2.22	-34.52	30	Neural Netw
3.53	2.22	-34.52		Random Fore
3.53	2.22	-34.52		Ridge Regres
3.53	**2.22**	**-34.52**	**30**	

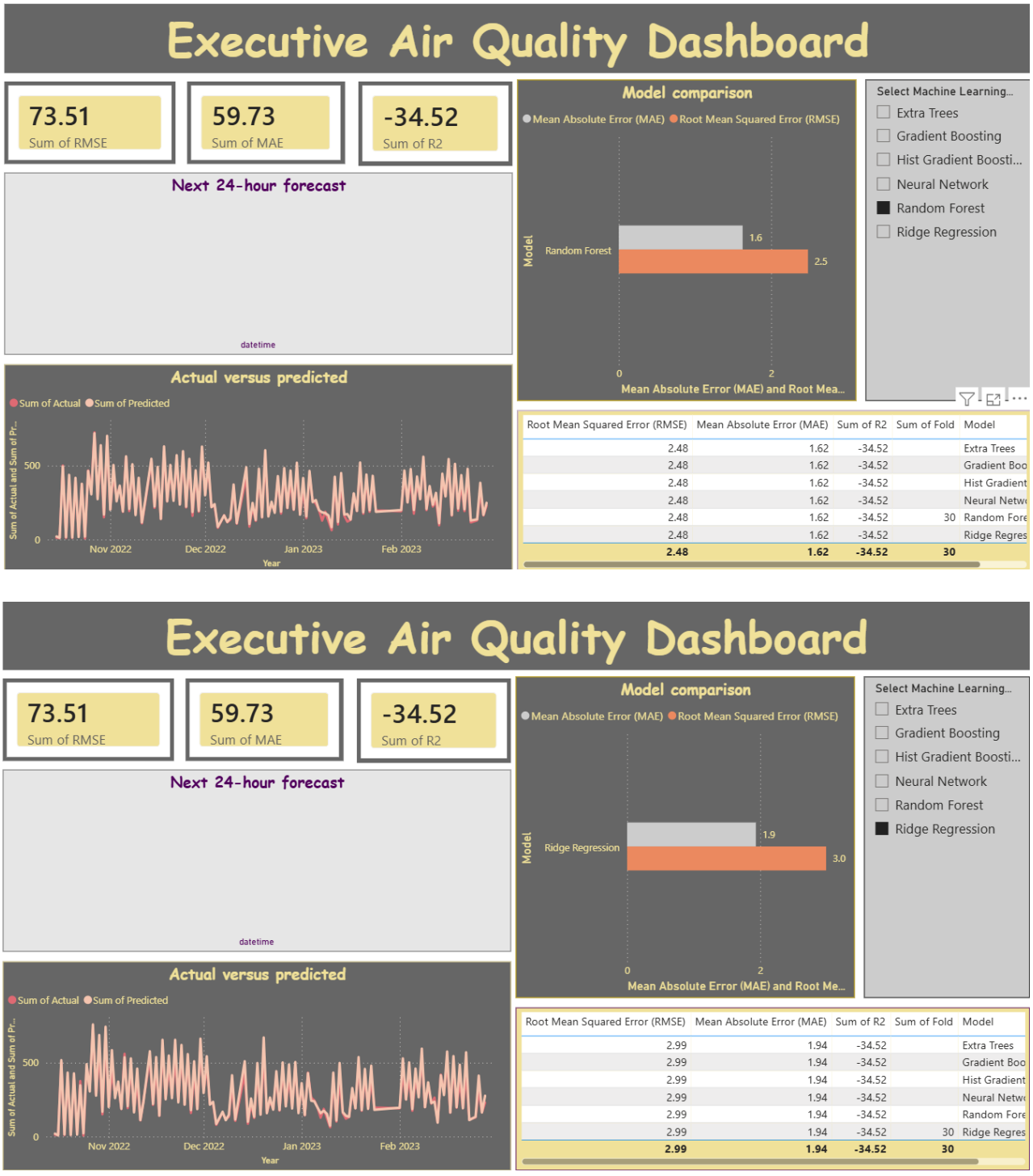


Figure 20: Executive Air Quality Dashboard

(Acquired from Power BI)

The dashboard includes a model-comparison chart, a 24-hour forecast chart, an actual versus predicted chart, a model-selection slicer and a table of evaluation metrics. The model slicer lets me choose from Random Forest, Gradient Boosting, Neural Network, Extra Trees, Hist Gradient Boosting and Ridge Regression. The dashboard was designed to provide the results from the forecasting to the decision makers without having to understand the Python output directly.

All forecast trends, model comparisons and prediction results are consolidated on the dashboard. Currently, the KPI cards shown, however, are aggregating the MAE, RMSE and R^2 by the sum. These measures need to be adjusted to reflect meaningful measures for the specific model before being used for performance reporting on the dashboard. Filters for date and pollutant should be also checked for proper interaction with all the relevant visuals.

4.3 Testing and Evaluation

The data preparation, feature construction, target-leakage prevention, chronological model validation, holdout evaluation, forecasting, and export generation were tested. Metrics for all of the models and both target pollutants were generated for the revised model pipeline. The final holdout results were not used to validate the walk forward results.

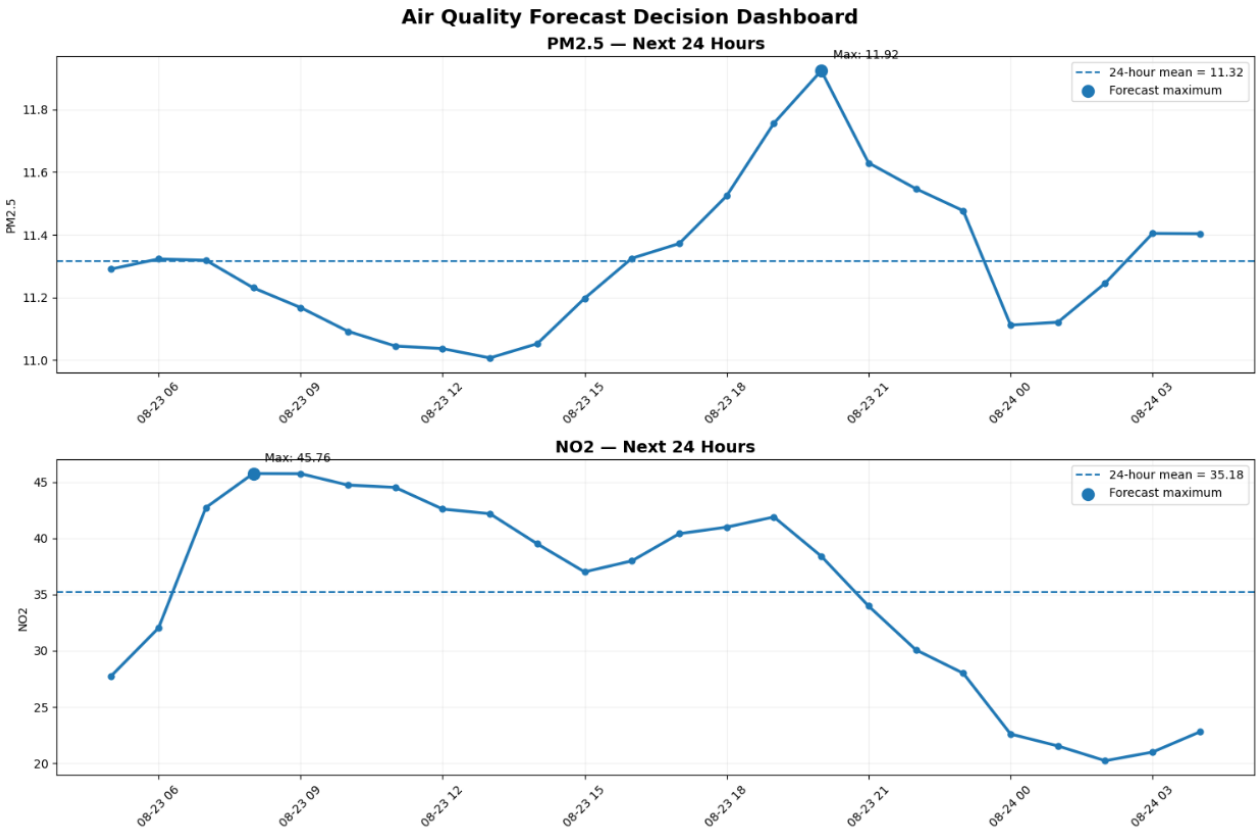


Figure 21: Air Quality Forecast Decision Dashboard

The forecasting code produced 24 hourly forecasts of $PM_{2.5}$ and NO_2, as well as the forecast summary files. The independent 24-hour backtest, which was carried out for performance from the various historical forecast sources, revealed that there were no consistent strong performances on all periods.

Observations were visualised using Power BI to show model performance metrics, actual versus predicted observations and forecast trends. But the dashboard KPIs (sum of error metrics and R^2) are actually incorrect because they cannot be used as model-level KPIs. These aggregations should be replaced with appropriate measures and behavior of slicers checked, then retest the dashboard.

The overall outcome of the revised work is the methodological reliability due to the target leakage addressed and the time ordered validation. It also shows that additional testing is required before operational forecasting claims can be made.

4.4 Ethical Evaluation

Avoiding the intentional collection of personally identifiable information was a project goal. However, there are data ownership, data provenance, data security, access and Māori data sovereignty issues that need to be addressed in the future. Predictions must not be made as absolute statements, especially when for public health or traffic related purposes. Observed and predicted values should be clearly separated on dashboards, and information about data quality problems and uncertainty should be communicated.

The assessment shows the need to explicitly convey uncertainty and the limitations of the model. Projections should not be represented as a certainty and users should be notified of poor monitoring coverage or back-test results.

4.5 Maintenance and Scaling

Automated ingestion of sensor and traffic and meteorological data should replace manual data loading from excel for production deployment. Monitoring should be used to track missingness, data drift and forecast errors and also changing pollutant relationships, and retraining should be considered if pre-defined performance thresholds are breached.

Further scaling may involve adding other Auckland monitoring sites, database or cloud infrastructure and API for authorised dashboard applications.

5. Individual Discussion

Personal Contribution

My main contribution to the development phase was the practical implementation of the data analytics and machine-learning pipeline. This included transforming raw excel data to a usable analytical data, creating time-series features, training and comparing machine learning models, assessing model performance and producing 24-hour forecast and decision-support outputs.

A major take-home message was that whereas choosing an algorithm was only one component of successful machine-learning implementation, it was a far from the sole one. The first datasets were in various formats, had different time-stamps and some were more complete than others. The data needed

to be cleaned and harmonized before modelling can be conducted. The project thus reinforced my knowledge of the link between data engineering and machine learning.

The lack of availability of some of the pollutants proposed for the target list was a big technical challenge. The project proposal included O_3, which was not provided in the final dataset. I did not model artificial O_3 values, but just restricted it to $PM_{2.5}$ and NO_2. This resulted in the scope of the final MVP being changed but remained analytical in nature.

A second problem was the lack of data. Environmental monitoring data are not always complete and the project data showed significant variations in pollutants monitored. Interpolation and limited forward or backward filling were thus employed, but this procedure also showed that treatment of missing values can impact the behaviour of the model.

Learning through revising the modelling process. The original R^2s were near 1.0, suggesting possible leakage between the targets, so I changed the predictors to be based on past values instead of contemporaneous values of the target. Then I re-trained the original 3 model types and trained three more models, including Extra Trees, Hist Gradient Boosting and Ridge Regression. This afforded a wider comparison of the predictive methods.

I also ran five-fold forward-looking and a holdout test on the data. Both Extra Trees and Hist Gradient Boosting were chosen for both pollutants in terms of validation RMSE though the latter had the best holdout RMSE for NO_2. This illustrated the necessity of maintaining the separation of model selection based on validation and independent test evaluation.

The 24-hour forecasting process was extended to produce recursive hourly forecasts and to test forecast windows in the past. The backtesting results were not uniform over the period, indicating that a good backtesting performance does not ensure a consistent performance in the forecasting process in the future. Better future predictions will need appropriate future weather and traffic inputs, better baseline comparisons and ongoing assessment of forecast error.

I also developed a Power BI executive dashboard to share model comparisons, forecast and actuals versus predictions. It helped me learn more about conveying the results of an analysis to those who are

not technical. I did find that the current KPI cards sum the evaluation data, and the dashboard needs to be corrected before any data can be trusted on the dashboard.

In general, it deepened my insight into the responsibility of machine learning, and the importance of data quality, leakage prevention, and time-aware evaluation and communication. Future work should focus on making it more explainable, more accurate in predicting uncertainty, make a dashboard more easy to validate and be able to reproduce the entire pipeline.

6. Conclusion and Future Directions

6.1 Conclusion

The project was built an end-to-end MVP for integrating traffic and air-quality observations and features, engineering time-series features, training machine-learning models and generating 24-hour forecasts for $PM_{2.5}$ and NO_2 in Auckland Centre. No suitable observations were provided in the data for O_2 and so it was not included.

The target leakage was remedied, a comparison of six regression models was performed and five-fold "walk-forward" cross validation with a separate chronological holdout test was conducted. Validation RMSE was used to choose Extra Trees for both pollutants. Its holdout RMSE was 1.4076 for $PM_{2.5}$ and 3.9392 for NO_2. A 24-hour backtest also showed some variation depending on the forecast window in the past, indicating that some further improvements are still needed to increase the reliability of the forecasts.

The project also delivered forecast exports and a Power BI executive dashboard to convey model comparisons, forecast trends and prediction outcomes. To use the KPI aggregations shown in the dashboard, they need to be corrected first. Overall, the project illustrates a workable analytical prototype and a more strident evaluation process but not yet the reliability of operational forecasting.

6.2 Practical Implications

The MVP helps to showcase how historical environment and traffic information can be leveraged into proactive decision-support outputs. An operational dashboard may be developed in the future that could display pollutant predictions, uncertainty bounds, data-quality descriptions and threshold alerts. These outputs might be used for environmental monitoring, communication planning and investigation of any

potential link between traffic conditions and pollution. However, forecasts must be used to assist operations and should not make decisions. Further expansion needs to also take into account data governance, including data governance with Māori, and engagement of data that aligns with responsibilities under Te Tiriti o Waitangi.

The Power BI dashboard offers a useful user interface for model comparison and forecasting trend analysis. Model specific KPI measures, date and pollutant filtering and visual interactions need to be validated before using them for operational decisions. The uncertainty of the forecast and backtesting performance should also be clearly stated to the stakeholders.

6.3 Future Directions

A list of five priorities is suggested. Primarily it is required to improve the recursive 24 hour forecasting approach by adding reliable future weather and traffic information. Second, improve evaluation via historical rolling-origin backtests, appropriate baseline models and prediction intervals. Third, explore pollutant coverage and add O3 observations and other monitoring sites if available. Fourth, enhance the interpretability of the chosen Extra Trees models and track the quality of the data, drift, and errors in the forecasts. Fifth, validate, correct and operationalize Power BI KPI Measures, model selection, date and pollutant filtering.

These enhancements would help to create a more accurate and transparent forecasting capability for future environmental decision making.

7. Projects and GitHub

The GitHub Repository is:

8. References

Chuang, I. T., & Chen, Q. (2025). Urban street dynamics: Assessing the relationship of sidewalk width and pedestrian activity in Auckland, New Zealand, based on mobile phone data. *Urban Studies*, *62*(8), 1546-1565. https://journals.sagepub.com/doi/pdf/10.1177/00420980241293659

Martínez-García, M., & Hernández-Lemus, E. (2022). Data integration challenges for machine learning in precision medicine. *Frontiers in medicine*, *8*, 784455. https://www.frontiersin.org/journals/medicine/articles/10.3389/fmed.2021.784455/pdf

New Zealand Government Chief Data Steward. (2020). Ngā Tikanga Paihere: Data ethics guidance. *data.govt.nz*. https://www.data.govt.nz/toolkit/data-ethics/nga-tikanga-paihere/

New Zealand Government Chief Data Steward. (2021). Government Data Strategy 2021. *data.govt.nz*. https://www.data.govt.nz/leadership/government-data-strategy-2021/

Office of the Privacy Commissioner. (2026). *Privacy principles*. https://www.privacy.org.nz/privacy-principles/

Piotrowski, P., Rutyna, I., Baczyński, D., & Kopyt, M. (2022). Evaluation metrics for wind power forecasts: A comprehensive review and statistical analysis of errors. *Energies*, *15*(24), 9657. https://www.mdpi.com/1996-1073/15/24/9657

9. Appendices

In [2]:
```
import os
import warnings
warnings.filterwarnings("ignore")

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor
from sklearn.neural_network import MLPRegressor
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

# Optional SHAP
try:
    import shap
    SHAP_AVAILABLE = True
except ImportError:
    SHAP_AVAILABLE = False
```

FILE PATHS

In [3]:
```
TRAFFIC_PATH = r"D:\Traffic_Data_Queen St_Atlternative.xlsx"
AIR_AQI_PATH = r"D:\1C. Dashboard_data_Queen Street.xlsx"
AIR_POLLUTION_PATH = r"D:\2C. Dashboard_data_Queen Street.xlsx"

OUTPUT_DIR = r"D:\Auckland_Air_Quality_MVP"

os.makedirs(OUTPUT_DIR, exist_ok=True)
```

LOAD DATA

In [4]:
```
traffic = pd.read_excel(TRAFFIC_PATH)
aqi = pd.read_excel(AIR_AQI_PATH)
pollution = pd.read_excel(AIR_POLLUTION_PATH)

print("Traffic shape:", traffic.shape)
print("AQI shape:", aqi.shape)
print("Pollution shape:", pollution.shape)
```

```
Traffic shape: (45528, 14)
AQI shape: (877795, 5)
Pollution shape: (245955, 5)
```

In [5]:
```
traffic
```

Out[5]:

	calendar_date	time_hh24	2007-1	2007-2	2007-3	2007-4	2007-5	2007-6	2007-7	2007-8	2007-9	2007-10	2007-11	Unnamed: 13
0	2018-01-01	0.0	72	98	68	139	88	36	38	37	92	103	141	912
1	2018-01-01	1.0	94	72	95	78	21	34	29	41	56	62	69	651
2	2018-01-01	2.0	157	159	170	117	54	40	44	63	123	170	122	1219
3	2018-01-01	3.0	192	192	151	177	101	50	55	86	118	179	184	1485
4	2018-01-01	4.0	155	186	108	168	105	47	47	62	125	93	173	1269
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
45523	NaT	NaN	0	78	151	156	178	0	93	0	80	0	47	783
45524	NaT	NaN	0	78	157	99	118	0	76	0	65	0	66	659
45525	NaT	NaN	0	100	154	154	180	0	82	0	105	0	56	831
45526	NaT	NaN	0	72	166	116	139	0	68	0	83	0	78	722
45527	NaT	NaN	0	101	147	147	165	0	76	0	108	0	67	811

45528 rows × 14 columns

In [6]:
```
aqi
```

Out[6]:

	Site	Date	Time	Parameter	Value
0	QUS	2006-01-01	00:00:00	AQI	25.0
1	QUS	2006-01-01	01:00:00	AQI	26.0
2	QUS	2006-01-01	02:00:00	AQI	19.0
3	QUS	2006-01-01	03:00:00	AQI	18.0
4	QUS	2006-01-01	04:00:00	AQI	24.0
...	...	...	...	...	...
877790	QUS	2023-03-27	23:00:00	AQI	12.0
877791	QUS	2023-03-28	00:00:00	AQI	11.0
877792	QUS	2023-03-28	01:00:00	AQI	6.0
877793	QUS	2023-03-28	02:00:00	AQI	7.0
877794	QUS	2023-03-28	03:00:00	AQI	8.0

877795 rows × 5 columns

In [7]:
```
pollution
```

Out[7]:

	Site	Date	Time	Parameter	Value
0	QUS	2022-07-01	01:00:00	NO	5.000000
1	QUS	2022-07-01	02:00:00	NO	4.600000
2	QUS	2022-07-01	03:00:00	NO	5.000000
3	QUS	2022-07-01	04:00:00	NO	5.200000
4	QUS	2022-07-01	05:00:00	NO	5.900000
...	...	...	...	...	...
245950	QUS	2012-12-27	00:00:00	NOx	119.379522
245951	QUS	2012-12-28	00:00:00	NOx	77.732527
245952	QUS	2012-12-29	00:00:00	NOx	77.062901
245953	QUS	2012-12-30	00:00:00	NOx	71.255332
245954	QUS	2012-12-31	00:00:00	NOx	95.183501

245955 rows × 5 columns

CLEAN TRAFFIC DATA

In [8]:
```
traffic["datetime"] = (
    pd.to_datetime(traffic["calendar_date"], errors="coerce")
    + pd.to_timedelta(traffic["time_hh24"], unit="h")
)

traffic = traffic.dropna(subset=["datetime"]).copy()

# Use the supplied total traffic column where available
if "Unnamed: 13" in traffic.columns:
    traffic["traffic_volume"] = pd.to_numeric(
        traffic["Unnamed: 13"], errors="coerce"
    )
else:
    # Otherwise sum available traffic columns
    traffic_cols = [
        c for c in traffic.columns
        if c not in ["calendar_date", "time_hh24", "datetime"]
    ]
    traffic["traffic_volume"] = traffic[traffic_cols].apply(
        pd.to_numeric, errors="coerce"
    ).sum(axis=1)

traffic = traffic[["datetime", "traffic_volume"]]

traffic = (
    traffic.groupby("datetime", as_index=False)["traffic_volume"]
    .mean()
    .sort_values("datetime")
)

print("\nTraffic data:")
print(traffic.head())
```

```
Traffic data:
             datetime  traffic_volume
0 2018-01-01 00:00:00           912.0
1 2018-01-01 01:00:00           651.0
2 2018-01-01 02:00:00          1219.0
3 2018-01-01 03:00:00          1485.0
4 2018-01-01 04:00:00          1269.0
```

CLEAN AIR QUALITY DATA

In [9]:
```python
def prepare_air_data(df):
    data = df.copy()

    data["Date"] = pd.to_datetime(data["Date"], errors="coerce")
    data["Time"] = pd.to_timedelta(data["Time"].astype(str), errors="coerce")

    data["datetime"] = data["Date"] + data["Time"]

    data["Value"] = pd.to_numeric(data["Value"], errors="coerce")

    data["Parameter"] = (
        data["Parameter"]
        .astype(str)
        .str.strip()
        .str.upper()
    )

    data = data.dropna(
        subset=["datetime", "Value", "Parameter"]
    )

    return data[["datetime", "Parameter", "Value"]]


aqi_clean = prepare_air_data(aqi)
pollution_clean = prepare_air_data(pollution)

air = pd.concat(
    [aqi_clean, pollution_clean],
    ignore_index=True
)
```

In [10]:
```python
# Remove exact duplicate records
air = air.drop_duplicates(
    subset=["datetime", "Parameter"],
    keep="first"
)
```

In [11]:
```
# Convert long format into columns
air = air.pivot_table(
    index="datetime",
    columns="Parameter",
    values="Value",
    aggfunc="mean"
).reset_index()

air.columns.name = None

air = air.sort_values("datetime")

print("\nAvailable air-quality variables:")
print([c for c in air.columns if c != "datetime"])
```

```
Available air-quality variables:
['AQI', 'CITY_CENTRE_TVCOUNT', 'NO', 'NO2', 'NOX', 'PM10', 'PM2.5', 'RH', 'TEMP', 'TOTAL_PEDESTRIANS', 'TRAFFICV', 'WD', 'WS']
```

STANDARDISE POLLUTANT NAMES

In [12]:
```
rename_map = {}

for col in air.columns:
    clean = col.replace("_", "").replace(".", "")

    if clean == "PM25":
        rename_map[col] = "PM2.5"

    elif clean == "PM10":
        rename_map[col] = "PM10"

    elif clean == "NO2":
        rename_map[col] = "NO2"

    elif clean == "O3":
        rename_map[col] = "O3"

    elif clean == "NOX":
        rename_map[col] = "NOx"

    elif clean == "NO":
        rename_map[col] = "NO"

    elif clean == "AQI":
        rename_map[col] = "AQI"

air = air.rename(columns=rename_map)

print("\nStandardised variables:")
print(air.columns.tolist())
```

```
Standardised variables:
['datetime', 'AQI', 'CITY_CENTRE_TVCOUNT', 'NO', 'NO2', 'NOx', 'PM10', 'PM2.5', 'RH', 'TEMP', 'TOTAL_PEDESTRIANS', 'TRAFFICV', 'WD', 'WS']
```

MERGE AIR QUALITY + TRAFFIC

In [13]:
```
data = pd.merge_asof(
    air.sort_values("datetime"),
    traffic.sort_values("datetime"),
    on="datetime",
    direction="nearest",
    tolerance=pd.Timedelta("1 hour")
)

data = data.sort_values("datetime")

print("\nMerged dataset:")
print(data.head())
print("Shape:", data.shape)
```

```
Merged dataset:
    datetime  AQI  CITY_CENTRE_TVCOUNT  NO  NO2         NOx  PM10  PM2.5  RH  \
0 2004-01-31  NaN                  NaN NaN  NaN  208.021944   NaN    NaN NaN
1 2004-02-01  NaN                  NaN NaN  NaN  170.431389   NaN    NaN NaN
2 2004-02-02  NaN                  NaN NaN  NaN  238.845556   NaN    NaN NaN
3 2004-02-03  NaN                  NaN NaN  NaN  167.775556   NaN    NaN NaN
4 2004-02-04  NaN                  NaN NaN  NaN  162.131111   NaN    NaN NaN

   TEMP  TOTAL_PEDESTRIANS  TRAFFICV  WD  WS  traffic_volume
0   NaN                NaN       NaN NaN NaN             NaN
1   NaN                NaN       NaN NaN NaN             NaN
2   NaN                NaN       NaN NaN NaN             NaN
3   NaN                NaN       NaN NaN NaN             NaN
4   NaN                NaN       NaN NaN NaN             NaN
Shape: (155603, 15)
```

RESAMPLE TO HOURLY FREQUENCY

In [13]:
```
data = (
    data.set_index("datetime")
    .resample("1h")
    .mean(numeric_only=True)
    .reset_index()
)

data = data.sort_values("datetime")
```

HANDLE MISSING VALUES

In [14]:
```
data = data.sort_values("datetime").copy()
data = data.drop_duplicates(subset=["datetime"], keep="last")
data = data.set_index("datetime").asfreq("h").reset_index()

# Do not interpolate or backward-fill target pollutants.
# Keep original missing target observations as missing.
desired_targets = ["PM2.5", "NO2", "O3"]

available_targets = [
    t for t in desired_targets if t in data.columns
]

print("Available targets:", available_targets)

# Fill only explanatory variables, using past information.
# Never use bfill or two-sided interpolation for model inputs.
target_columns = available_targets

numeric_columns = data.select_dtypes(
    include=np.number
).columns.tolist()

exogenous_columns = [
    c for c in numeric_columns
    if c not in target_columns
]

data[exogenous_columns] = data[exogenous_columns].ffill(limit=6)

print("Hourly observations:", len(data))
print("Target missingness:")
print(data[target_columns].isna().sum())
```

```
Available targets: ['PM2.5', 'NO2']
Hourly observations: 171461
Target missingness:
PM2.5    118243
NO2       24425
dtype: int64
```

IDENTIFY TARGET POLLUTANTS

In [15]:
```
desired_targets = ["PM2.5", "NO2", "O3"]

available_targets = [
    target for target in desired_targets
    if target in data.columns
]

print("\nRequested target pollutants:", desired_targets)
print("Available target pollutants:", available_targets)

if not available_targets:
    raise ValueError(
        "PM2.5, NO2 and O3 were not found in the supplied datasets. "
        "Check the Parameter values in your Excel files."
    )
```

```
Requested target pollutants: ['PM2.5', 'NO2', 'O3']
Available target pollutants: ['PM2.5', 'NO2']
```

EXPLORATORY DATA ANALYSIS

In [16]:
```
print("\nDescriptive statistics:")
print(data.describe())
```

```
Descriptive statistics:
                      datetime            AQI  CITY_CENTRE_TVCOUNT             NO  \
count                   171461  144008.000000         45684.000000  151315.000000
mean    2013-11-11 02:00:00      25.795609         20263.929209      55.962619
min     2004-01-31 00:00:00     -32.000000           390.000000      -1.700000
25%     2008-12-21 01:00:00      16.000000          7723.000000      14.500000
50%     2013-11-11 02:00:00      24.000000         20537.500000      37.425850
75%     2018-10-02 03:00:00      34.000000         31001.250000      78.200000
max     2023-08-23 04:00:00     697.000000         51393.000000     792.090000
std                     NaN      16.239438         12814.644558      57.688756

                 NO2            NOx          PM10         PM2.5            RH  \
count  147036.000000  147296.000000  79999.000000  53218.000000  23642.000000
mean       40.822379     149.141410     17.549849      7.360742     65.290235
min       -17.800000      -9.900000      0.000000      0.000000     27.800000
25%        22.766625      51.103744     12.500000      5.100000     56.800000
50%        36.940000     111.000000     16.300000      6.700000     66.000000
75%        54.274835     199.657940     21.218552      8.900000     74.700000
max       540.962500    4334.414712    386.900000    209.400000     89.300000
std        25.687235     145.547859      7.964758      3.685793     11.835828

                TEMP  TOTAL_PEDESTRIANS      TRAFFICV            WD  \
count  57393.000000       45942.000000  45624.000000  49814.000000
mean      16.800478        6000.156458    674.985096    196.182667
min        0.000000           0.000000      0.000000      0.000000
25%       14.000000        1261.250000    402.000000    156.000000
50%       17.000000        4815.000000    663.000000    189.000000
75%       20.000000        9594.000000    938.000000    236.000000
max       29.000000       26634.000000   1995.000000    354.000000
std        4.040716        5237.797178    342.427969     74.914329
```

```
                WS  traffic_volume
count  49467.000000    44904.000000
mean       0.521656      677.305407
min        0.000000        0.000000
25%        0.300000      403.000000
50%        0.500000      664.000000
75%        0.600000      947.000000
max       52.800000     1995.000000
std        0.371454      343.438502
```

In [17]:

```
# DATASET OVERVIEW DASHBOARD

fig, axes = plt.subplots(1, 3, figsize=(16, 5))

# Number of observations
axes[0].bar(
    ["Merged\nObservations"],
    [len(data)]
)
axes[0].set_title("Dataset Size", fontweight="bold")
axes[0].set_ylabel("Number of Records")

# Available pollutants
axes[1].bar(
    available_targets,
    [data[x].notna().sum() for x in available_targets]
)
axes[1].set_title("Available Target Observations", fontweight="bold")
axes[1].set_ylabel("Valid Observations")

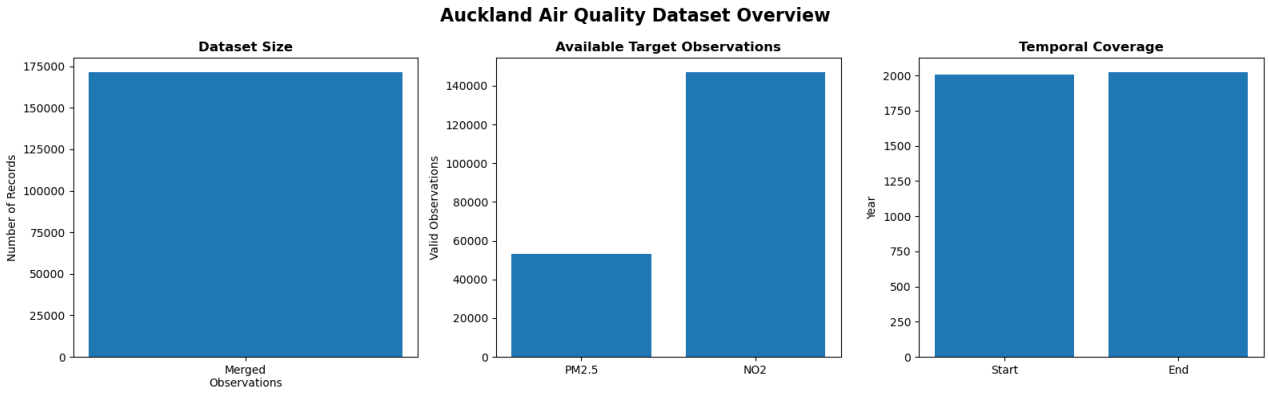
# Date coverage
axes[2].bar(
    ["Start", "End"],
    [
        data["datetime"].min().year,
        data["datetime"].max().year
    ]
)
axes[2].set_title("Temporal Coverage", fontweight="bold")
axes[2].set_ylabel("Year")

plt.suptitle(
    "Auckland Air Quality Dataset Overview",
    fontsize=16,
    fontweight="bold"
)

plt.tight_layout()

plt.savefig(
    os.path.join(
        OUTPUT_DIR,
        "dataset_overview_dashboard.png"
    ),
    dpi=300,
    bbox_inches="tight"
)

.show()
```



In [18]:

```python
# POLLUTANT DISTRIBUTION

fig, axes = plt.subplots(
    1,
    len(available_targets),
    figsize=(14, 5)
)

if len(available_targets) == 1:
    axes = [axes]

for ax, target in zip(axes, available_targets):

    values = data[target].dropna()

    ax.boxplot(
        values,
        vert=True,
        patch_artist=True
    )

    ax.set_title(
        f"{target} Distribution",
        fontweight="bold"
    )

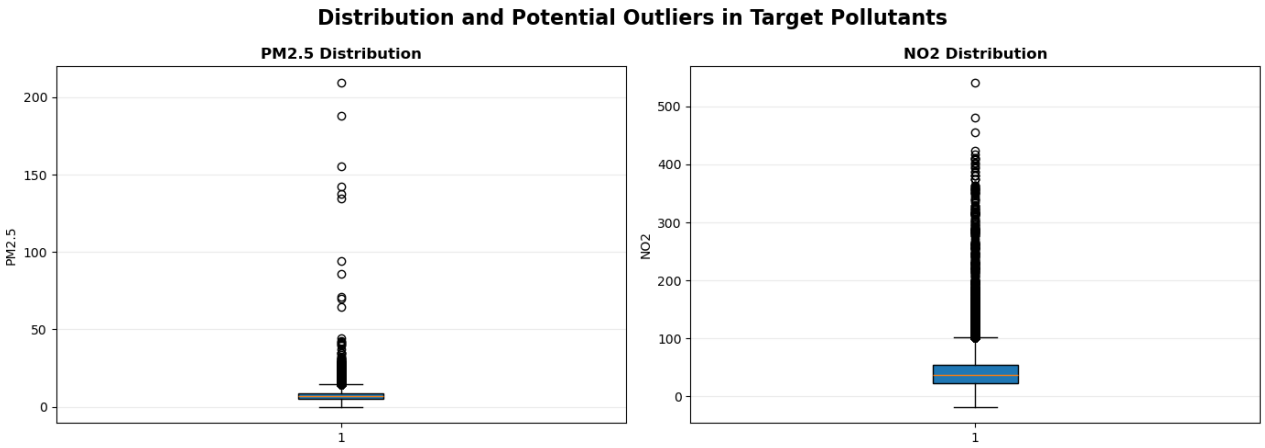
    ax.set_ylabel(target)
    ax.grid(axis="y", alpha=0.25)

plt.suptitle(
    "Distribution and Potential Outliers in Target Pollutants",
    fontsize=16,
    fontweight="bold"
)

plt.tight_layout()

plt.savefig(
    os.path.join(
        OUTPUT_DIR,
        "pollutant_distribution_boxplots.png"
    ),
    dpi=300,
    bbox_inches="tight"
)

plt.show()
```


Distribution and Potential Outliers in Target Pollutants
PM2.5 Distribution
PM2.5
200
150
100
50
0
1
NO2 Distribution
NO2
500
400
300
200
100
0
1

In [20]:

```python
# 1. CONFIGURE OUTPUT DIRECTORY

OUTPUT_DIR = r"D:\Auckland_Air_Quality_MVP"

# Create the folder if it does not exist
os.makedirs(OUTPUT_DIR, exist_ok=True)

print("Output folder:", os.path.abspath(OUTPUT_DIR))
print("Folder exists:", os.path.isdir(OUTPUT_DIR))

# 2. VALIDATE REQUIRED DATA

if "data" not in globals():
    raise NameError(
        "The DataFrame 'data' was not found. "
        "Run the cell that loads your dataset first."
    )

if "datetime" not in data.columns:
    raise KeyError(
        "The 'datetime' column is missing. "
        "Check your dataset's column names."
    )

# Convert datetime to the correct format
data["datetime"] = pd.to_datetime(
    data["datetime"],
    errors="coerce"
)

# Identify available target variables
# Uses your existing available_targets if it is defined
if "available_targets" in globals():
    available_targets = [
        col for col in available_targets
        if col in data.columns and col != "datetime"
    ]
else:
    # Automatically detect numeric measurement columns
    available_targets = [
        col for col in data.select_dtypes(include="number").columns
        if col != "datetime"
    ]

if not available_targets:
    raise ValueError(
        "No valid numeric target columns were found. "
        "Check your dataset and available_targets."
    )

print("\nTargets to plot:", available_targets)

# 3. CREATE TIME-SERIES PLOTS

successful_plots = []
skipped_targets = []

for target in available_targets:
```

```
try:
    # Select and clean the required columns
    plot_data = data[["datetime", target]].copy()

    plot_data["datetime"] = pd.to_datetime(
        plot_data["datetime"],
        errors="coerce"
    )

    plot_data[target] = pd.to_numeric(
        plot_data[target],
        errors="coerce"
    )

    # Remove invalid timestamps and missing measurements
    plot_data = plot_data.dropna(
        subset=["datetime", target]
    )

    # Sort observations chronologically
    plot_data = plot_data.sort_values(
        "datetime"
    ).reset_index(drop=True)

    # Skip targets without usable observations
    if plot_data.empty:
        print(f"Skipping {target}: no valid observations.")
        skipped_targets.append(target)
        continue

    # Calculate rolling mean
    # A 24-row window represents 24 hours only when
    # observations are consistently hourly.
    plot_data["rolling_24h"] = (
        plot_data[target]
        .rolling(window=24, min_periods=1)
        .mean()
    )

    # Create figure
    fig, ax = plt.subplots(figsize=(15, 6))

    ax.plot(
        plot_data["datetime"],
        plot_data[target],
        linewidth=0.7,
        alpha=0.45,
        label="Observed concentration"
    )

    ax.plot(
        plot_data["datetime"],
        plot_data["rolling_24h"],
        linewidth=2,
        label="24-observation rolling mean"
    )

    # Titles and axis labels
    ax.set_title(
        f"{target} Concentration: Time-Series Trend",
        fontsize=15,
        fontweight="bold"
    )
```

```
        ax.set_xlabel("Date")
        ax.set_ylabel(f"{target} concentration")
        ax.legend()
        ax.grid(alpha=0.2)

        fig.autofmt_xdate()
        fig.tight_layout()

        # Create a safe filename
        safe_target = re.sub(
            r'[<>:"/\\|?*]+',
            "_",
            str(target)
        ).strip(" .")

        filename = f"{safe_target}_enhanced_time_series.png"
        output_path = os.path.join(OUTPUT_DIR, filename)

        # Save plot
        fig.savefig(
            output_path,
            dpi=300,
            bbox_inches="tight"
        )

        plt.show()
        plt.close(fig)

        # Confirm that the file was saved
        if os.path.isfile(output_path):
            successful_plots.append(output_path)
            print(f"Saved successfully: {output_path}")
        else:
            print(f"Warning: output file not found for {target}")

    except Exception as error:
        print(f"Error processing {target}: {error}")
        skipped_targets.append(target)

# 4. FINAL SUMMARY

print("\n" + "=" * 55)
print("TIME-SERIES VISUALISATION SUMMARY")
print("=" * 55)

print(f"Plots saved successfully: {len(successful_plots)}")
print(f"Targets skipped or failed: {len(skipped_targets)}")
print(f"Output directory: {os.path.abspath(OUTPUT_DIR)}")

if successful_plots:
    print("\nSaved files:")
    for path in successful_plots:
        print(path)

if skipped_targets:
    print("\nSkipped or failed targets:")
    for target in skipped_targets:
        print(target)

print("\nProcessing completed.")
```

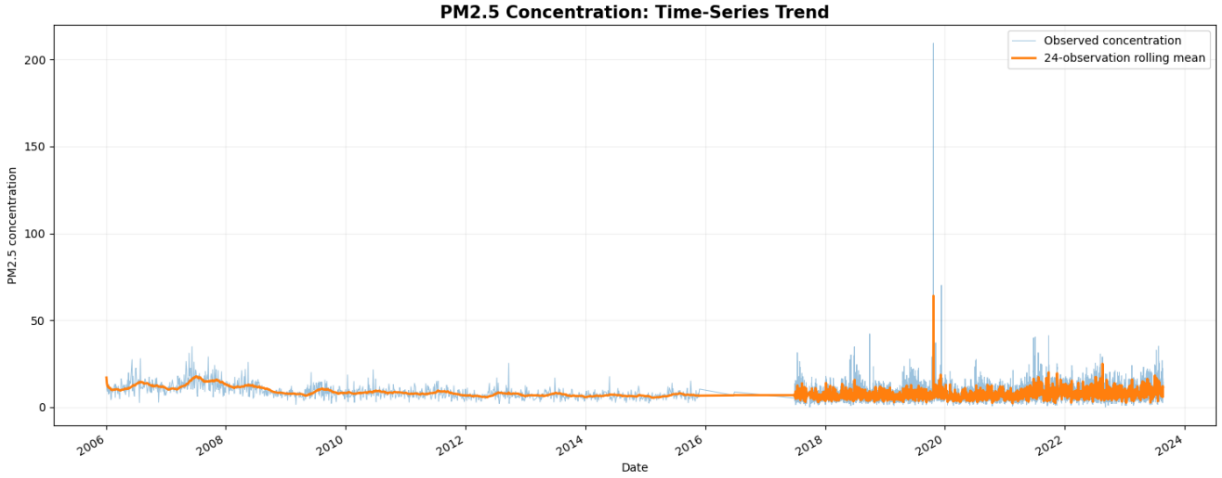
```
Output folder: D:\Auckland_Air_Quality_MVP
Folder exists: True
```

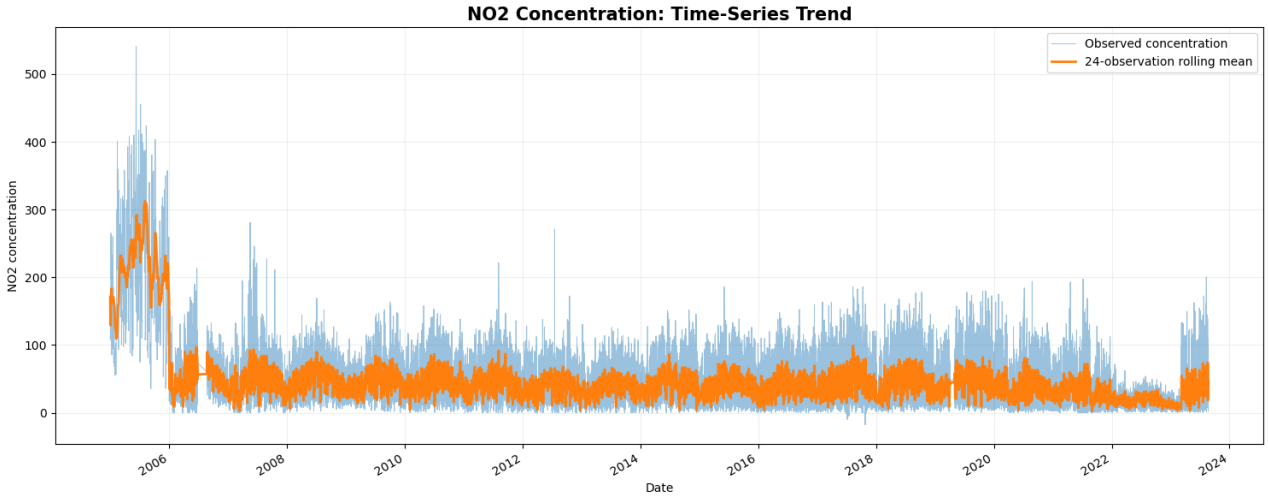
```
Targets to plot: ['PM2.5', 'NO2']
Error processing PM2.5: name 're' is not defined
Error processing NO2: name 're' is not defined

============================================================
TIME-SERIES VISUALISATION SUMMARY
============================================================
Plots saved successfully: 0
Targets skipped or failed: 2
Output directory: D:\Auckland_Air_Quality_MVP

Skipped or failed targets:
PM2.5
NO2

Processing completed.
```





DATA MINING: HOURLY PATTERNS

In [21]:
```python
for target in available_targets:

    hourly_pattern = (
        data.groupby(
            data["datetime"].dt.hour
        )[target]
        .mean()
    )

    plt.figure(figsize=(13, 6))

    plt.plot(
        hourly_pattern.index,
        hourly_pattern.values,
        marker="o",
        linewidth=2.5,
        markersize=6
    )

    # Highlight morning and evening periods
    plt.axvspan(
        7, 9,
        alpha=0.12,
        label="Morning peak window"
    )

    plt.axvspan(
        16, 19,
        alpha=0.12,
        label="Evening peak window"
    )

    plt.xticks(range(24))

    plt.title(
        f"24-Hour Pollution Profile: {target}",
        fontsize=15,
        fontweight="bold"
    )

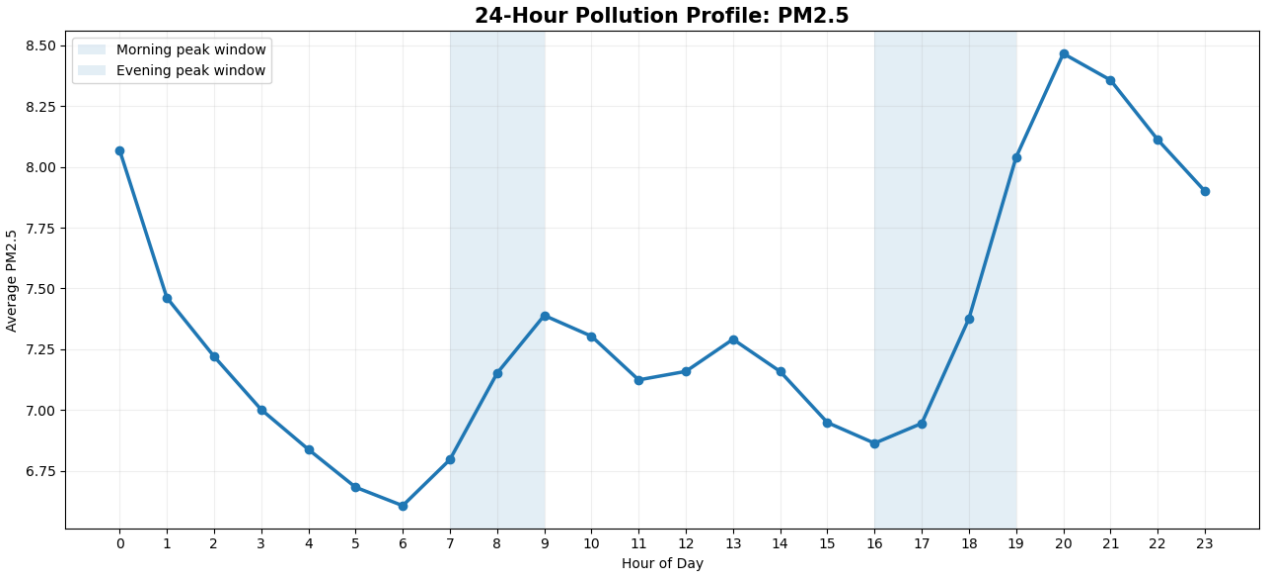
    plt.xlabel("Hour of Day")
    plt.ylabel(f"Average {target}")

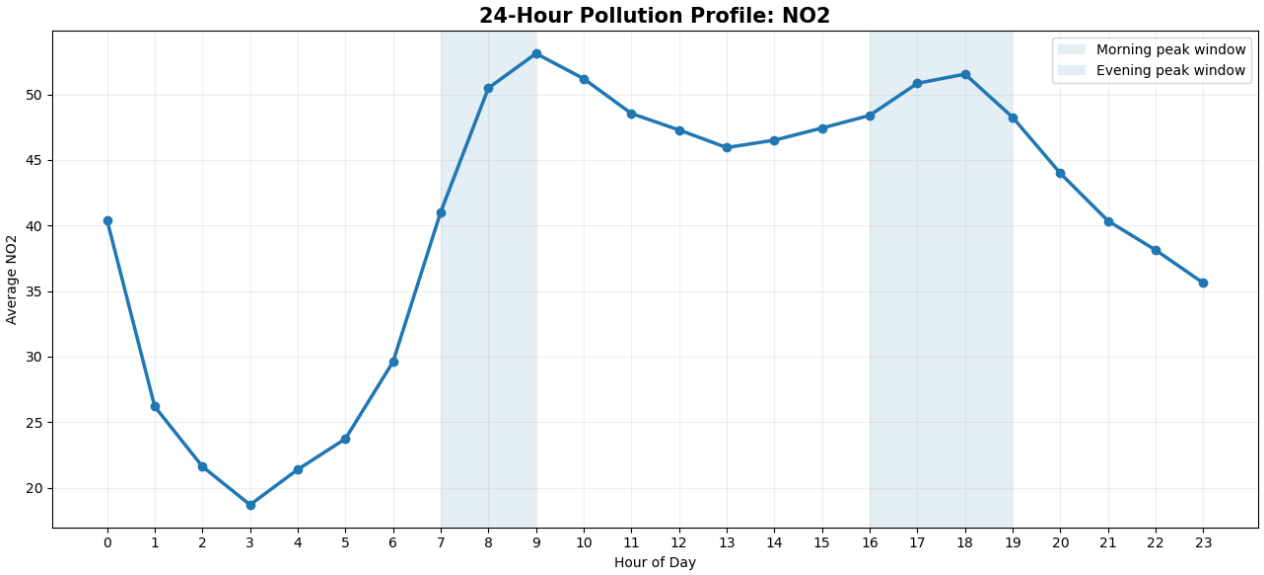
    plt.grid(alpha=0.2)
    plt.legend()

    plt.tight_layout()

    plt.savefig(
        os.path.join(
            OUTPUT_DIR,
            f"{target}_hourly_pattern_enhanced.png"
        ),
        dpi=300,
        bbox_inches="tight"
    )

    plt.show()
```


24-Hour Pollution Profile: PM2.5
Morning peak window
Evening peak window
Average PM2.5
8.50
8.25
8.00
7.75
7.50
7.25
7.00
6.75
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23
Hour of Day


24-Hour Pollution Profile: NO2
Morning peak window
Evening peak window
Average NO2
50
45
40
35
30
25
20
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23
Hour of Day

HOUR × DAY-OF-WEEK POLLUTION HEATMAP

In [22]:
```
for target in available_targets:

    heatmap_data = data.copy()

    heatmap_data["hour"] = (
        heatmap_data["datetime"].dt.hour
    )

    heatmap_data["day_name"] = (
        heatmap_data["datetime"]
        .dt.day_name()
    )

    day_order = [
        "Monday",
        "Tuesday",
        "Wednesday",
        "Thursday",
        "Friday",
        "Saturday",
        "Sunday"
    ]

    heatmap = heatmap_data.pivot_table(
        index="day_name",
        columns="hour",
        values=target,
        aggfunc="mean"
    )

    heatmap = heatmap.reindex(day_order)

    plt.figure(figsize=(15, 6))

    plt.imshow(
        heatmap,
        aspect="auto",
        interpolation="nearest"
    )

    plt.colorbar(
        label=f"Average {target}"
    )

    plt.xticks(
        range(24),
        range(24)
    )

    plt.yticks(
        range(7),
        day_order
    )

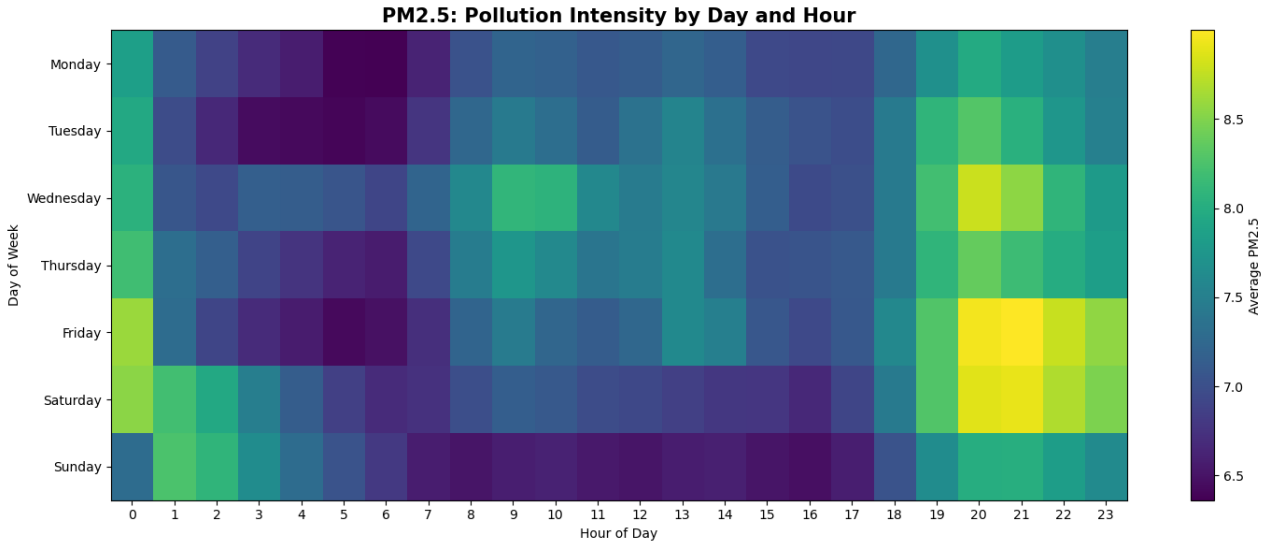
    plt.title(
        f"{target}: Pollution Intensity by Day and Hour",
        fontsize=15,
        fontweight="bold"
    )
```

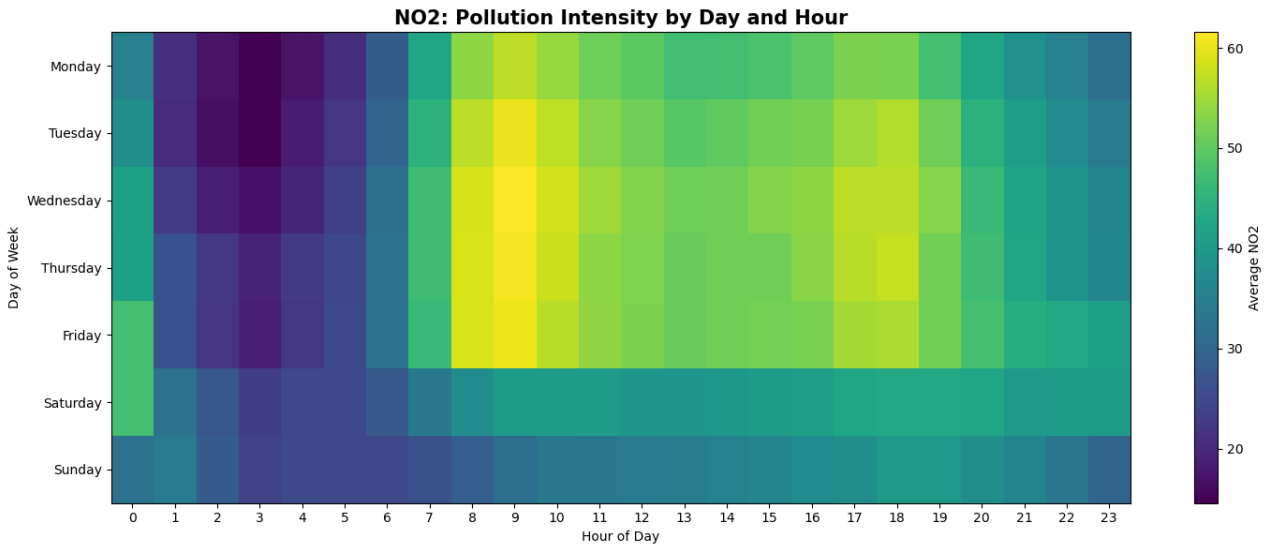
```
plt.xlabel("Hour of Day")
plt.ylabel("Day of Week")

plt.tight_layout()

plt.savefig(
    os.path.join(
        OUTPUT_DIR,
        f"{target}_hour_day_heatmap.png"
    ),
    dpi=300,
    bbox_inches="tight"
)

plt.show()
```





MODEL PIPELINE

In [23]:
```python
# Six models + leakage prevention + walk-forward validation
from sklearn.base import clone
from sklearn.ensemble import (
    RandomForestRegressor,
    GradientBoostingRegressor,
    ExtraTreesRegressor,
    HistGradientBoostingRegressor
)
from sklearn.linear_model import Ridge
from sklearn.neural_network import MLPRegressor
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.model_selection import TimeSeriesSplit
from sklearn.metrics import (
    mean_absolute_error,
    mean_squared_error,
    r2_score
)

OUTPUT_DIR = r"D:\Auckland_Air_Quality_MVP"
os.makedirs(OUTPUT_DIR, exist_ok=True)

# Keep only pollutants that genuinely exist in the data.
TARGETS = [
    t for t in ["PM2.5", "NO2", "O3"]
    if t in data.columns
]
```

In [24]:

```
# ------------------------------------------------------------
# FEATURES: ONLY PAST TARGET AND PAST EXPLANATORY DATA
# ------------------------------------------------------------

def make_model_table(source, target):
    d = source.sort_values("datetime").copy()
    d = d.drop_duplicates("datetime", keep="last")
    d = d.reset_index(drop=True)

    # Use a regular hourly time grid.
    d = d.set_index("datetime").asfreq("h").reset_index()

    # Causal filling of explanatory variables only.
    other_targets = [
        t for t in ["PM2.5", "NO2", "O3"]
        if t in d.columns
    ]

    numeric_cols = d.select_dtypes(
        include=np.number
    ).columns.tolist()

    exog_cols = [
        c for c in numeric_cols
        if c not in other_targets
    ]

    d[exog_cols] = d[exog_cols].ffill(limit=6)

    # Calendar features are known in advance.
    d["hour"] = d["datetime"].dt.hour
    d["dayofweek"] = d["datetime"].dt.dayofweek
    d["month"] = d["datetime"].dt.month
    d["dayofyear"] = d["datetime"].dt.dayofyear

    d["hour_sin"] = np.sin(2 * np.pi * d["hour"] / 24)
    d["hour_cos"] = np.cos(2 * np.pi * d["hour"] / 24)
    d["doy_sin"] = np.sin(
        2 * np.pi * d["dayofyear"] / 365.25
    )
    d["doy_cos"] = np.cos(
        2 * np.pi * d["dayofyear"] / 365.25
    )

    feature_cols = [
        "hour", "dayofweek", "month", "dayofyear",
        "hour_sin", "hour_cos", "doy_sin", "doy_cos"
    ]
```

```
    # Historical target features: shift before rolling.
    for lag in [1, 2, 3, 6, 12, 24, 48, 72]:
        name = f"{target}_lag_{lag}"
        d[name] = d[target].shift(lag)
        feature_cols.append(name)

    for window in [3, 6, 12, 24]:
        mean_name = f"{target}_mean_{window}"
        std_name = f"{target}_std_{window}"

        d[mean_name] = (
            d[target].shift(1).rolling(window).mean()
        )
        d[std_name] = (
            d[target].shift(1).rolling(window).std()
        )

        feature_cols.extend([mean_name, std_name])

    # Other pollutants, weather and traffic are used with a
    # one-hour lag, not their same-hour values.
    for col in exog_cols:
        lag_name = f"{col}_lag1"
        d[lag_name] = d[col].shift(1)
        feature_cols.append(lag_name)

    # Explicit leakage guard.
    assert target not in feature_cols
    assert all(
        not name.startswith(target + "_lag0")
        for name in feature_cols
    )

    # Remove rows without an observed target or sufficient
    # historical information. Do not fill missing targets.
    d = d.replace([np.inf, -np.inf], np.nan)
    d = d.dropna(subset=[target] + feature_cols).copy()

    return d, feature_cols
```

```python
# ------------------------------------------------------------
# SIX MODELS
# ------------------------------------------------------------

def build_models():
    return {
        "Random Forest": RandomForestRegressor(
            n_estimators=200,
            max_depth=15,
            min_samples_leaf=2,
            random_state=42,
            n_jobs=-1
        ),

        "Gradient Boosting": GradientBoostingRegressor(
            n_estimators=150,
            learning_rate=0.05,
            max_depth=3,
            loss="huber",
            random_state=42
        ),

        "Neural Network": Pipeline([
            ("imputer", SimpleImputer(strategy="median")),
            ("scaler", StandardScaler()),
            ("model", MLPRegressor(
                hidden_layer_sizes=(64, 32),
                activation="relu",
                max_iter=300,
                early_stopping=False,
                random_state=42
            ))
        ]),

        "Extra Trees": ExtraTreesRegressor(
            n_estimators=200,
            max_depth=15,
            min_samples_leaf=2,
            random_state=42,
            n_jobs=-1
        ),

        "Hist Gradient Boosting": HistGradientBoostingRegressor(
            max_iter=150,
            learning_rate=0.05,
            max_leaf_nodes=15,
            l2_regularization=1.0,
            random_state=42
        ),

        "Ridge Regression": Pipeline([
            ("imputer", SimpleImputer(strategy="median")),
            ("scaler", StandardScaler()),
            ("model", Ridge(alpha=10.0))
        ])
    }
```

```
# ------------------------------------------------------------
# METRICS
# ------------------------------------------------------------

def calculate_metrics(y_true, y_pred):
    return {
        "MAE": mean_absolute_error(y_true, y_pred),
        "RMSE": np.sqrt(mean_squared_error(y_true, y_pred)),
        "R2": (
            r2_score(y_true, y_pred)
            if len(y_true) > 1 else np.nan
        )
    }


# ------------------------------------------------------------
# WALK-FORWARD VALIDATION + FINAL HOLDOUT
# ------------------------------------------------------------

all_test_results = []
all_cv_results = []
all_predictions = []
model_objects = {}

for target in TARGETS:

    print(f"\n========== {target} ==========")

    model_table, features = make_model_table(data, target)

    X = model_table[features]
    y = model_table[target]
    timestamps = model_table["datetime"]

    n = len(model_table)

    if n < 100:
        print(f"Insufficient modelling rows for {target}: {n}")
        continue

    # Final 15% is reserved as untouched test data.
    test_start = int(n * 0.85)

    X_dev, y_dev = X.iloc[:test_start], y.iloc[:test_start]
    X_test, y_test = X.iloc[test_start:], y.iloc[test_start:]

    time_test = timestamps.iloc[test_start:]

    # Walk-forward validation uses only development data.
    splitter = TimeSeriesSplit(n_splits=5)
    cv_scores = []
    models = build_models()

    for model_name, prototype in models.items():

        fold_number = 0

        for train_idx, val_idx in splitter.split(X_dev):

            fold_number += 1
            model = clone(prototype)
```

```
        X_train_fold = X_dev.iloc[train_idx]
        y_train_fold = y_dev.iloc[train_idx]
        X_val_fold = X_dev.iloc[val_idx]
        y_val_fold = y_dev.iloc[val_idx]

        model.fit(X_train_fold, y_train_fold)
        val_pred = model.predict(X_val_fold)

        metrics = calculate_metrics(y_val_fold, val_pred)

        cv_scores.append({
            "Pollutant": target,
            "Model": model_name,
            "Fold": fold_number,
            **metrics
        })

cv_df = pd.DataFrame(cv_scores)
all_cv_results.append(cv_df)

# Select using mean walk-forward RMSE only.
cv_summary = (
    cv_df.groupby(["Pollutant", "Model"], as_index=False)
    [["MAE", "RMSE", "R2"]]
    .mean()
    .rename(columns={
        "MAE": "CV_MAE",
        "RMSE": "CV_RMSE",
        "R2": "CV_R2"
    })
)

best_name = (
    cv_summary.sort_values("CV_RMSE")
    .iloc[0]["Model"]
)

print("Selected from walk-forward validation:", best_name)

# Evaluate every model on the same final chronological test.
fitted_models = {}

for model_name, prototype in models.items():

    model = clone(prototype)
    model.fit(X_dev, y_dev)

    pred = model.predict(X_test)
    metrics = calculate_metrics(y_test, pred)

    all_test_results.append({
        "Pollutant": target,
        "Model": model_name,
        "Train_Observations": len(X_dev),
        "Test_Observations": len(X_test),
        "Test_Start": time_test.iloc[0],
        "Test_End": time_test.iloc[-1],
        **metrics
    })
```

```
        prediction_rows = pd.DataFrame({
            "datetime": time_test.values,
            "Pollutant": target,
            "Model": model_name,
            "Actual": y_test.values,
            "Predicted": pred
        })

        all_predictions.append(prediction_rows)
        fitted_models[model_name] = model

    # Refit selected model on all available eligible observations
    # for use in the future forecast only.
    final_model = clone(models[best_name])
    final_model.fit(X, y)

    model_objects[target] = {
        "model": final_model,
        "model_name": best_name,
        "features": features,
        "history": data.sort_values("datetime").copy(),
        "model_table": model_table
    }

    print(cv_summary.sort_values("CV_RMSE").to_string(index=False))
```

```
========== PM2.5 ==========
Selected from walk-forward validation: Extra Trees
Pollutant                 Model   CV_MAE  CV_RMSE    CV_R2
    PM2.5           Extra Trees 1.108544 1.738572 0.728976
    PM2.5 Hist Gradient Boosting 1.113815 1.751670 0.724505
    PM2.5          Random Forest 1.127469 1.769904 0.719900
    PM2.5      Gradient Boosting 1.126625 1.793698 0.711515
    PM2.5       Ridge Regression 1.178290 1.807279 0.708944
    PM2.5         Neural Network 1.663409 2.370730 0.479436

========== NO2 ==========
Selected from walk-forward validation: Extra Trees
Pollutant                 Model    CV_MAE   CV_RMSE     CV_R2
      NO2           Extra Trees  6.532988  9.003493  0.643608
      NO2          Random Forest  6.561182  9.098245  0.645176
      NO2      Gradient Boosting  6.548966  9.127713  0.643487
      NO2 Hist Gradient Boosting  6.682957  9.200446  0.628532
      NO2       Ridge Regression  8.123337 10.496627  0.493845
      NO2         Neural Network 11.944186 15.094704 -0.082369
```

EXPORT EVALUATION TABLES

In [25]:
```
model_results = pd.DataFrame(all_test_results)
walk_forward_results = pd.concat(
    all_cv_results, ignore_index=True
)
prediction_results = pd.concat(
    all_predictions, ignore_index=True
)

cv_summary_all = (
    walk_forward_results
    .groupby(["Pollutant", "Model"], as_index=False)
    [["MAE", "RMSE", "R2"]]
    .mean()
    .rename(columns={
        "MAE": "CV_MAE",
        "RMSE": "CV_RMSE",
        "R2": "CV_R2"
    })
)

model_results = model_results.merge(
    cv_summary_all,
    on=["Pollutant", "Model"],
    how="left"
)

model_results.to_csv(
    os.path.join(OUTPUT_DIR, "model_comparison.csv"),
    index=False
)

walk_forward_results.to_csv(
    os.path.join(OUTPUT_DIR, "walk_forward_results.csv"),
    index=False
)

prediction_results.to_csv(
    os.path.join(OUTPUT_DIR, "test_predictions.csv"),
    index=False
)

print("\nFINAL HOLDOUT MODEL COMPARISON")
print(
    model_results.sort_values(
        ["Pollutant", "RMSE"]
    ).to_string(index=False)
)
```

```
FINAL HOLDOUT MODEL COMPARISON
Pollutant                 Model  Train_Observations  Test_Observations          Test_Start            Test_End      MAE     RMSE       R2    CV_MAE   CV_RMSE     CV_R2
      NO2 Hist Gradient Boosting                6711               1185 2022-10-15 23:00:00 2023-02-28 12:00:00 2.796586 3.662421 0.693690  6.682957  9.200446  0.628532
      NO2      Gradient Boosting                6711               1185 2022-10-15 23:00:00 2023-02-28 12:00:00 2.851642 3.690969 0.688896  6.548966  9.127713  0.643487
      NO2          Random Forest                6711               1185 2022-10-15 23:00:00 2023-02-28 12:00:00 3.018983 3.888791 0.654654  6.561182  9.098245  0.645176
      NO2            Extra Trees                6711               1185 2022-10-15 23:00:00 2023-02-28 12:00:00 3.084981 3.939201 0.645643  6.532988  9.003493  0.643608
      NO2       Ridge Regression                6711               1185 2022-10-15 23:00:00 2023-02-28 12:00:00 3.845315 4.814247 0.470724  8.123337 10.496627  0.493845
      NO2         Neural Network                6711               1185 2022-10-15 23:00:00 2023-02-28 12:00:00 4.510964 5.806199 0.230145 11.944186 15.094704 -0.082369
    PM2.5            Extra Trees               14803               2613 2022-10-25 11:00:00 2023-02-28 12:00:00 0.954209 1.407619 0.782864  1.108544  1.738572  0.728976
    PM2.5 Hist Gradient Boosting               14803               2613 2022-10-25 11:00:00 2023-02-28 12:00:00 0.971355 1.429569 0.776040  1.113815  1.751670  0.724505
    PM2.5          Random Forest               14803               2613 2022-10-25 11:00:00 2023-02-28 12:00:00 0.985751 1.440607 0.772568  1.127469  1.769904  0.719900
    PM2.5      Gradient Boosting               14803               2613 2022-10-25 11:00:00 2023-02-28 12:00:00 0.977685 1.456604 0.767489  1.126625  1.793698  0.711515
    PM2.5       Ridge Regression               14803               2613 2022-10-25 11:00:00 2023-02-28 12:00:00 1.065898 1.578104 0.727082  1.178290  1.807279  0.708944
    PM2.5         Neural Network               14803               2613 2022-10-25 11:00:00 2023-02-28 12:00:00 1.174591 1.661837 0.697352  1.663409  2.370730  0.479436
```

SHAP EXPLAINABILITY

In [26]:
```
pip install shap
```

```
Defaulting to user installation because normal site-packages is not writeable
Requirement already satisfied: shap in c:\users\alorg\appdata\roaming\python\python313\site-packages (0.52.0)
Requirement already satisfied: numpy>=2 in c:\programdata\anaconda3\lib\site-packages (from shap) (2.3.5)
Requirement already satisfied: scipy in c:\programdata\anaconda3\lib\site-packages (from shap) (1.16.3)
Requirement already satisfied: scikit-learn in c:\programdata\anaconda3\lib\site-packages (from shap) (1.7.2)
Requirement already satisfied: pandas in c:\programdata\anaconda3\lib\site-packages (from shap) (2.3.3)
Requirement already satisfied: tqdm>=4.27.0 in c:\programdata\anaconda3\lib\site-packages (from shap) (4.67.1)
Requirement already satisfied: packaging>20.9 in c:\programdata\anaconda3\lib\site-packages (from shap) (25.0)
Requirement already satisfied: slicer==0.0.8 in c:\users\alorg\appdata\roaming\python\python313\site-packages (from shap) (0.0.8)
Requirement already satisfied: numba in c:\programdata\anaconda3\lib\site-packages (from shap) (0.62.1)
Requirement already satisfied: llvmlite in c:\programdata\anaconda3\lib\site-packages (from shap) (0.45.1)
Requirement already satisfied: cloudpickle in c:\programdata\anaconda3\lib\site-packages (from shap) (3.1.1)
Requirement already satisfied: colorama in c:\programdata\anaconda3\lib\site-packages (from tqdm>=4.27.0->shap) (0.4.6)
Requirement already satisfied: python-dateutil>=2.8.2 in c:\programdata\anaconda3\lib\site-packages (from pandas->shap) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in c:\programdata\anaconda3\lib\site-packages (from pandas->shap) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in c:\programdata\anaconda3\lib\site-packages (from pandas->shap) (2025.2)
Requirement already satisfied: six>=1.5 in c:\programdata\anaconda3\lib\site-packages (from python-dateutil>=2.8.2->pandas->shap) (1.17.0)
Requirement already satisfied: joblib>=1.2.0 in c:\programdata\anaconda3\lib\site-packages (from scikit-learn->shap) (1.5.2)
Requirement already satisfied: threadpoolctl>=3.1.0 in c:\programdata\anaconda3\lib\site-packages (from scikit-learn->shap) (3.5.0)
Note: you may need to restart the kernel to use updated packages.
```

In [28]:
```
# Check whether SHAP is available
if SHAP_AVAILABLE:

    # Create output folder if it does not exist
    os.makedirs(OUTPUT_DIR, exist_ok=True)

    print("Available model objects:")
    for target, obj in model_objects.items():
        print(f"\nTarget: {target}")
        if isinstance(obj, dict):
            print("Keys:", list(obj.keys()))
        else:
            print("Object type:", type(obj).__name__)

    for target, obj in model_objects.items():

        # Check dictionary structure
        if not isinstance(obj, dict):
            print(f"Skipping {target}: model entry is not a dictionary.")
            continue

        # Support common alternative key names
        model = obj.get(
            "best_model",
            obj.get("model", obj.get("best_estimator"))
        )

        features = obj.get(
            "features",
            obj.get("feature_columns", obj.get("X_columns"))
        )

        model_data = obj.get(
            "data",
            obj.get("model_data", obj.get("df"))
        )
```

```
        # Skip entries with missing information
        if model is None:
            print(
                f"Skipping {target}: no trained model found. "
                f"Available keys: {list(obj.keys())}"
            )
            continue

        if features is None or model_data is None:
            print(
                f"Skipping {target}: features or data are missing. "
                f"Available keys: {list(obj.keys())}"
            )
            continue

        # Validate feature columns
        features = list(features)
        missing_features = [
            col for col in features
            if col not in model_data.columns
        ]

        if missing_features:
            print(
                f"Skipping {target}: missing columns "
                f"{missing_features}"
            )
            continue

        # Select sample and remove invalid values
        X_sample = model_data[features].tail(500).copy()
        X_sample = X_sample.apply(
            lambda col: __import__("pandas").to_numeric(
                col, errors="coerce"
            )
        )
        X_sample = X_sample.dropna()

        if X_sample.empty:
            print(f"Skipping {target}: no valid feature rows.")
            continue

        # SHAP TreeExplainer for supported tree-based models
        if isinstance(
            model,
            (
                RandomForestRegressor,
                GradientBoostingRegressor
            )
        ):
            try:
                explainer = shap.TreeExplainer(model)
                shap_values = explainer.shap_values(X_sample)

                plt.figure()
                shap.summary_plot(
                    shap_values,
                    X_sample,
                    show=False
                )
```

```
                plt.tight_layout()

                safe_target = re.sub(
                    r'[<>:"/\\|?*]+', "_", str(target)
                )

                output_path = os.path.join(
                    OUTPUT_DIR,
                    f"{safe_target}_SHAP.png"
                )

                plt.savefig(
                    output_path,
                    dpi=300,
                    bbox_inches="tight"
                )
                plt.show()
                plt.close()

                print(f"SHAP plot saved: {output_path}")

            except Exception as e:
                print(f"SHAP failed for {target}: {e}")

        else:
            print(
                f"Skipping SHAP for {target}: "
                f"unsupported model type {type(model).__name__}. "
                "This section currently supports "
                "RandomForestRegressor and "
                "GradientBoostingRegressor."
            )

else:
    print(
        "\nSHAP is not installed. Install it with:\n"
        "!pip install shap"
    )
```

```
Available model objects:

Target: PM2.5
Keys: ['model', 'model_name', 'features', 'history', 'model_table']

Target: NO2
Keys: ['model', 'model_name', 'features', 'history', 'model_table']
Skipping PM2.5: features or data are missing. Available keys: ['model', 'model_name', 'features', 'history', 'model_table']
Skipping NO2: features or data are missing. Available keys: ['model', 'model_name', 'features', 'history', 'model_table']
```

24-HOUR FORECAST FUNCTION

In [31]:
```
required_columns = [
    "AQI",
    "CITY_CENTRE_TVCOUNT",
    "TOTAL_PEDESTRIANS",
    "TRAFFICV",
    "traffic_volume"
]

print("Dataset columns:")
print(data.columns.tolist())

print("\nRequired source columns:")
for col in required_columns:
    if col in data.columns:
        print(f"FOUND: {col} | dtype: {data[col].dtype}")
    else:
        print(f"MISSING: {col}")
```

```
Dataset columns:
['datetime', 'AQI', 'CITY_CENTRE_TVCOUNT', 'NO', 'NO2', 'NOx', 'PM10', 'PM2.5', 'RH', 'TEMP', 'TOTAL_PEDESTRIANS', 'TRAFFICV', 'WD', 'WS', 'traffic_volume']

Required source columns:
FOUND: AQI | dtype: float64
FOUND: CITY_CENTRE_TVCOUNT | dtype: float64
FOUND: TOTAL_PEDESTRIANS | dtype: float64
FOUND: TRAFFICV | dtype: float64
FOUND: traffic_volume | dtype: float64
```

In [35]:
```
# ------------------------------------------------------------
# 1. Validate inputs
# ------------------------------------------------------------

os.makedirs(OUTPUT_DIR, exist_ok=True)

if "datetime" not in data.columns:
    raise ValueError("The dataset must contain a 'datetime' column.")

data = data.copy()
data["datetime"] = pd.to_datetime(data["datetime"], errors="coerce")
data = data.dropna(subset=["datetime"])
data = data.sort_values("datetime")
data = data.drop_duplicates(subset=["datetime"], keep="last")

if not model_objects:
    raise ValueError("model_objects is empty. Train your models first.")
```

```python
# ------------------------------------------------------------
# 2. Forecasting function
# ------------------------------------------------------------

def forecast_next_24_hours(source, target, model, features):

    raw = source.copy()
    features = list(features)

    if target not in raw.columns:
        raise ValueError(
            f"Target '{target}' is not present in the dataset."
        )

    raw["datetime"] = pd.to_datetime(
        raw["datetime"], errors="coerce"
    )
    raw = raw.dropna(subset=["datetime"])
    raw = raw.sort_values("datetime")
    raw = raw.drop_duplicates("datetime", keep="last")

    # Convert all non-date columns to numeric where possible
    for col in raw.columns:
        if col != "datetime":
            raw[col] = pd.to_numeric(raw[col], errors="coerce")

    # Preserve the location of the latest actual target value
    raw = (
        raw.set_index("datetime")
        .asfreq("h")
        .reset_index()
    )

    actual_target = raw[target].notna()
    observed_positions = np.flatnonzero(actual_target.to_numpy())

    if len(observed_positions) < 73:
        raise ValueError(
            f"{target}: only {len(observed_positions)} actual "
            "observations are available. At least 73 are required."
        )

    origin_idx = observed_positions[-1]
    history = raw.iloc[:origin_idx + 1].copy().reset_index(drop=True)

    # Forward-fill using past values only.
    # This handles short or long gaps without using future observations.
    numeric_cols = history.select_dtypes(
        include=np.number
    ).columns.tolist()

    history[numeric_cols] = history[numeric_cols].ffill()

    forecasts = []
```

```python
# ------------------------------------------------------------
# 3. Forecast the next 24 hours recursively
# ------------------------------------------------------------

for step in range(24):

    next_time = (
        history["datetime"].iloc[-1]
        + pd.Timedelta(hours=1)
    )

    hour = next_time.hour
    dayofweek = next_time.dayofweek
    month = next_time.month
    dayofyear = next_time.dayofyear

    time_features = {
        "hour": hour,
        "dayofweek": dayofweek,
        "month": month,
        "dayofyear": dayofyear,
        "hour_sin": np.sin(2 * np.pi * hour / 24),
        "hour_cos": np.cos(2 * np.pi * hour / 24),
        "doy_sin": np.sin(2 * np.pi * dayofyear / 365.25),
        "doy_cos": np.cos(2 * np.pi * dayofyear / 365.25),
    }

    row = {}

    # Generate exactly the feature names expected by the model
    for feature in features:

        # Time features
        if feature in time_features:
            row[feature] = time_features[feature]
            continue

        # Rolling features: e.g. PM2.5_mean_6, PM2.5_std_24
        rolling_match = re.fullmatch(
            r"(.+)_(mean|std)_(\d+)", feature
        )

        if rolling_match:
            base_col = rolling_match.group(1)
            statistic = rolling_match.group(2)
            window = int(rolling_match.group(3))

            if base_col not in history.columns:
                row[feature] = np.nan
                continue

            series = history[base_col].ffill().dropna()
            values = series.tail(window)

            if len(values) < window:
                row[feature] = np.nan
            elif statistic == "mean":
                row[feature] = float(values.mean())
            else:
                row[feature] = float(values.std(ddof=1))

            continue
```

```
        # Lag features: e.g. AQI_lag1, PM2.5_lag_1
        lag_match = re.fullmatch(
            r"(.+)_lag_?(\d+)", feature
        )

        if lag_match:
            base_col = lag_match.group(1)
            lag = int(lag_match.group(2))

            if base_col not in history.columns:
                row[feature] = np.nan
                continue

            series = history[base_col].ffill()

            if len(series) >= lag:
                value = series.iloc[-lag]
                row[feature] = (
                    float(value) if pd.notna(value) else np.nan
                )
            else:
                row[feature] = np.nan

            continue

        # Direct features, if any, use the latest known value
        if feature in history.columns:
            value = history[feature].iloc[-1]
            row[feature] = (
                float(value) if pd.notna(value) else np.nan
            )
        else:
            row[feature] = np.nan

    # Keep exactly the training feature order
    X_future = pd.DataFrame(
        [[row.get(col, np.nan) for col in features]],
        columns=features
    )

    X_future = X_future.replace(
        [np.inf, -np.inf], np.nan
    )

    if X_future.isna().any().any():
        missing = X_future.columns[
            X_future.isna().any()
        ].tolist()

        raise ValueError(
            f"{target}: these training features could not be "
            f"constructed: {missing}. Check whether these columns "
            "exist in the source data and whether the same feature "
            "engineering was used during model training."
        )

    prediction = float(model.predict(X_future)[0])

    forecasts.append({
        "datetime": next_time,
        "Pollutant": target,
        "Forecast": prediction,
        "Forecast_Step": step + 1,
        "Model": model_objects[target].get(
            "model_name", type(model).__name__
        ),
```

```
            ),
            "Forecast_Assumption":
                "Recursive target forecast; other variables held "
                "at their latest available values"
        })

        # Append the forecast hour.
        # Other variables retain their last known values.
        new_row = history.iloc[-1].copy()
        new_row["datetime"] = next_time
        new_row[target] = prediction

        history = pd.concat(
            [history, new_row.to_frame().T],
            ignore_index=True
        )

        history["datetime"] = pd.to_datetime(history["datetime"])

        for col in history.columns:
            if col != "datetime":
                history[col] = pd.to_numeric(
                    history[col], errors="coerce"
                )

        # Fill any remaining gaps using past values only
        numeric_cols = history.select_dtypes(
            include=np.number
        ).columns.tolist()

        history[numeric_cols] = history[numeric_cols].ffill()

    return pd.DataFrame(forecasts)


# ------------------------------------------------------------
# 4. Generate all pollutant forecasts
# ------------------------------------------------------------

forecast_frames = []

for target, obj in model_objects.items():

    if not isinstance(obj, dict):
        print(f"Skipping {target}: model entry is not a dictionary.")
        continue

    if "model" not in obj or "features" not in obj:
        print(
            f"Skipping {target}: 'model' or 'features' is missing. "
            f"Available keys: {list(obj.keys())}"
        )
        continue

    print(f"Forecasting {target}...")

    forecast_part = forecast_next_24_hours(
        data,
        target,
        obj["model"],
        obj["features"]
    )
```

```
    forecast_frames.append(forecast_part)

if not forecast_frames:
    raise ValueError(
        "No forecasts were generated. Check model_objects."
    )

forecast_long = pd.concat(
    forecast_frames, ignore_index=True
)

forecast_24 = (
    forecast_long.pivot(
        index="datetime",
        columns="Pollutant",
        values="Forecast"
    )
    .reset_index()
)

# ------------------------------------------------------------
# 5. Export forecast results
# ------------------------------------------------------------

forecast_long.to_csv(
    os.path.join(OUTPUT_DIR, "forecast_24h_long.csv"),
    index=False
)

forecast_24.to_csv(
    os.path.join(OUTPUT_DIR, "24_hour_air_quality_forecast.csv"),
    index=False
)

forecast_24.to_excel(
    os.path.join(OUTPUT_DIR, "24_hour_air_quality_forecast.xlsx"),
    index=False
)

print("\n24-HOUR AIR QUALITY FORECAST")
print(forecast_24.to_string(index=False))

# ------------------------------------------------------------
# 6. Summary statistics
# ------------------------------------------------------------

forecast_summary = (
    forecast_long.groupby(
        ["Pollutant", "Model"], as_index=False
    )
    .agg(
        Forecast_Mean=("Forecast", "mean"),
        Forecast_Max=("Forecast", "max"),
        Forecast_Min=("Forecast", "min"),
        Forecast_Std=("Forecast", "std"),
        Forecast_Hours=("Forecast", "count")
    )
)

forecast_summary.to_csv(
    os.path.join(OUTPUT_DIR, "forecast_summary.csv"),
    index=False
)
```

```
print("\nFORECAST SUMMARY")
print(forecast_summary.to_string(index=False))

# ------------------------------------------------------------
# 7. Plot historical values and 24-hour forecasts
# ------------------------------------------------------------

for target in forecast_long["Pollutant"].unique():

    if target not in data.columns:
        continue

    history_plot = (
        data[["datetime", target]]
        .dropna()
        .sort_values("datetime")
        .tail(72)
    )

    future_plot = (
        forecast_long[
            forecast_long["Pollutant"] == target
        ]
        .sort_values("datetime")
    )

    plt.figure(figsize=(14, 5))

    plt.plot(
        history_plot["datetime"],
        history_plot[target],
        label="Historical observations"
    )

    plt.plot(
        future_plot["datetime"],
        future_plot["Forecast"],
        marker="o",
        linestyle="--",
        label="24-hour forecast"
    )

    if not future_plot.empty:
        plt.axvline(
            future_plot["datetime"].iloc[0],
            linestyle=":",
            label="Forecast origin"
        )

    plt.title(f"{target}: Historical and 24-Hour Forecast")
    plt.xlabel("Date and time")
    plt.ylabel(target)
    plt.legend()
    plt.xticks(rotation=30)
    plt.tight_layout()

    safe_target = re.sub(
        r'[<>:"/\\|?*]+', "_", str(target)
    )
```

```
    plt.savefig(
        os.path.join(
            OUTPUT_DIR,
            f"{safe_target}_24_hour_forecast.png"
        ),
        dpi=300,
        bbox_inches="tight"
    )

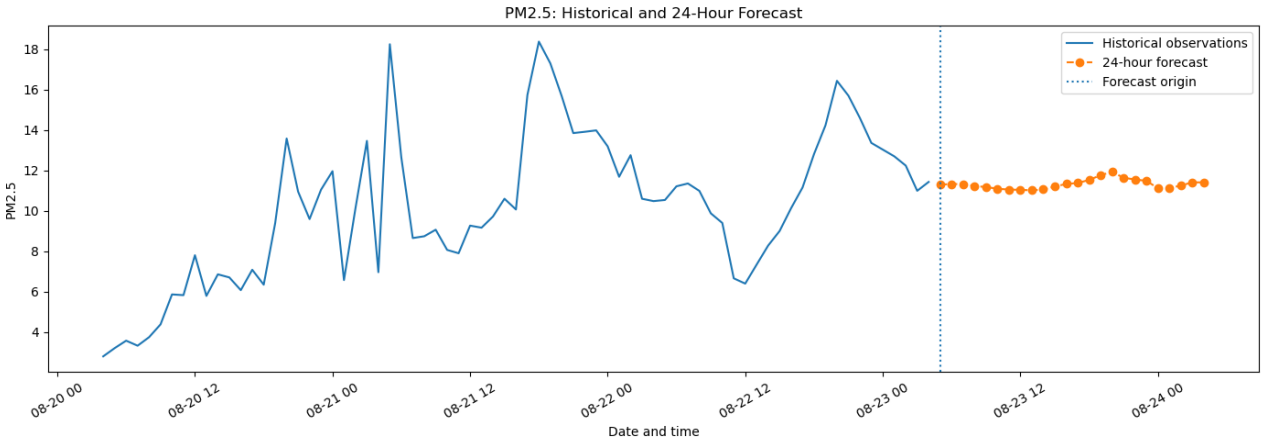
    plt.show()
    plt.close()

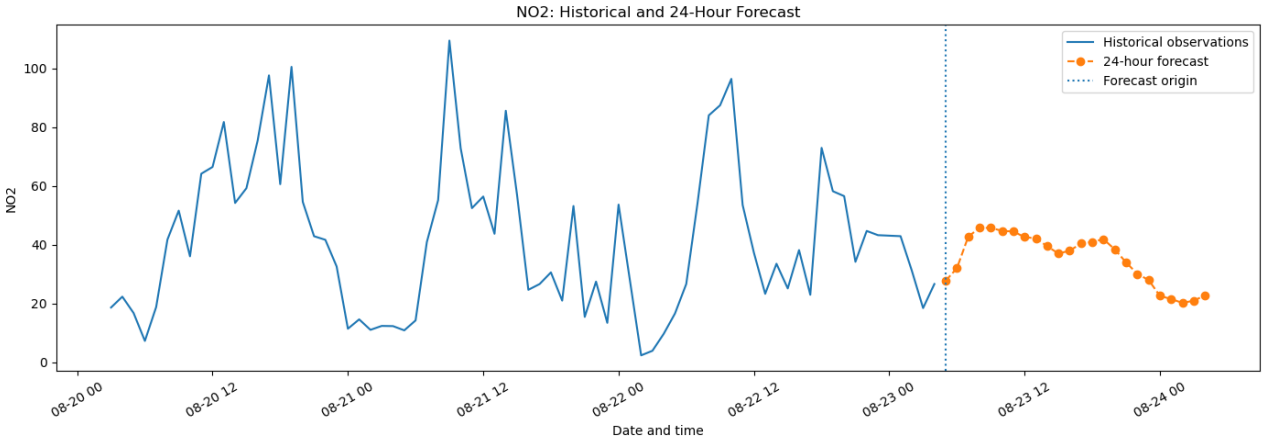
print(f"\nAll forecast files saved to: {OUTPUT_DIR}")
```

```
Forecasting PM2.5...
Forecasting NO2...

24-HOUR AIR QUALITY FORECAST
             datetime       NO2     PM2.5
2023-08-23 05:00:00 27.758516 11.290417
2023-08-23 06:00:00 32.024053 11.322567
2023-08-23 07:00:00 42.735139 11.318339
2023-08-23 08:00:00 45.760511 11.230347
2023-08-23 09:00:00 45.747299 11.167202
2023-08-23 10:00:00 44.735177 11.091345
2023-08-23 11:00:00 44.525660 11.044268
2023-08-23 12:00:00 42.602818 11.035936
2023-08-23 13:00:00 42.186919 11.006224
2023-08-23 14:00:00 39.509472 11.051569
2023-08-23 15:00:00 37.011608 11.196617
2023-08-23 16:00:00 37.991655 11.324822
2023-08-23 17:00:00 40.409605 11.371773
2023-08-23 18:00:00 40.990378 11.523920
2023-08-23 19:00:00 41.895674 11.756492
2023-08-23 20:00:00 38.406972 11.924355
2023-08-23 21:00:00 33.972629 11.629722
2023-08-23 22:00:00 30.076096 11.547087
2023-08-23 23:00:00 27.999278 11.476764
2023-08-24 00:00:00 22.589007 11.111250
2023-08-24 01:00:00 21.527802 11.120286
2023-08-24 02:00:00 20.213008 11.245310
2023-08-24 03:00:00 20.983346 11.404232
2023-08-24 04:00:00 22.782286 11.403338

FORECAST SUMMARY
Pollutant       Model  Forecast_Mean  Forecast_Max  Forecast_Min  Forecast_Std  Forecast_Hours
      NO2 Extra Trees      35.184788     45.760511     20.213008      8.793385              24
    PM2.5 Extra Trees      11.316424     11.924355     11.006224      0.237918              24
```




NO2: Historical and 24-Hour Forecast
Historical observations
24-hour forecast
Forecast origin
NO2
Date and time
0
20
40
60
80
100
08-20 00
08-20 12
08-21 00
08-21 12
08-22 00
08-22 12
08-23 00
08-23 12
08-24 00

```
All forecast files saved to: D:\Auckland_Air_Quality_MVP
```

24-HOUR WALK-FORWARD BACKTEST

In [36]:

```python
backtest_rows = []

for target, obj in model_objects.items():

    table = obj["model_table"].sort_values("datetime").copy()
    features = obj["features"]

    # Use the selected model family.
    prototype = build_models()[obj["model_name"]]

    # Candidate origins are spaced seven days apart to reduce
    # overlap between forecast windows.
    first_origin = max(72, int(len(table) * 0.70))
    last_origin = len(table) - 24

    origins = list(range(first_origin, last_origin, 168))

    for origin in origins:

        train = table.iloc[:origin].copy()
        future = table.iloc[origin:origin + 24].copy()

        if len(future) != 24:
            continue

        X_train = train[features]
        y_train = train[target]

        model = clone(prototype)
        model.fit(X_train, y_train)

        # Forecast using recursive target lags and only
        # information available at the forecast origin.
        origin_time = table.iloc[origin - 1]["datetime"]

        raw_until_origin = data[
            data["datetime"] <= origin_time
        ].copy()

        if raw_until_origin[target].notna().sum() < 73:
            continue

        recursive = forecast_next_24_hours(
            raw_until_origin,
            target,
            model,
            features
        )

        # Match each predicted timestamp with the observed value.
        actual = future[
            ["datetime", target]
        ].rename(columns={target: "Actual"})
```

```
        matched = actual.merge(
            recursive[["datetime", "Forecast"]],
            on="datetime",
            how="inner"
        ).dropna()

        if len(matched) != 24:
            # Missing actual observations mean this origin is
            # not a complete 24-hour evaluation window.
            continue

        metrics = calculate_metrics(
            matched["Actual"],
            matched["Forecast"]
        )

        backtest_rows.append({
            "Pollutant": target,
            "Model": obj["model_name"],
            "Origin": origin_time,
            "Horizon_Hours": 24,
            "Evaluated_Hours": len(matched),
            **metrics
        })

backtest_24h = pd.DataFrame(backtest_rows)

backtest_24h.to_csv(
    os.path.join(OUTPUT_DIR, "24h_walk_forward_backtest.csv"),
    index=False
)

if not backtest_24h.empty:
    print("\n24-HOUR WALK-FORWARD BACKTEST")
    print(backtest_24h.to_string(index=False))

    backtest_summary = (
        backtest_24h.groupby(
            ["Pollutant", "Model"], as_index=False
        )
        [["MAE", "RMSE", "R2"]]
        .mean()
    )

    backtest_summary.to_csv(
        os.path.join(OUTPUT_DIR, "24h_backtest_summary.csv"),
        index=False
    )
else:
    print(
        "No complete 24-hour windows could be evaluated. "
        "Check target coverage and historical feature availability."
    )
```

```
24-HOUR WALK-FORWARD BACKTEST
Pollutant       Model              Origin  Horizon_Hours  Evaluated_Hours      MAE     RMSE         R2
    PM2.5 Extra Trees 2022-06-26 13:00:00             24               24 2.807325 3.989304  -0.519299
    PM2.5 Extra Trees 2022-07-11 18:00:00             24               24 2.385235 3.013377   0.088741
    PM2.5 Extra Trees 2022-07-21 07:00:00             24               24 1.369753 1.939413   0.271412
    PM2.5 Extra Trees 2022-07-28 07:00:00             24               24 5.899589 6.612934  -2.436955
    PM2.5 Extra Trees 2022-08-04 19:00:00             24               24 4.063522 5.844083  -0.210621
    PM2.5 Extra Trees 2022-08-11 19:00:00             24               24 1.973482 2.390060  -1.302092
    PM2.5 Extra Trees 2022-08-23 17:00:00             24               24 2.414092 2.615510  -0.115628
    PM2.5 Extra Trees 2022-08-30 17:00:00             24               24 2.716295 3.077675 -11.062345
    PM2.5 Extra Trees 2022-09-06 17:00:00             24               24 3.257185 3.853710  -0.096836
    PM2.5 Extra Trees 2022-09-13 17:00:00             24               24 2.176484 2.925468  -0.731513
    PM2.5 Extra Trees 2022-09-29 20:00:00             24               24 2.782460 3.205184  -0.660316
    PM2.5 Extra Trees 2022-10-06 20:00:00             24               24 2.274464 2.604721  -3.657823
    PM2.5 Extra Trees 2022-10-21 03:00:00             24               24 2.580356 3.109768   0.100259
    PM2.5 Extra Trees 2022-10-28 14:00:00             24               24 4.337173 5.199281  -1.404375
    PM2.5 Extra Trees 2022-11-04 14:00:00             24               24 1.980509 2.134847  -3.001769
    PM2.5 Extra Trees 2022-11-20 02:00:00             24               24 1.315172 2.007344  -0.126676
    PM2.5 Extra Trees 2022-11-27 02:00:00             24               24 4.011494 4.367843  -4.541583
    PM2.5 Extra Trees 2022-12-04 02:00:00             24               24 1.576746 1.916636  -1.769546
    PM2.5 Extra Trees 2022-12-23 02:00:00             24               24 1.155178 1.630536   0.230848
    PM2.5 Extra Trees 2022-12-30 02:00:00             24               24 2.037920 3.301991  -0.530639
    PM2.5 Extra Trees 2023-01-22 01:00:00             24               24 0.882008 1.130714  -0.097502
    PM2.5 Extra Trees 2023-02-05 07:00:00             24               24 1.968223 2.256197  -0.861642
    PM2.5 Extra Trees 2023-02-12 07:00:00             24               24 1.884956 2.241157  -0.587012
    PM2.5 Extra Trees 2023-02-19 07:00:00             24               24 1.878952 2.138086  -1.493496
```

SAVE FORECAST

In [38]:

```python
forecast_24.to_csv(
    os.path.join(
        OUTPUT_DIR,
        "24_hour_air_quality_forecast.csv"
    ),
    index=False
)

forecast_24.to_excel(
    os.path.join(
        OUTPUT_DIR,
        "24_hour_air_quality_forecast.xlsx"
    ),
    index=False
)
```

FORECAST VISUALISATION

In [39]:
```python
for target in available_targets:

    # Last 48 hours of historical observations
    historical = (
        data[
            ["datetime", target]
        ]
        .dropna()
        .tail(48)
    )

    future = forecast_24[
        ["datetime", target]
    ].dropna()

    plt.figure(figsize=(15, 6))

    # Historical values
    plt.plot(
        historical["datetime"],
        historical[target],
        marker="o",
        markersize=4,
        linewidth=2,
        label="Historical"
    )

    # Forecast values
    plt.plot(
        future["datetime"],
        future[target],
        marker="o",
        markersize=5,
        linewidth=2.5,
        linestyle="--",
        label="24-hour forecast"
    )

    # Forecast boundary
    plt.axvline(
        future["datetime"].iloc[0],
        linestyle=":",
        linewidth=2,
        label="Forecast begins"
    )

    plt.title(
        f"{target}: Historical Values and Next 24-Hour Forecast",
        fontsize=15,
        fontweight="bold"
    )

    plt.xlabel("Date and Time")
    plt.ylabel(target)

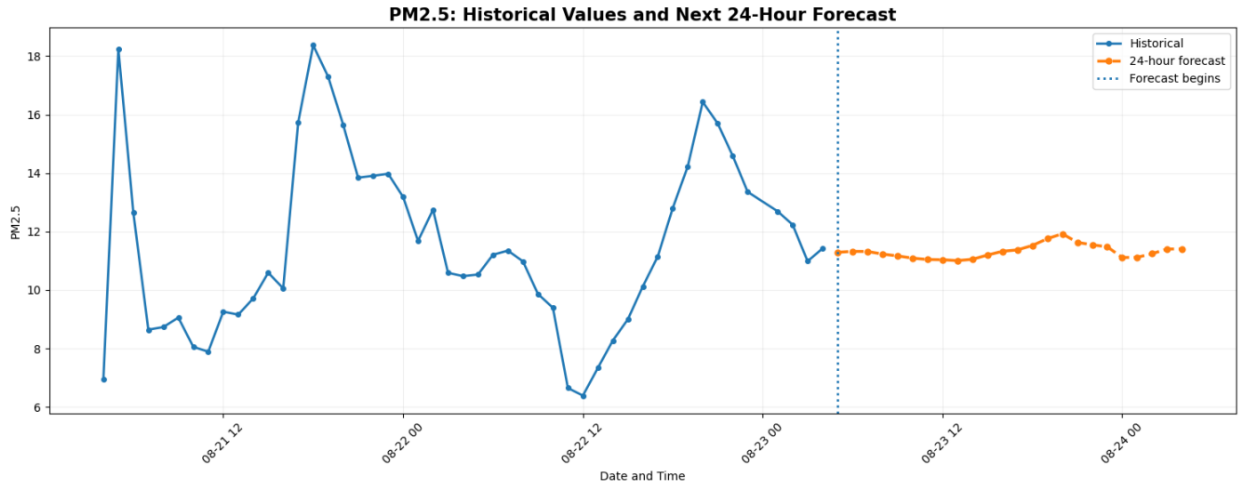
    plt.xticks(rotation=45)

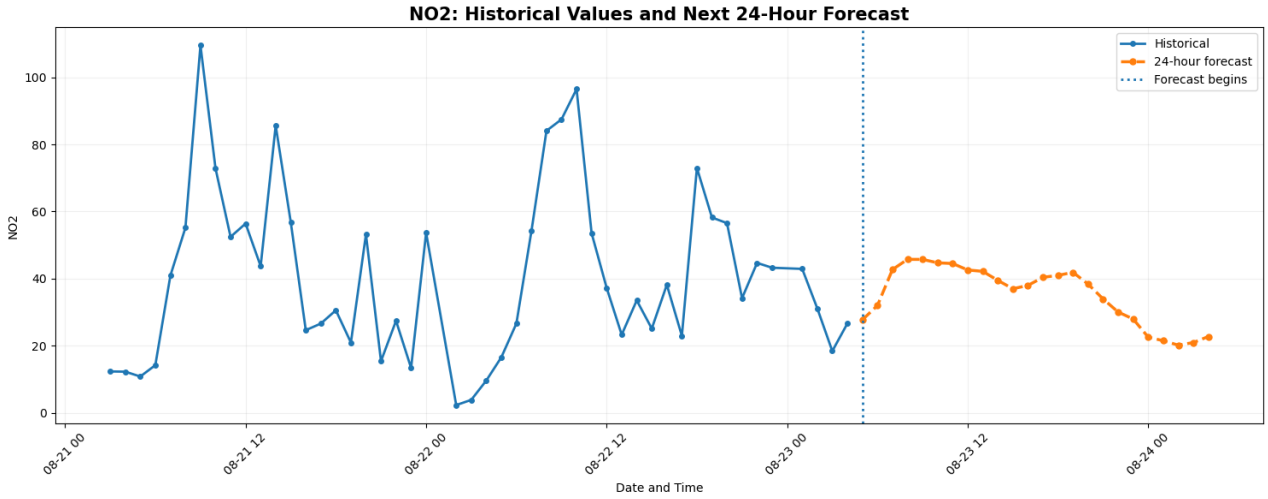
    plt.legend()
    plt.grid(alpha=0.2)
```

```
plt.tight_layout()

plt.savefig(
    os.path.join(
        OUTPUT_DIR,
        f"{target}_24_hour_forecast_enhanced.png"
    ),
    dpi=300,
    bbox_inches="tight"
)

plt.show()
```





24-HOUR FORECAST DECISION DASHBOARD

In [40]:
```
fig, axes = plt.subplots(
    len(available_targets),
    1,
    figsize=(15, 5 * len(available_targets))
)

if len(available_targets) == 1:
    axes = [axes]

for ax, target in zip(axes, available_targets):

    forecast_values = forecast_24[target]

    ax.plot(
        forecast_24["datetime"],
        forecast_values,
        marker="o",
        linewidth=2.5,
        markersize=5
    )

    # Mean forecast line
    mean_value = forecast_values.mean()

    ax.axhline(
        mean_value,
        linestyle="--",
        linewidth=1.5,
        label=f"24-hour mean = {mean_value:.2f}"
    )

    # Mark maximum forecast
    max_index = forecast_values.idxmax()

    ax.scatter(
        forecast_24.loc[max_index, "datetime"],
        forecast_values.loc[max_index],
        s=100,
        zorder=5,
        label="Forecast maximum"
    )

    ax.annotate(
        f"Max: {forecast_values.loc[max_index]:.2f}",
        (
            forecast_24.loc[max_index, "datetime"],
            forecast_values.loc[max_index]
        ),
        xytext=(10, 10),
        textcoords="offset points"
    )

    ax.set_title(
        f"{target} — Next 24 Hours",
        fontsize=14,
        fontweight="bold"
    )

    ax.set_ylabel(target)
    ax.grid(alpha=0.2)
    ax.legend()
```

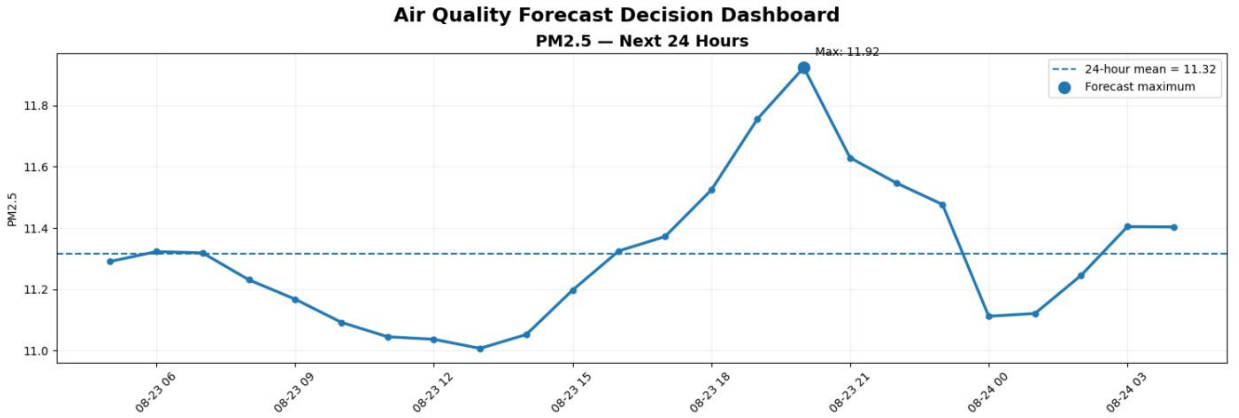
```
    ax.tick_params(axis="x", rotation=45)

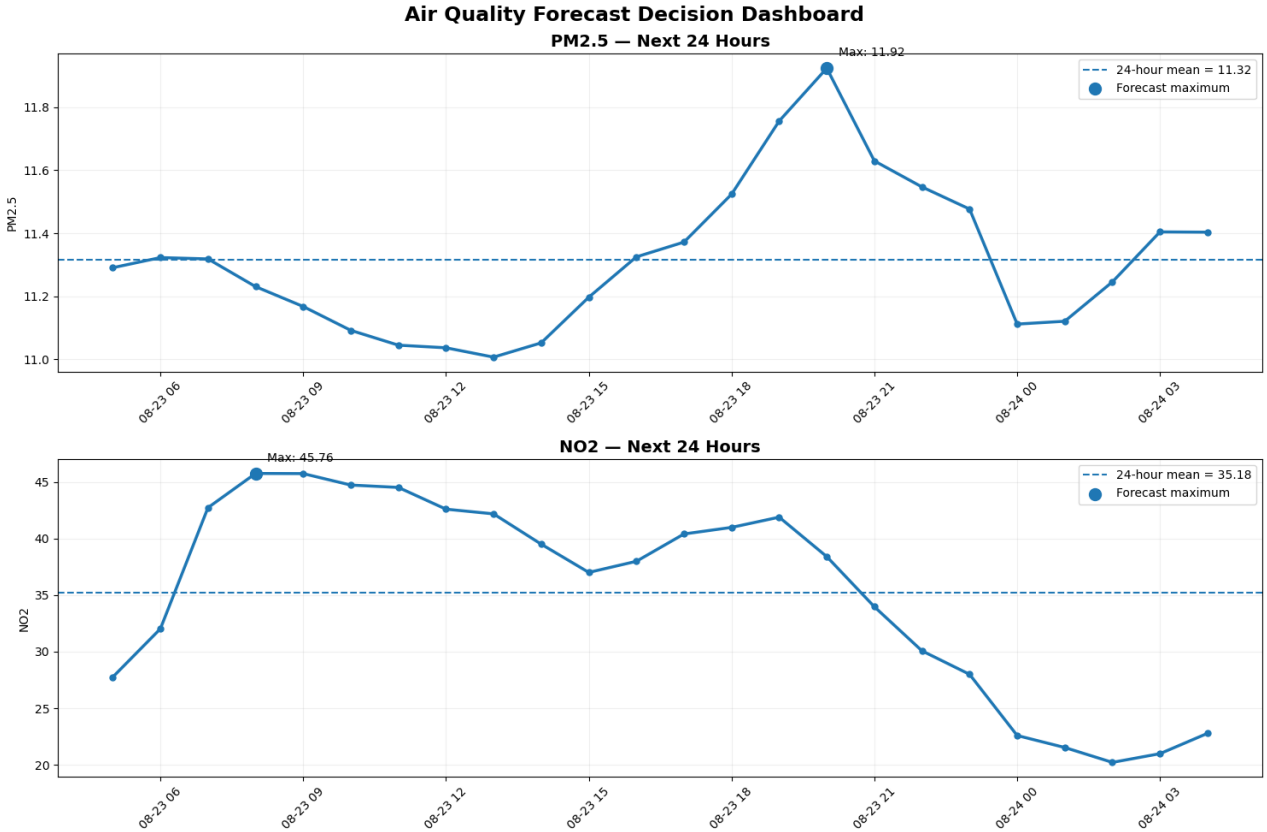
plt.suptitle(
    "Air Quality Forecast Decision Dashboard",
    fontsize=17,
    fontweight="bold"
)

plt.tight_layout()

plt.savefig(
    os.path.join(
        OUTPUT_DIR,
        "24_hour_forecast_decision_dashboard.png"
    ),
    dpi=300,
    bbox_inches="tight"
)

plt.show()
```





CORRELATION ANALYSIS

In [41]:

```
correlation = data.select_dtypes(
    include=np.number
).corr()

print("\nCorrelation matrix:")
print(correlation)

correlation.to_csv(
    os.path.join(
        OUTPUT_DIR,
        "correlation_matrix.csv"
    )
)


# CORRELATION HEATMAP

plt.figure(figsize=(15, 12))

plt.imshow(
    correlation,
    aspect="auto",
    interpolation="nearest"
)

plt.colorbar(
    label="Correlation coefficient"
)

plt.xticks(
    range(len(correlation.columns)),
    correlation.columns,
    rotation=90
)

plt.yticks(
    range(len(correlation.columns)),
    correlation.columns
)

plt.title(
    "Correlation Structure of Traffic, Weather and Air Quality Variables",
    fontsize=15,
    fontweight="bold"
)

plt.tight_layout()

plt.savefig(
    os.path.join(
        OUTPUT_DIR,
        "correlation_heatmap.png"
    ),
    dpi=300,
    bbox_inches="tight"
)
```

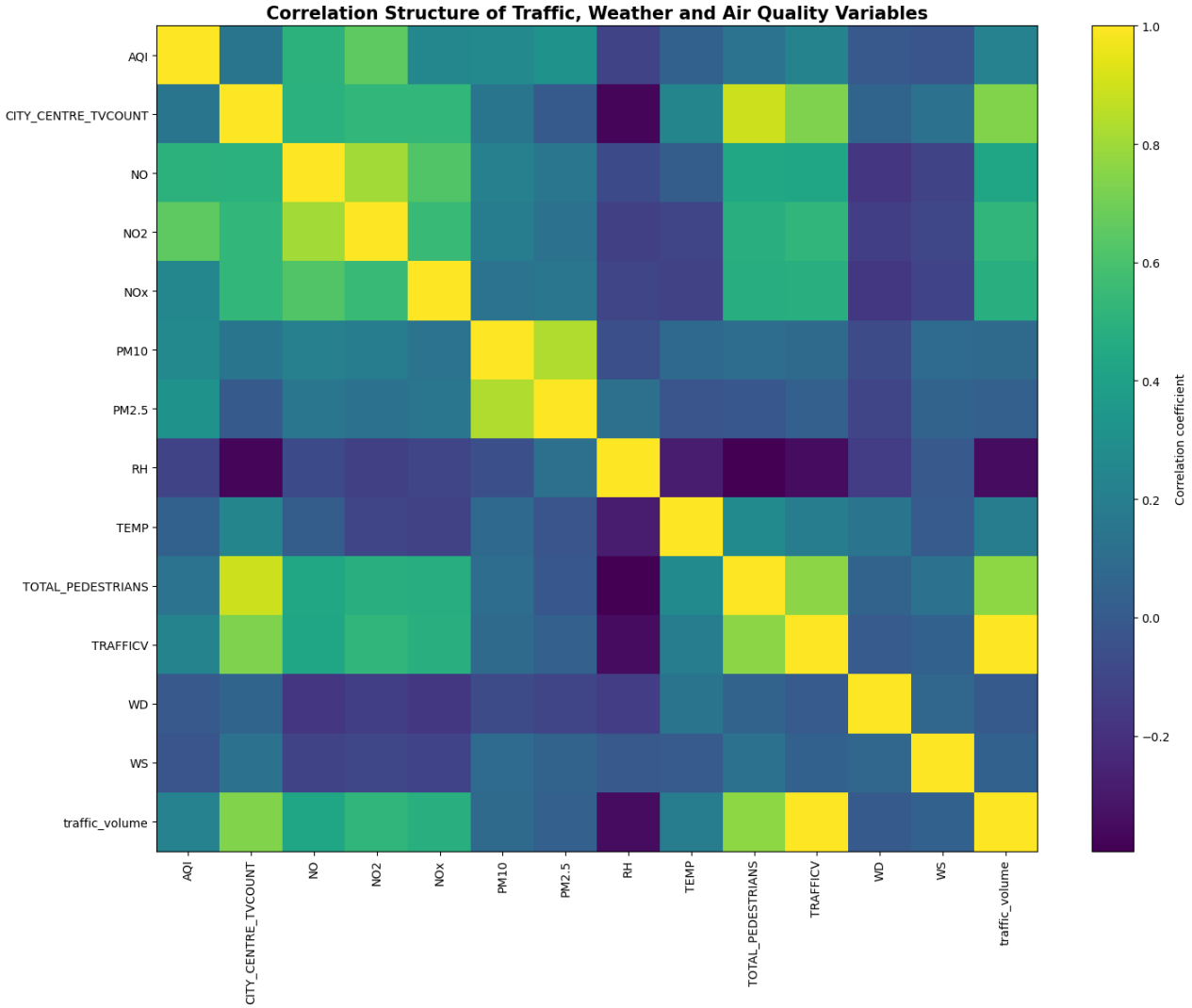
```
plt.show()
```

```
Correlation matrix:
                         AQI  CITY_CENTRE_TVCOUNT        NO       NO2  \
AQI                 1.000000             0.147711  0.501967  0.655124
CITY_CENTRE_TVCOUNT 0.147711             1.000000  0.495974  0.527074
NO                  0.501967             0.495974  1.000000  0.805883
NO2                 0.655124             0.527074  0.805883  1.000000
NOx                 0.252035             0.527761  0.622812  0.549852
PM10                0.259823             0.149382  0.215480  0.193442
PM2.5               0.313627             0.001597  0.152958  0.126960
RH                 -0.108885            -0.370731 -0.083509 -0.129975
TEMP                0.039956             0.239408  0.017990 -0.105006
TOTAL_PEDESTRIANS   0.138002             0.894863  0.435060  0.487276
TRAFFICV            0.229438             0.731762  0.428769  0.522105
WD                 -0.002961             0.055066 -0.172695 -0.134448
WS                 -0.026338             0.125728 -0.111132 -0.099635
traffic_volume      0.222811             0.734705  0.431860  0.523750

                         NOx      PM10     PM2.5        RH      TEMP  \
AQI                 0.252035  0.259823  0.313627 -0.108885  0.039956
CITY_CENTRE_TVCOUNT 0.527761  0.149382  0.001597 -0.370731  0.239408
NO                  0.622812  0.215480  0.152958 -0.083509  0.017990
NO2                 0.549852  0.193442  0.126960 -0.129975 -0.105006
NOx                 1.000000  0.138105  0.152454 -0.104392 -0.122088
PM10                0.138105  1.000000  0.833254 -0.052873  0.082482
PM2.5               0.152454  0.833254  1.000000  0.117993 -0.026488
RH                 -0.104392 -0.052873  0.117993  1.000000 -0.287520
TEMP               -0.122088  0.082482 -0.026488 -0.287520  1.000000
TOTAL_PEDESTRIANS   0.472073  0.104579 -0.016583 -0.394582  0.268174
TRAFFICV            0.477539  0.084969  0.032525 -0.350584  0.192925
WD                 -0.166609 -0.076483 -0.105209 -0.139511  0.140838
WS                 -0.111439  0.093354  0.051974 -0.007080  0.003678
traffic_volume      0.480183  0.086146  0.033371 -0.346046  0.195735

                    TOTAL_PEDESTRIANS  TRAFFICV        WD        WS  \
AQI                          0.138002  0.229438 -0.002961 -0.026338
CITY_CENTRE_TVCOUNT          0.894863  0.731762  0.055066  0.125728
NO                           0.435060  0.428769 -0.172695 -0.111132
NO2                          0.487276  0.522105 -0.134448 -0.099635
NOx                          0.472073  0.477539 -0.166609 -0.111439
PM10                         0.104579  0.084969 -0.076483  0.093354
PM2.5                       -0.016583  0.032525 -0.105209  0.051974
RH                          -0.394582 -0.350584 -0.139511 -0.007080
TEMP                         0.268174  0.192925  0.140838  0.003678
TOTAL_PEDESTRIANS            1.000000  0.759693  0.049020  0.123375
TRAFFICV                     0.759693  1.000000  0.004132  0.040930
WD                           0.049020  0.004132  1.000000  0.067019
WS                           0.123375  0.040930  0.067019  1.000000
traffic_volume               0.761994  0.999842  0.002726  0.037286
```

	traffic_volume
AQI	0.222811
CITY_CENTRE_TVCOUNT	0.734705
NO	0.431860
NO2	0.523750
NOx	0.480183
PM10	0.086146
PM2.5	0.033371
RH	-0.346046
TEMP	0.195735
TOTAL_PEDESTRIANS	0.761994
TRAFFICV	0.999842
WD	0.002726
WS	0.037286
traffic_volume	1.000000



TRAFFIC VS POLLUTION

In [42]:
```
for target in available_targets:

    if "traffic_volume" in data.columns:

        relationship = data[
            ["traffic_volume", target]
        ].dropna()

        # Reduce points for clearer visualisation
        if len(relationship) > 5000:
            relationship = relationship.sample(
                5000,
                random_state=42
            )

        plt.figure(figsize=(10, 6))

        plt.scatter(
            relationship["traffic_volume"],
            relationship[target],
            alpha=0.35,
            s=20
        )

        plt.title(
            f"Traffic Volume vs {target}",
            fontsize=15,
            fontweight="bold"
        )

        plt.xlabel(
            "Hourly Traffic Volume"
        )

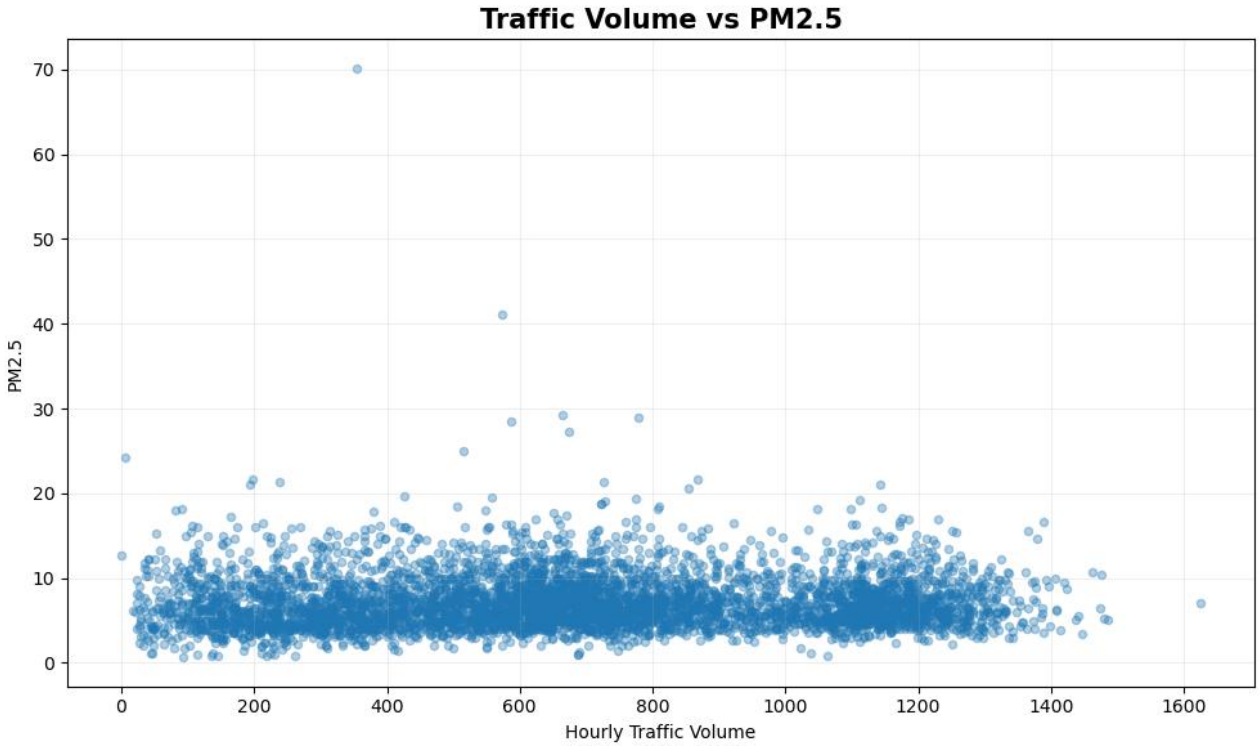
        plt.ylabel(
            target
        )

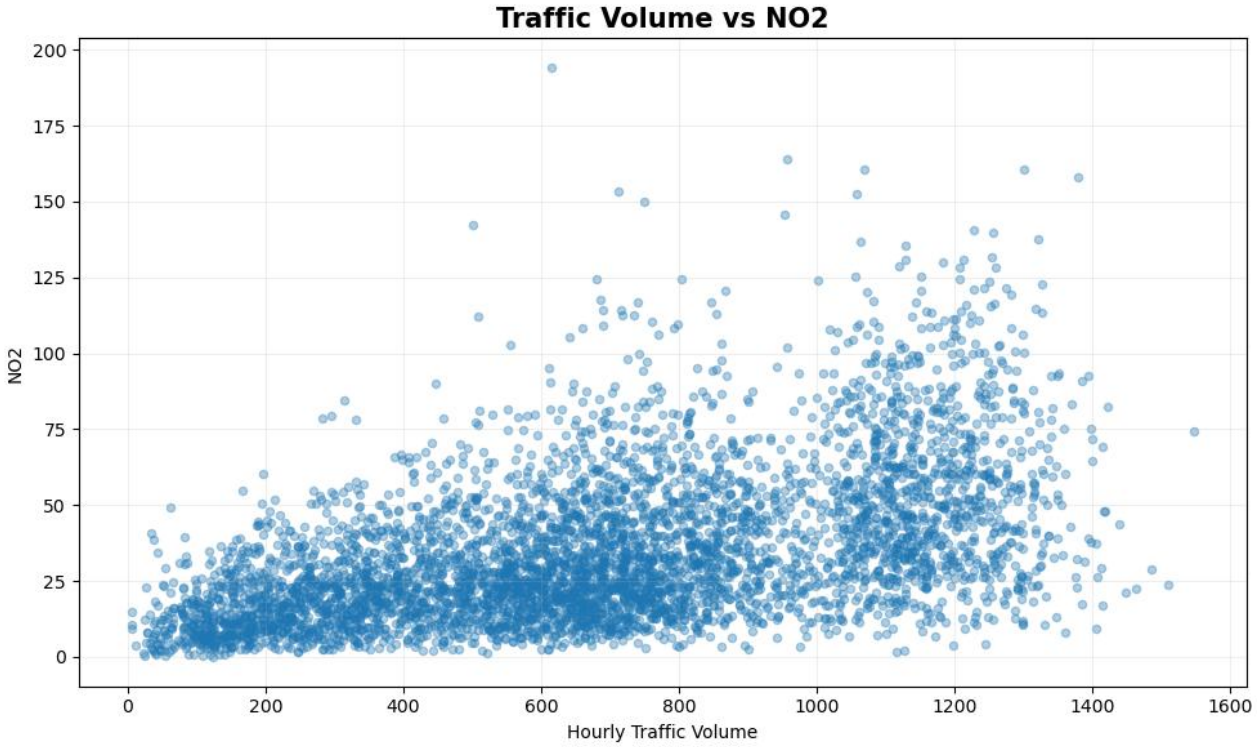
        plt.grid(alpha=0.2)

        plt.tight_layout()

        plt.savefig(
            os.path.join(
                OUTPUT_DIR,
                f"traffic_vs_{target}.png"
            ),
            dpi=300,
            bbox_inches="tight"
        )

        plt.show()
```


Traffic Volume vs PM2.5
PM2.5
Hourly Traffic Volume
0
10
20
30
40
50
60
70
0
200
400
600
800
1000
1200
1400
1600


Traffic Volume vs NO2
NO2
Hourly Traffic Volume
0
25
50
75
100
125
150
175
200
0
200
400
600
800
1000
1200
1400
1600

AIR QUALITY SUMMARY FOR DECISION MAKING

In [43]:
```
summary = []

for target in available_targets:

    summary.append({
        "Pollutant": target,
        "Mean": data[target].mean(),
        "Minimum": data[target].min(),
        "Maximum": data[target].max(),
        "Std_Dev": data[target].std(),
        "Latest_Value": data[target].iloc[-1],
        "Forecast_24h_Mean":
            forecast_24[target].mean(),
        "Forecast_24h_Max":
            forecast_24[target].max()
    })

summary_df = pd.DataFrame(summary)

print("\n==============================")
print("AIR QUALITY DECISION SUMMARY")
print("==============================")

print(summary_df)

summary_df.to_csv(
    os.path.join(
        OUTPUT_DIR,
        "air_quality_summary.csv"
    ),
    index=False
)
```

```
==============================
AIR QUALITY DECISION SUMMARY
==============================
  Pollutant       Mean  Minimum   Maximum    Std_Dev  Latest_Value  \
0     PM2.5   7.360742      0.0  209.4000   3.685793       11.4247
1       NO2  40.822379    -17.8  540.9625  25.687235       26.6432

   Forecast_24h_Mean  Forecast_24h_Max
0          11.316424         11.924355
1          35.184788         45.760511
```

CREATE MASTER OUTPUT FILE

In [44]:
```
with pd.ExcelWriter(
    os.path.join(
        OUTPUT_DIR,
        "Auckland_Air_Quality_MVP_Results.xlsx"
    )
) as writer:

    data.to_excel(
        writer,
        sheet_name="Clean_Data",
        index=False
    )

    model_results.to_excel(
        writer,
        sheet_name="Model_Results",
        index=False
    )

    forecast_24.to_excel(
        writer,
        sheet_name="24H_Forecast",
        index=False
    )

    summary_df.to_excel(
        writer,
        sheet_name="Decision_Summary",
        index=False
    )

    correlation.to_excel(
        writer,
        sheet_name="Correlation"
    )
```

EXECUTIVE AIR QUALITY SUMMARY DASHBOARD

In [45]:
```
fig, axes = plt.subplots(
    1,
    3,
    figsize=(18, 6)
)


# PANEL 1: CURRENT VALUES

x = np.arange(len(summary_df))

axes[0].bar(
    x,
    summary_df["Latest_Value"]
)

axes[0].set_xticks(x)
axes[0].set_xticklabels(
    summary_df["Pollutant"]
)

axes[0].set_title(
    "Latest Observed Values",
    fontweight="bold"
)

axes[0].set_ylabel(
    "Concentration"
)

axes[0].grid(
    axis="y",
    alpha=0.2
)


# PANEL 2: 24-HOUR FORECAST MEAN

axes[1].bar(
    x,
    summary_df["Forecast_24h_Mean"]
)

axes[1].set_xticks(x)
axes[1].set_xticklabels(
    summary_df["Pollutant"]
)

axes[1].set_title(
    "24-Hour Forecast Mean",
    fontweight="bold"
)

axes[1].set_ylabel(
    "Forecast concentration"
)
```

```
axes[1].grid(
    axis="y",
    alpha=0.2
)


# PANEL 3: FORECAST MAXIMUM

axes[2].bar(
    x,
    summary_df["Forecast_24h_Max"]
)

axes[2].set_xticks(x)
axes[2].set_xticklabels(
    summary_df["Pollutant"]
)

axes[2].set_title(
    "24-Hour Forecast Maximum",
    fontweight="bold"
)

axes[2].set_ylabel(
    "Maximum forecast"
)

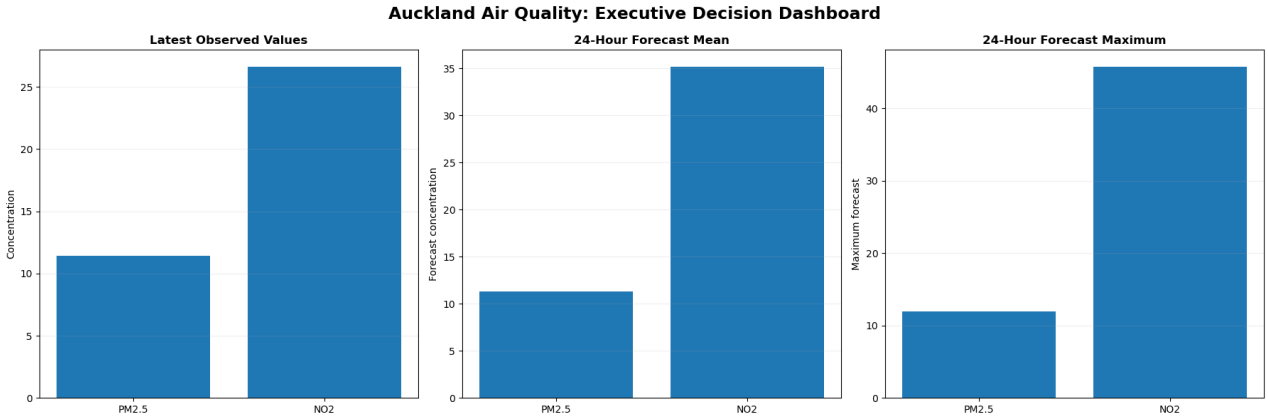
axes[2].grid(
    axis="y",
    alpha=0.2
)


plt.suptitle(
    "Auckland Air Quality: Executive Decision Dashboard",
    fontsize=17,
    fontweight="bold"
)

plt.tight_layout()

plt.savefig(
    os.path.join(
        OUTPUT_DIR,
        "executive_air_quality_dashboard.png"
    ),
    dpi=300,
    bbox_inches="tight"
)

plt.show()
```


Auckland Air Quality: Executive Decision Dashboard
Latest Observed Values
Concentration
0
5
10
15
20
25
PM2.5
NO2
24-Hour Forecast Mean
Forecast concentration
0
5
10
15
20
25
30
35
PM2.5
NO2
24-Hour Forecast Maximum
Maximum forecast
0
10
20
30
40
PM2.5
NO2

FINAL OUTPUT

In [46]:
```
print("\n==========================================")
print("MVP ANALYSIS COMPLETED SUCCESSFULLY")
print("==========================================")

print(f"\nResults saved to:")
print(OUTPUT_DIR)

print("\nFiles generated include:")

print("\nDATA OUTPUTS")
print("- model_comparison.csv")
print("- 24_hour_air_quality_forecast.csv")
print("- 24_hour_air_quality_forecast.xlsx")
print("- Auckland_Air_Quality_MVP_Results.xlsx")
print("- air_quality_summary.csv")
print("- correlation_matrix.csv")

print("\nVISUAL OUTPUTS")
print("- dataset_overview_dashboard.png")
print("- pollutant_distribution_boxplots.png")
print("- enhanced pollutant time-series charts")
print("- enhanced hourly pattern charts")
print("- hour-day pollution heatmaps")
print("- model_performance_comparison.png")
print("- actual_vs_predicted charts")
print("- feature importance charts")
print("- top feature contribution charts")
print("- enhanced 24-hour forecast charts")
print("- 24-hour forecast decision dashboard")
print("- correlation_heatmap.png")
print("- traffic vs pollution charts")
print("- executive_air_quality_dashboard.png")

if SHAP_AVAILABLE:
    print("- SHAP explainability charts")
else:
    print(
        "\nNote: SHAP visualisations were not generated "
        "because the SHAP package is not installed."
    )
```

```
==========================================
MVP ANALYSIS COMPLETED SUCCESSFULLY
==========================================

Results saved to:
D:\Auckland_Air_Quality_MVP

Files generated include:

DATA OUTPUTS
- model_comparison.csv
- 24_hour_air_quality_forecast.csv
- 24_hour_air_quality_forecast.xlsx
- Auckland_Air_Quality_MVP_Results.xlsx
- air_quality_summary.csv
- correlation_matrix.csv
```

```
VISUAL OUTPUTS
- dataset_overview_dashboard.png
- pollutant_distribution_boxplots.png
- enhanced pollutant time-series charts
- enhanced hourly pattern charts
- hour-day pollution heatmaps
- model_performance_comparison.png
- actual_vs_predicted charts
- feature importance charts
- top feature contribution charts
- enhanced 24-hour forecast charts
- 24-hour forecast decision dashboard
- correlation_heatmap.png
- traffic vs pollution charts
- executive_air_quality_dashboard.png
- SHAP explainability charts
```

POWER BI DATA EXPORTS

In [47]:

```python
# 1. Historical observations for date and pollutant filtering
historical_export = data.copy()
historical_export["Date"] = historical_export["datetime"].dt.date
historical_export["Year"] = historical_export["datetime"].dt.year
historical_export["Month"] = historical_export["datetime"].dt.month

historical_export.to_csv(
    os.path.join(OUTPUT_DIR, "powerbi_historical_data.csv"),
    index=False
)

# 2. Model performance: all six models, both pollutants
model_results.to_csv(
    os.path.join(OUTPUT_DIR, "powerbi_model_metrics.csv"),
    index=False
)

# 3. Actual versus predicted values for interactive charts
prediction_results.to_csv(
    os.path.join(OUTPUT_DIR, "powerbi_test_predictions.csv"),
    index=False
)

# 4. Walk-forward validation fold results
walk_forward_results.to_csv(
    os.path.join(OUTPUT_DIR, "powerbi_walk_forward.csv"),
    index=False
)

# 5. Long-format 24-hour forecast table
forecast_long.to_csv(
    os.path.join(OUTPUT_DIR, "powerbi_forecast.csv"),
    index=False
)

# 6. Forecast summary
forecast_summary.to_csv(
    os.path.join(OUTPUT_DIR, "powerbi_forecast_summary.csv"),
    index=False
)
```

```python
# 7. Optional 24-hour backtest results
if "backtest_24h" in globals():
    backtest_24h.to_csv(
        os.path.join(OUTPUT_DIR, "powerbi_24h_backtest.csv"),
        index=False
    )

print("Power BI data exports completed:")
for filename in [
    "powerbi_historical_data.csv",
    "powerbi_model_metrics.csv",
    "powerbi_test_predictions.csv",
    "powerbi_walk_forward.csv",
    "powerbi_forecast.csv",
    "powerbi_forecast_summary.csv"
]:
    print(os.path.join(OUTPUT_DIR, filename))
```

```
Power BI data exports completed:
D:\Auckland_Air_Quality_MVP\powerbi_historical_data.csv
D:\Auckland_Air_Quality_MVP\powerbi_model_metrics.csv
D:\Auckland_Air_Quality_MVP\powerbi_test_predictions.csv
D:\Auckland_Air_Quality_MVP\powerbi_walk_forward.csv
D:\Auckland_Air_Quality_MVP\powerbi_forecast.csv
D:\Auckland_Air_Quality_MVP\powerbi_forecast_summary.csv
```
